



NetSentinel

שם התלמיד: נועם קמרי 217604834

שם המנחה: גל בר-און

שם בית הספר: מקיף ה' דרכא



תוכן עניינים

2	תוכן עניינים
5	מבוא
5	ייזום:
5	תיאור ראשוני של המערכת
5	למה בחרתי בפרויקט
5	הגדרת הלקוח
5	הגדרת יעדים ומטרות
6	בעיות ותועלות
6	סקירת פתרונות קיימים
7	סקירת טכנולוגית הפרויקט
7	תיחום הפרויקט
8	אפיון
8	תיאור מפורט של המערכת
8	פירוט יכולות לפי סוג משתמש
8	פירוט הבדיקות המתוכננות
11	ניהול סיכונים בפרויקט
12	תיאור תחום הידע - פרק מילולי
12	פירוט היכולות בצד לקוח (סוכן הקצה)
15	פירוט היכולות בצד שרת (הניהול המרכזי)
17	מבנה הפרויקט
17	תיאור החומרה – רכיבים שונים והקשרים ביניהם
18	תיאור הטכנולוגיה במערכת
18	שפת תכנות וסביבת פיתוח:
19	החלטות ארכיטקטוניות
19	למה TCP ולא UDP
19	Scalability
19	צווארי בקבוק
20	תיאור זרימת המידע במערכת
22	תרשים UML
23	Sequence Diagram
24	תיאור האלגוריתמים המרכזיים בפרויקט
24	ניסוח וניתוח של הבעיה האלגוריתמית
24	תיאור אלגוריתמים קיימים לפתרון הבעיה
25	סקירת הפתרון הנבחר ונימוק הבחירה
25	ניתוח ביצועים
26	תיאור סביבת הפיתוח
26	פירוט כלי הפיתוח הדרושים לפיתוח המערכת:

26	פירוט הסביבה והכלים הנדרשים לבדיקות המערכת:
27	תיאור פרוטוקול התקשורת
27	תיאור מילולי של פרוטוקול התקשורת
28	פירוט כלל ההודעות הזורמות במערכת
29	תיאור מסכי המערכת
29	מסכי אפליקציית סוכן הקצה (Agent)
30	מסכי אפליקציית השרת המרכזי (Central Server)
30	תרשים מסכים והמעברים ביניהם (Screen Flow Diagram)
32	תיאור מבני הנתונים במערכת
32	מבני נתונים מקומיים בזיכרון המערכת
32	1. מילון ניהול סוכנים בשרת (ConcurrentDictionary)
33	2. תור פאקטות נכנסות בסוכן (ConcurrentQueue)
34	3. מילון מונים לזיהוי אנומליות (Dictionary)
35	אחסון מבוסס קבצים (קובץ PCAP)
35	1. כותרת גלובלית (Global Header)
36	2. כותרת חבילה (Packet Header)
37	סקירת חולשות ואיומים במערכת
37	שכבת האפליקציה
38	שכבת התעבורה
38	הפעלת המערכת
39	מימוש הפרויקט
39	ארגון הקבצים
40	סקירת מודולים ומחלקות
40	מודולים ומחלקות מיובאים (ספריות חיצוניות)
41	טבלת ריכוז מחלקות הפרויקט
42	פירוט המחלקות - צד לקוח (מחשב קצה)
42	המחלקה: PcapHandler
43	המחלקה: PacketQueue
44	המחלקה: PacketScanner
47	המחלקה: AnomalyBlocker
49	המחלקה: Statistics
51	המחלקה: ProtocolBarsVM
52	המחלקה: ProtocolInfo
53	המחלקה: Scanner
55	המחלקה: ChooseDevice
56	המחלקה: CaptureInterfaceItem
57	פירוט המחלקות - צד שרת (מחשב ראשי)
57	המחלקה: MainWindow (בצד השרת)
59	המחלקה: ConnectedAgent
60	המחלקה: ProtocolStat

61	פירוט בעיות אלגוריתמיות ומנגנונים מיוחדים
61	1. אלגוריתם ניהול, סנכרון ועיבוד תור חבילות מקבילי ((Concurrent Packet Queue
61	א. הסבר על היכולת והאתגר האלגוריתמי
63	2. אלגוריתם זיהוי וחילוץ פרוטוקולים בשכבת האפליקציה ((Parsing
66	מסמך בדיקות
67	מדריך למשתמש
67	עץ קבצים עיקריים
68	התקנת המערכת
68	שלב א': התקנת מנהל התקן ללכידת חבילות רשת ((Npcap
68	שלב ב': התקנת סביבת הריצה של NET 8.0
69	סוגי משתמשים ואופן הפעלת המערכת
69	הגדרת משתמשי המערכת והרשאות נדרשות
69	מדריך הפעלה מלא שלב אחר שלב
69	שלב 1: הפעלת השרת המרכזי
70	שלב 2: הפעלת הסוכן ובחירת כרטיס רשת
71	שלב 3: מסך הניטור הראשי והגרפים
72	סיכום אישי ורפלקציה על תהליך הפיתוח
72	תיאור תהליך העבודה, האתגרים ודרכי הפתרון
72	תהליך הלמידה – תובנות, טכנולוגיות וכלים חדשים
73	3. ראייה לאחור, חקר עצמי וסיכום ((Reflection
74	ביבליוגרפיה
74	ספריות קוד פתוח
74	תיעוד רשמי של Microsoft
76	נספחים
77	קבצי קוד - צד לקוח NetworkScanner
77	PacketScanner.cs
85	Scanner.xaml.cs
97	Statistics.cs
101	AnomalyBlocker.cs
103	PacketQueue.cs
104	ProtocolBarsVM.c
108	PcapHandler.cs
115	ChooseDevice.xaml.cs
119	קבצי קוד - צד שרת ScannerServer
119	MainWindow.xaml.cs
123	ConnectedAgent.cs

מבוא

ייזום:

תיאור ראשוני של המערכת

NetSentinel היא מערכת מבוזרת לניטור תעבורת רשת ואיתור אנומליות, המיועדת להקנות למשתמש נראות ושליטה על המתרחש ברשת המקומית שלו. המערכת מורכבת מ"סוכנים" המותקנים על גבי תחנות קצה, ומבצעים הסנפה בזמן אמת של חבילות הרשת, ומשרת מרכזי המרכז את הנתונים, מנתח אנומליות ומציג ממשק ניהול אינטואיטיבי. המוצר המוגמר מאפשר למשתמש לראות התפלגות פרוטוקולים, לזהות התנהגויות חריגות ברשת (כגון עומסי התחברות חריגים או פקטות חריגות) ולנהל מדיניות אבטחה בסיסית באמצעות חסימת כתובות שמקורן אינו תקין.

למה בחרתי בפרויקט

בחרתי בפרויקט זה מתוך רצון להעמיק בתחום התקשורת ולהבין כיצד מידע עובר ברשת ברמה הנמוכה של הפאקטות, תמיד אהבתי את תחום הרשתות והוא עניין אותי יותר מתחומים אחרים. האתגרים המרכזיים שצפיתי בתחילת הדרך היו בעיקר סביב יעילות וביצועים, כמו איך לקרוא ולנתח אלפי פאקטות בשנייה בלי לתקוע את ממשק המשתמש, איך לזהות התנהגויות חריגות ברשת, ואיך לתרגם את כל המידע הזה לגרפים שיעזרו להבין את תמונת המצב ברשת.

הגדרת הלקוח

קהל היעד של המערכת כולל אנשי טכנולוגיה, מפתחי תוכנה, ואנשים באופן כללי המעוניינים בכלי לניתוח רשתות (Network Analysis) לצורכי אבחון, למידה או הגנה על סביבת העבודה האישית שלהם.

הגדרת יעדים ומטרות

המטרה המרכזית היא לפתח מערכת מבוזרת שיוודעת לנטר תעבורה בעצמה. ברמה התפעולית, המערכת שואפת להציג נתוני רשת בזמן אמת בעזרת גרפים וסטטיסטיקות שיעזרו למשתמש להבין מי מתקשר עם מי ברשת. מבחינת טיפול באנומליות, היעד הוא לזהות דפוסי תעבורה חריגים כמו SYN Flood או ARP Anomalies, ולאפשר למשתמש לבצע פעולות Traffic Filtering מיידיות כדי לשמור על הרצף התפקודי.

בעיות ותועלות

הבעיה שפרויקט זה בא לפתור היא חוסר הידיעה של המשתמש הממוצע לגבי המידע שיוצא ונכנס למחשב שלו, מה שמקשה על זיהוי פעילות חריגה ברשת. התועלת העיקרית של המערכת היא מתן שקיפות נוחה לרשת, כך שהמשתמש יכול לזהות בזמן אמת מקרים של חוסר עקביות או עומסים מחשידים, ולעצור אותם בזמן.

סקירת פתרונות קיימים

סקירת פתרונות קיימים

בשוק קיימים היום כלים מקצועיים רבים בתחום ניטור הרשת וזיהוי האנומליות, הנחלקים למספר קטגוריות עיקריות.

כלי ניתוח פאקטות

Wireshark הוא הכלי הנפוץ ביותר לניתוח עמוק של פאקטות. הוא מציג מידע מפורט מאוד על כל חבילה, אבל הוא פסיבי לחלוטין ולא מזהה אנומליות ולא מתריע עליהן. בנוסף הוא עובד על מחשב אחד בלבד ואין לו ניהול מרכזי. Nmap מיועד למיפוי רשתות וסריקת פורטים, אבל לא ללכידה ולניטור שוטף של תעבורה.

מערכות IDS

Snort ו-Suricata הן מערכות זיהוי חדירות שעובדות לפי חוקים מוגדרים מראש. הן יעילות מאוד בזיהוי מתקפות מוכרות, אבל דורשות תחזוקה שוטפת של מאגר החוקים ותצורה מורכבת. הן מיועדות לסביבות ארגוניות ודורשות ידע טכני מעמיק להתקנה ותפעול.

כלי ניתוח רשת NDR

Zeek מנתח תעבורת רשת ויוצר לוגים מפורטים, אבל הוא לא מתריע בעצמו ולא חוסם תעבורה. הוא משמש בעיקר כשכבת מידע שמזינה מערכות אחרות.

פלטפורמות SIEM

Security Onion היא פלטפורמה שמשלבת מספר כלים יחד כולל Zeek ו-Suricata, ומיועדת לניטור רשת ארגונית שלמה. ההתקנה והתפעול שלה מורכבים ודורשים משאבים רבים.

הערך המוסף של NetSentinel

בניגוד לכלים שתוארו, NetSentinel מציע גישה שונה שמתאימה למשתמש הפרטי או לרשת קטנה. הייחוד שלו הוא שילוב של מספר יכולות בכלי אחד קל לתפעול: לכידת פאקטות בזמן אמת, זיהוי

אנומליות אקטיבי, חסימה מיידית דרך Windows Firewall, וניהול מרכזי של מספר מחשבים במקביל ממסך אחד. כלים כמו Suricata או Security Onion מציעים יכולות עמוקות יותר, אבל מחייבים ידע טכני מקצועי ותשתית ארגונית. NetSentinel נועד להיות נגיש, פשוט להפעלה, ולתת נראות מיידית לכל מי שרוצה להבין מה קורה ברשת שלו.

סקירת טכנולוגית הפרויקט

הטכנולוגיות המרכזיות בהן נעשה שימוש בפרויקט אינן טכנולוגיות חדשות או בלתי מוכרות, אלא מבוססות על פרוטוקולי תקשורת סטנדרטיים וספריות מוכרות ללכידת פאקטות. עם זאת, קיימים מספר סייגים ודרישות מיוחדות להפעלת המערכת. הסייג המרכזי הוא הצורך בגישה ברמה נמוכה לכרטיס הרשת, מה שמחייב התקנה מוקדמת של מנהל התקן ייעודי (למשל Npcap) על עמדות הקצה. בנוסף, קיימת מגבלה הנובעת מדרישות האבטחה של מערכת ההפעלה, ולפיה הפעלת סוכן הקצה דורשת בהכרח הרשאות מנהל (Administrator) על מנת לאפשר האזנה לתעבורה נכנסת ויוצאת וביצוע פעולות סינון אקטיביות של כתובות.

תיחום הפרויקט

הפרויקט מתמקד בניטור שכבות התקשורת והרשת כדי לזהות חריגות תעבורה ברשת המקומית. עיקר העיסוק הוא ברמה של קריאת פאקטות, ניתוח כתובות, וזיהוי אנומליות בפרוטוקולים נפוצים בשכבות הנמוכות של מודל התקשורת. חשוב להדגיש שהמערכת אינה עוסקת בניתוח תוכן של מידע מוצפן ברמת האפליקציה, למשל פענוח תעבורת HTTPS, והיא אינה נועדה להחליף מערכות Firewall ארגוניות מורכבות או תוכנות אנטי וירוס החוקרות קבצים זדוניים. המטרה היא בסך הכל לספק כלי שנותן למשתמש תמונת מצב של הרשת, ומאפשר לו להבחין בתעבורה ובהתנהגויות חריגות.

אפיון

תיאור מפורט של המערכת

המערכת נועדה לספק פתרון שלם לניטור התעבורה שעוברת בכרטיס הרשת של המחשב. היא מחולקת לשני חלקים מרכזיים. החלק הראשון הוא סוכן הקצה שרץ ברקע וקורא כל פאקטה שעוברת, מנתח את סוג הפרוטוקול שלה ושומר סטטיסטיקות. החלק השני הוא השרת המרכזי שמקבל את הנתונים האלו דרך הרשת ומציג אותם למשתמש. מעבר לסטטיסטיקה, המערכת אמורה לזהות מצבים שבהם יש חריגות, כמו למשל כמות גדולה מאוד של בקשות התחברות שלא נענות, ולהתריע על כך כדי שהמשתמש יוכל להחליט אם לחסום את המקור.

פירוט יכולות לפי סוג משתמש

סוג המשתמש היחיד במערכת הוא מנהל הרשת המקומית שהוא בעצם זה שמריץ את התוכנה. היכולות של המשתמש כוללות:

- הפעלה ועצירה של סורק התעבורה בכל רגע נתון
- צפייה בגרפים וסטטיסטיקות של סוגי הפאקטות ברשת בזמן אמת
- קבלת התראות על אנומליות ודפוסים חריגים בתעבורה
- הוספת כתובות IP לרשימת סינון
- שמירת נתוני התעבורה לקובץ במחשב לצורך גיבוי או ניתוח עתידי
- פילטור התעבורה (פקטות) לפי IP, Port, MAC פרוטוקולים ועוד

פירוט הבדיקות המתוכננות

בדיקת לכידת התעבורה: מטרת הבדיקה היא לוודא שסוכן הקצה מצליח להאזין לכרטיס הרשת ולתרגם את המידע. הבדיקה תתבצע על ידי לחיצה על כפתור התחלת ההסנפה, ולאחר מכן ביצוע פעולת רשת פשוטה כמו פתיחת דפדפן או שליחת פינג למחשב אחר. נצפה לראות את הפאקטות החדשות זורמות ומופיעות בטבלה שעל המסך בזמן אמת.

בדיקת זיהוי אנומליות: מטרת הבדיקה היא לוודא שהמערכת מזהה דפוסי תעבורה לא תקינים ומציגה התראה מתאימה למשתמש. הבדיקה תתבצע על ידי יצירת סימולציה של עומס חריג ברשת, למשל שליחת כמות גדולה מאוד של בקשות פינג או בקשות התחברות ממחשב מסוים בזמן קצר. נצפה לראות שהמערכת מזהה את ההתנהגות החריגה הזו ביחס לתעבורה הרגילה, ומקפיצה באופן מיידי חלונת התראה על המסך עם פרטי מקור החריגה.

בדיקת מנגנון הסינון: מטרת הבדיקה היא לוודא שיש למערכת יכולת אקטיבית לעצור תעבורה. הבדיקה תתבצע לאחר קבלת התראה על כתובת חשודה. נלחץ על כפתור החסימה המופיע בממשק, ולאחר מכן ננסה ליצור תקשורת יזומה אל אותה כתובת. נצפה לראות שמערכת ההפעלה לא מאפשרת לתקשורת לעבור ושהכתובת אכן נחסמה בהצלחה.

בדיקת סינון וחיפוש פאקטות: מטרת הבדיקה היא לוודא שתיבת הסינון בממשק מסננת את המידע בצורה נכונה. הבדיקה תתבצע על ידי הקלדה של פרוטוקול מסוים (למשל UDP) או כתובת IP ספציפית בתיבת הטקסט של הפילטר בזמן שהטבלה מלאה בנתונים. נצפה לראות שהטבלה מתעדכנת באופן מיידי ומציגה רק את השורות שמתאימות בדיוק למה שהקלדנו, ושאר השורות מוסתרות.

בדיקת שמירת קובץ PCAP: מטרת הבדיקה היא לוודא שהמערכת מסוגלת לייצא את הנתונים לקובץ סטנדרטי שניתן לפתוח בתוכנות אחרות. הבדיקה תתבצע על ידי לחיצה על כפתור עזירת ההסנפה, ואז לחיצה על כפתור Save to PCAP. נצפה לראות שקופצת הודעת הצלחה על המסך, ושנוצר קובץ חדש על שולחן העבודה שמכיל את כל הפאקטות שנלכדו.

תכנון וניהול לו"ז זמנים לפיתוח המערכת

מספר איטרציה	תיאור אבן הדרך והמטרה	תכנון לוז ראשוני	לוז בפועל
1	ליבת הסורק והסוכן המקומי: הקמת תשתית הלכידה בעזרת SharpPcap, המרת נתונים לאובייקטים והצגת הפאקטות בטבלה.	08.02.26 - 01.02.26	08.02.26 - 01.02.26
2	תקשורת שרת לקוח מאובטחת: בניית חיבור הSockets בין הסוכנים לשרת המרכזי והטמעת הצפנת RSA להגנת המידע.	20.02.26 - 09.02.26	22.02.26 - 09.02.26
3	לוגיקת זיהוי אנומליות וסטטיסטיקה: פיתוח האלגוריתמים שמזהים עומסים חריגים ואיסוף נתונים לגרפים בשרת.	05.03.26 - 21.02.26	10.03.26 - 23.02.26
4	ממשק משתמש וסינון אקטיבי: עיצוב מסכי הWPF בשרת המרכזי ומימוש האפשרות לחסימת כתובות וסינון תעבורה.	20.03.26 - 06.03.26	22.03.26 - 11.03.26
5	ניהול קבצים ותכונות מתקדמות: מימוש אפשרויות עצירת הסנפה, פתיחת ששן חדש, ושמירת הנתונים לקובץ PCAP.	05.04.26 - 21.03.26	08.04.26 - 23.03.26

ניהול סיכונים בפרויקט

1. קריסה או איטיות של הממשק תחת עומס תעבורה

- הסיכון: ברשתות עמוסות זורמות אלפי פאקטות בשנייה. ניסיון לצייר כל פאקטה בנפרד על המסך בזמן אמת עלול לתקוע את ממשק המשתמש ולהקריס את התוכנה.
- תכנון להתמודדות: הפרדת תהליכים כך שקליטת הנתונים תתבצע ברקע, ושימוש בתור נתונים כדי להעביר את המידע למסך בצורה מבוקרת.
- מה בוצע בפועל: השתמשתי במבנה נתונים בשם ConcurrentQueue שהוא בטוח לשימוש בין תהליכים. בנוסף, הוספתי טיימר פנימי שמושך פאקטות מהתור בקבוצות כל 50 מילישניות ומעדכן את המסך בבת אחת. זה פתר לחלוטין את בעיית התקיעות.

2. ניתוק פתאומי של התקשורת מול השרת המרכזי

- הסיכון: מכיוון שהמערכת מבוזרת ומשתמשת בתקשורת Sockets, ניתוק פתאומי של סוכן קצה (למשל עקב נפילת אינטרנט או כיבוי מחשב) עלול לזרוק שגיאה קריטית ולהקריס את השרת המרכזי.
- תכנון להתמודדות: כתיבת קוד בטוח שעוטף את כל פעולות הרשת ויודע להתמודד עם שגיאות תקשורת בצורה שקטה.
- מה בוצע בפועל: הוספתי בלוקים של קוד ללכידת שגיאות מסוג try catch סביב כל פעולות הקריאה והכתיבה ברשת. במקביל, הוספתי מנגנון דופק ששולח הודעות קטנות בין הלקוח לשרת כדי לוודא שהחיבור עדיין קיים מבלי להעמיס על הרשת.

3. צריכת זיכרון חריגה ועומס על המחשב

- הסיכון: אם המערכת תרוץ במשך שעות ותשמור כל פאקטה בזיכרון המקומי, הזיכרון של המחשב יתמלא לחלוטין והמערכת כולה תואט משמעותית.
- תכנון להתמודדות: הגבלת כמות הפאקטות השמורות בזיכרון התצוגה בכל רגע נתון.
- מה בוצע בפועל: הגדרתי משתנה קבוע שמגביל את התצוגה למספר מקסימלי של פאקטות. ברגע שהרשימה מתמלאת, הפאקטות הישנות ביותר נמחקות מהזיכרון באופן אוטומטי במקביל להכנסת החדשות, מה ששומר על צריכת זיכרון יציבה ונמוכה לאורך זמן.

תיאור תחום הידע - פרק מילולי

מכיוון שהפרויקט שלי בנוי בתצורה של מערכת מבוזרת, חילקתי את היכולות לשני חלקים: היכולות שרצות אצל סוכן הקצה (הלקוח שמוחקן על המחשב המנוטר) והיכולות שרצות בשרת המרכזי שמנהל הכל.

פירוט היכולות בצד לקוח (סוכן הקצה)

- שם היכולת: הסנפת תעבורת רשת בזמן אמת (Sniffing)
- מהות היכולת: קריאה וניתוח של כל הפאקטות שעוברות דרך כרטיס הרשת המקומי, תרגומן למידע קריא והצגתן למשתמש.
- אוסף יכולות נדרשות:
 - ממשק משתמש – בחירת כרטיס רשת להאזנה
 - פתיחת כרטיס הרשת להאזנה במצב Promiscuous
 - קליטת נתונים גולמיים בלולאת רקע
 - פירוק ופענוח הפאקטה לחלקים (כתובות IP, פורטים, סוג פרוטוקול)
 - בדיקת תקינות וסינון מוקדם (Pre Filter)
 - הכנסת הפאקטות המעובדות לתור נתונים למניעת עומס
 - שליפה מהתור והצגת הנתונים בטבלת הממשק בזמן אמת
- אובייקטים נחוצים: כרטיס רשת, לולאת רקע מופרדת (Thread), תור נתונים (Queue), ממשק משתמש (טבלה).

- שם היכולת: זיהוי והגנה אקטיבית מפני אנומליות (SYN / ICMP Flood)
- מהות היכולת: ניתוח התעבורה לאיתור דפוסים חריגים של עומס בקשות, חסימת הכתובת התוקפת והתראה למשתמש.
- אוסף יכולות נדרשות:
 - זיהוי ובידוד פאקטות רלוונטיות מתוך כלל התעבורה (למשל פאקטות עם דגל SYN)
 - ספירת כמות הפאקטות לפי כתובת IP שולחת
 - בדיקה תקופתית (טיימר) של הכמויות מול ערכי סף שהוגדרו מראש
 - זיהוי חריגה והפעלת טריגר של מתקפה
 - הצפנת פרטי האירוע ושליחתם לשרת המרכזי לתיעוד
 - הקפצת חלון התראה גרפי בממשק המשתמש
 - שליחת פקודת מערכת ליצירת חוק בחומת האש שחוסם את כתובת התוקף
- אובייקטים נחוצים: טיימרים, מבנה נתונים מסוג מילון (Dictionary), הצפנה, תקשורת, חומת אש מערכתית, ממשק התראות.

● שם היכולת: ייצוא הסנפה לקובץ PCAP

- מהות היכולת: שמירת נתוני התעבורה שנקלטו לקובץ סטנדרטי מקומי לצורך ניתוח ובדיקה עתידית.
- אוסף יכולות נדרשות:
 - לחיצה על כפתור שמירה בממשק המשתמש
 - פתיחת חלונית לבחירת נתיב ושם הקובץ במחשב
 - יצירת קובץ וכתיבת כותרת ראשית (Global Header) בפורמט התקני
 - ריצה בלולאה על כל הפאקטות השמורות בזיכרון
 - כתיבה בינארית של כותרת ותוכן לכל פאקטה בנפרד
 - סגירת הקובץ בצורה בטוחה
 - הצגת הודעת הצלחה למשתמש
- אובייקטים נחוצים: ממשק משתמש, אובייקט ניהול קבצים (FileStream), ממיר נתונים בינארי.

● שם היכולת: סינון וחיפוש דינמי של תעבורה

- מהות היכולת: הצגת תעבורה רלוונטית בלבד למשתמש על ידי סינון טקסטואלי חי של כתובות או פרוטוקולים מתוך עשרות אלפי הפאקטות שנלכדו.
- אוסף יכולות נדרשות:
 - הקלדת טקסט בתיבת החיפוש בממשק המשתמש
 - האזנה לאירוע שינוי טקסט (Text Changed)
 - קריאה לפונקציית סינון פנימית
 - ריצה על רשימת הנתונים השמורה בזיכרון התצוגה
 - בדיקת התאמה בין הטקסט המוקלד לבין הערכים של כל פאקטה (למשל כתובת IP או סוג הפרוטוקול)
 - הסתרת השורות שלא תואמות והשארת התוצאות הרלוונטיות בלבד על המסך
- אובייקטים נחוצים: ממשק משתמש (תיבת טקסט), רשימת נתונים חכמה, פונקציות חיתוך טקסט.

- שם היכולת: תקשורת מאובטחת וסנכרון מול השרת (Heartbeat)
- מהות היכולת: יצירת חיבור רציף, יציב ומוצפן מול השרת המרכזי לשליחת נתונים, תוך דיווח קבוע על תקינות הסוכן.
- אוסף יכולות נדרשות:
 - יזימת התחברות לכתובת ולפורט של השרת המרכזי
 - קבלת המפתח הציבורי של השרת דרך זרם התקשורת
 - הפעלת טיימר רקע (למשל כל שנייה) לאיסוף סטטיסטיקות
 - המרת הנתונים למחרוזת טקסט וקידוד לבתים
 - הצפנת המידע בעזרת המפתח הציבורי שהתקבל (מנגנון RSA)
 - שליחת המידע המוצפן לשרת וטיפול בשגיאות במקרה של ניתוק פתאומי ברשת
- אובייקטים נחוצים: ערוץ תקשורת מסוג TcpClient, טיימר, מנגנון הצפנה, זרמי קריאה וכתיבה (Streams).

פירוט היכולות בצד שרת (הניהול המרכזי)

- שם היכולת: איסוף נתונים סטטיסטיים ותצוגה מרכזית
- מהות היכולת: קבלת נתוני תעבורה והתראות מסוכני הקצה ברשת, עיבודם והצגתם בזמן אמת למנהל המערכת.
- אוסף יכולות נדרשות:
 - פתיחת ערוץ האזנה בפורט 8888 (TcpListener) להמתנה לחיבורי סוכנים
 - קבלת הודעות מוצפנות מהרשת בלולאה אסינכרונית
 - פענוח ההודעות בעזרת המפתח הפרטי של השרת
 - מיון הנתונים לפי סוג (התראות ALERT, סך פאקטות STATS, או נתוני פרוטוקולים (PROTOCOL_STATS)
 - עדכון מודל הנתונים של התוכנה (אובייקט ConnectedAgent)
 - רענון ממשק המשתמש (רשימת הסוכנים והתפלגות הפרוטוקולים) בהתאם לנתונים החדשים
- אובייקטים נחוצים: תקשורת רשת, קורא נתונים (StreamReader), מבני נתונים מסוג ObservableCollection, ממשק משתמש ב-WPF.
- שם היכולת: ייצור והפצת מפתחות הצפנה (RSA)
- מהות היכולת: בניית תשתית אבטחה שמונעת האזנות סתר ברשת על ידי יצירת מפתחות ייעודיים וחלוקתם לסוכנים המחוברים.
- אוסף יכולות נדרשות:
 - אתחול אובייקט אבטחה בעת הפעלת מסך השרת הראשי
 - ייצור אוטומטי של זוג מפתחות אסימטריים (ציבורי ופרטי) ברמת אבטחה של 2048 ביט
 - שמירת המפתח הפרטי בזיכרון השרת בלבד
 - זיהוי חיבור חדש מצד סוכן קצה
 - ייצוא המפתח הציבורי לפורמט XML
 - שליחת המפתח הציבורי ישירות לסוכן שהתחבר על גבי זרם התקשורת
- אובייקטים נחוצים: מנגנון אבטחה מסוג RSACryptoServiceProvider, שרת תקשורת מאזין, כותב נתונים (StreamWriter).

● שם היכולת: ניהול סוכנים פעילים ומעקב סטטוס

- מהות היכולת: מעקב שוטף אחרי מצב הסוכנים המחוברים לשרת, ניהול המידע שלהם וזיהוי ניתוקים.
- אוסף יכולות נדרשות:
 - יצירת מזהה ייחודי (IP) לכל סוכן שמתחבר לשרת
 - שמירת הסוכן בתוך מבנה נתונים שבטוח לשימוש מקביל (כדי למנוע קריסות מריבוי תהליכים)
 - קבלת הודעות דופק (HEARTBEAT) תקופתיות מהסוכנים
 - עדכון סטטוס הסוכן ל"מחובר וסורק" (Online / Scanning) בעקבות קבלת הדופק
 - שינוי סטטוס הסוכן ל"מנותק" בממשק המשתמש כאשר החיבור נופל (בתוך בלוק finally)
- אובייקטים נחוצים: מילון נתונים בטוח (ConcurrentDictionary), מנגנון ניהול חריגות (Exceptions), מודל נתונים מותאם לממשק (INotifyPropertyChanged).

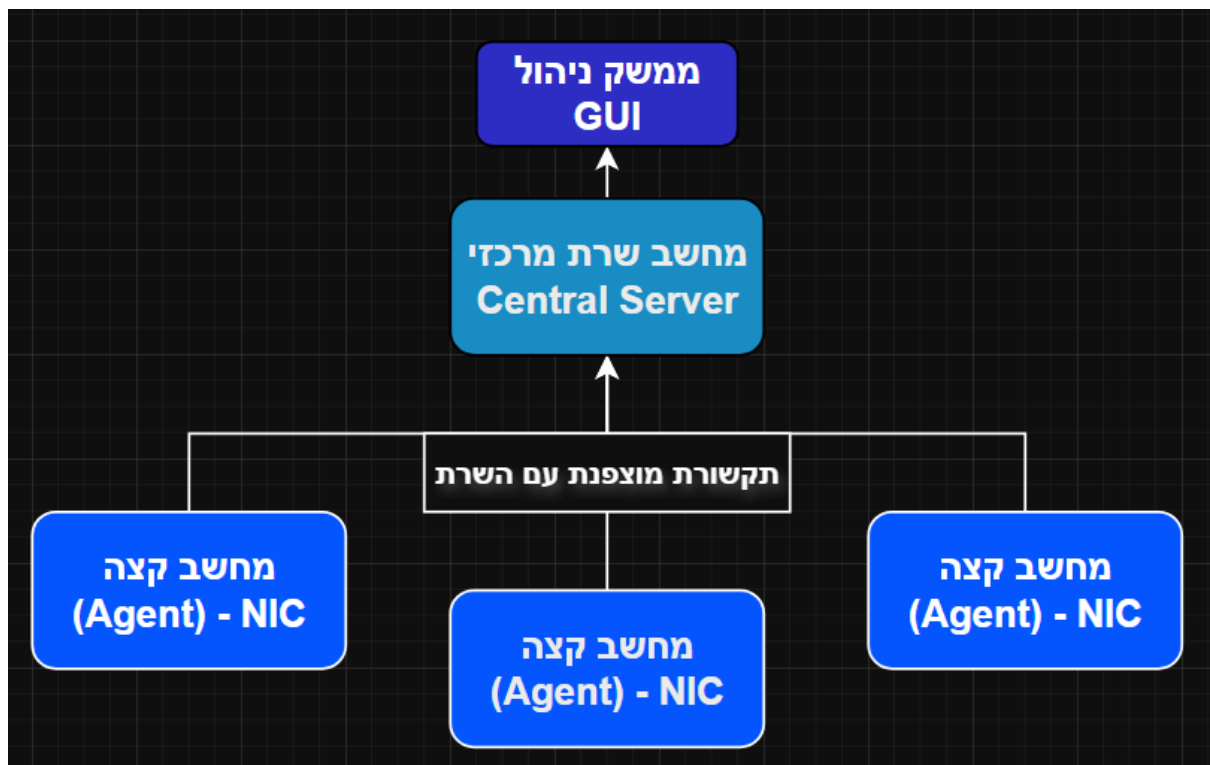
מבנה הפרויקט

תיאור החומרה - רכיבים שונים והקשרים ביניהם

המערכת בנויה בארכיטקטורת שרת לקוח (Client Server) מבוססת. מטרת מבנה זה היא לאפשר איסוף נתונים ממספר נקודות קצה שונות ברשת, וריכוזן בנקודת ניהול אחת לצורך ניתוח והצגה.

כדי להריץ את המערכת נדרשת החומרה הבאה:

- מחשב סוכן הקצה (Agent): מחשב רגיל (תחנת עבודה) המחובר לרשת, שעליו מותקנת תוכנת הסריקה המקומית. מחשב זה חייב לכלול כרטיס רשת (NIC) פיזי או אלחוטי תקין, דרכו עוברת התעבורה שאותה אני מנטר בפועל.
- מחשב שרת מרכזי (Central Server): מחשב ייעודי שפועל כנקודת הניהול של הרשת. הוא מאזין לבקשות נכנסות ומריץ את ממשק המשתמש המרכזי שמציג את הסטטיסטיקות וההתראות.



תיאור הטכנולוגיה במערכת

שפת תכנות וסביבת פיתוח:

- לוגיקה וצד שרת: הפרויקט כולו נכתב בסביבת Visual Studio בעזרת שפת C#. הבחירה בשפה הזו הייתה על מנת לתמוך בתכנות מונחה עצמים (OOP), וגם בגלל הניהול האוטומטי של הזיכרון (Garbage Collection) שעוזר למנוע קריסות כשיש עומס של אלפי פאקטות. בנוסף, השפה מאפשרת ניהול נוח של ריבוי תהליכים (Multi Threading).
- ממשק משתמש: לעיצוב ממשק המשתמש (GUI) נבחרה טכנולוגיית WPF שמבוססת על XAML. הטכנולוגיה הזו מאפשרת לבנות ממשק מודרני שמתעדכן בזמן אמת עם נתונים מהרשת, בלי לתקוע או להקפיא את המסך של התוכנה.

מערכת הפעלה: המערכת מיועדת לעבוד על מערכת ההפעלה Windows (גם בשרת וגם בלקוח). הסיבה לזה היא שסוכן הקצה משתמש בפקודות ספציפיות של חלונות (כמו הפעלת netsh כדי ליצור חוקי חסימה ב Firewall). בנוסף, חייבים להתקין על המחשב דרייבר בשם Npcap שעובד רק בסביבת Windows, ובלעדיו אי אפשר להאזין לכרטיס הרשת.

תקשורת (פרוטוקולים וספריות): התקשורת בפרויקט מחולקת לשני חלקים:

- לכידת תעבורה (Sniffing): כדי לקרוא את הפאקטות שעוברות ברשת, הוספו לפרויקט הספריות החיצוניות PacketDotNet ו SharpPcap. הספריות האלו נותנות גישה לכרטיס הרשת ברמה נמוכה (Low Level), קולטות את המידע הבינארי ומפענחות אותו לנתונים קריאים כמו כתובות IP ופורטים.
- תקשורת שרת לקוח: העברת הסטטיסטיקות וההתראות מהסוכנים לשרת המרכזי עובדת עם Sockets על גבי פרוטוקול TCP בפורט 8888. הבחירה ב TCP הייתה כדי להבטיח שהמידע יגיע בצורה אמינה ושלמה. כדי להגן על התקשורת מהאזנות סתר, כל המידע עובר הצפנה אסימטרית בעזרת מפתח RSA.

תחומי עניין מעורבים: הפרויקט נוגע במספר תחומים מרכזיים:

- אבטחת מידע וסייבר: זיהוי התנהגות חריגה ברשת, מניעת מתקפות עומס והצפנת מידע.
- חקר רשתות (Networking): הבנה של מודל השכבות, מבנה חבילות נתונים ופרוטוקולי תקשורת כמו TCP ו UDP.
- עיבוד נתונים מקבילי: ניהול תורים והתמודדות עם קצב נתונים מהיר מאוד מבלי לקרוס.

החלטות ארכיטקטוניות

למה TCP ולא UDP

אחת ההחלטות המרכזיות בתכנון פרוטוקול התקשורת בין הסוכנים לשרת הייתה הבחירה ב TCP. UDP היה יכול להיות אפשרות מהירה יותר, אבל הוא לא מתאים כאן מכמה סיבות. ראשית, המערכת שולחת הודעות קריטיות כמו התראות אבטחה ונתוני סטטיסטיקה שחייבים להגיע בשלמותן ובסדר הנכון. עם UDP אין ערובה לכך שהודעה תגיע בכלל, ובמקרה של התראה על מתקפה זה לא מקובל. שנית, מנגנון ה RSA שמוצפן בפרויקט מניח שההודעות מגיעות בסדר ושלמות, כי פענוח של הודעה חלקית יגרום לשגיאה. לכן TCP, שמבטיח העברה מסודרת ואמינה, היה הבחירה הנכונה.

Scalability

הארכיטקטורה הנוכחית מתאימה לרשת קטנה עד בינונית של עד כמה עשרות סוכנים. כל סוכן מחובר לשרת בערוץ TCP עצמאי, והשרת מריץ תהליכון נפרד לכל סוכן. כשמספר הסוכנים גדל, כמות התהליכונים גדלה בהתאם, מה שעלול להפוך לבעיה בסביבה ארגונית גדולה מאוד. פתרון אפשרי לעתיד יהיה מעבר לארכיטקטורת I/O אסינכרונית מלאה עם מספר מוגבל של תהליכונים שמטפלים בכל הסוכנים, בדומה למודל שמשתמשים בו שרתי web מודרניים.

צווארי בקבוק

צוואר הבקבוק המרכזי במערכת הוא ממשק המשתמש. ב Windows, כל עדכון גרפי חייב לעבור דרך ה UI Thread הראשי, מה שיוצר מגבלה טבעית על קצב ההצגה. גם אם הלכידה והעיבוד מתבצעים במהירות גבוהה על ידי ארבעת תהליכוני הרקע, בסופו של דבר כל הנתונים עוברים דרך נקודה אחת לפני שמוצגים על המסך. הפתרון שיושם הוא איסוף הפאקטות בתור ורענון הממשק אחת ל 50 מילישניות בלבד, מה שמאזן בין עדכניות המידע לבין עומס על ה UI Thread.

צוואר בקבוק שני קיים בצד ה RSA. גודל ההודעה שניתן להצפין ב RSA עם מפתח של 2048 ביט מוגבל לכ 245 בתים. הודעות ארוכות יותר יגרמו לשגיאה. בפרויקט זה ההודעות קצרות מספיק שזה לא בעיה, אבל זה שיקול חשוב אם בעתיד רוצים להעביר נתונים גדולים יותר.

תיאור זרימת המידע במערכת

1. לכידת תעבורה והצגה בממשק (Sniffing):

יכולת זו פועלת בצד סוכן הקצה באופן רציף כדי לקלוט נתונים מהרשת ולהציגם למשתמש ללא פגיעה בביצועי המחשב.

- תיאור הזרימה: התהליך מתחיל כאשר כרטיס הרשת קולט אותות מהראוטר. ספריות העזר (PacketDotNet ו SharpPcap) לוכדות את המידע הבינארי הגולמי, והקוד מפענח אותו לאובייקטים המכילים כתובות IP וסוגי פרוטוקולים. כדי למנוע קריסה של הממשק, כל פאקטה שפוענחה נכנסת לתור נתונים בטוח (ConcurrentQueue) בזיכרון המערכת. במקביל, טיימר גרפי שולף בכל חלקיק שנייה את הנתונים המצטברים בתור, ומרענן את טבלת התצוגה בממשק המשתמש.

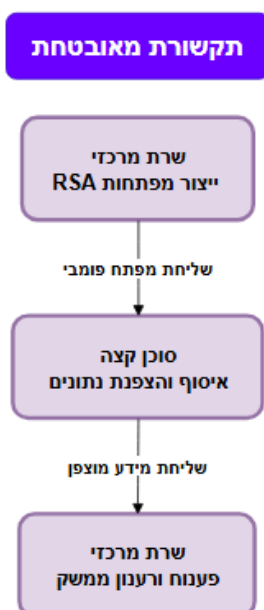
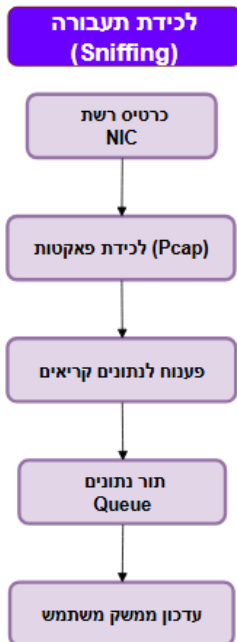
2. ניהול סוכנים פעילים (שרת מרכזי):

- תיאור הזרימה: השרת מנהל מבנה נתונים בטוח (ConcurrentDictionary) המכיל את כל הסוכנים המחוברים. בכל פעם שמתקבלת הודעת דופק מסוכן כלשהו, הסטטוס שלו מתעדכן כפעיל. במקביל, במקרה של ניתוק פתאומי מצד הלקוח (נפילת רשת או קריסה), פונקציית התקשורת בשרת לוכדת את החריגה. הקוד נכנס לבלוק הטיפול בשגיאות, סוגר את ערוץ התקשורת מול אותו סוכן כדי לפנות משאבים, ומעדכן את הסטטוס שלו ל"מנותק" בממשק מנהל הרשת.

3. תקשורת מאובטחת וסנכרון מול השרת:

תהליך זה אחראי על סנכרון הנתונים בין סוכני הקצה לשרת המרכזי בצורה מוצפנת שלא מאפשרת האזנת סתר.

- תיאור הזרימה: התהליך מתחיל כאשר השרת המרכזי מופעל ומייצר זוג מפתחות אבטחה (RSA). כאשר סוכן קצה מתחבר לשרת דרך פרוטוקול TCP, השרת שולח לו בחזרה את המפתח הפומבי. סוכן הקצה אוסף את נתוני הסטטיסטיקה של הפאקטות ואת הודעת הדופק, אורז אותם למחרוזת אחת, ומצפין אותם בעזרת המפתח הפומבי. המידע המוצפן נשלח ברשת חזרה אל השרת, אשר משתמש במפתח הפרטי השמור אצלו כדי לפענח את המידע ולרענן את טבלאות הסטטיסטיקה בממשק הניהול.



4. ייצוא הסנפה לקובץ PCAP (סוכן קצה): תהליך זה שומר את המידע מהזיכרון הזמני לקובץ פיזי על הדיסק הקשיח.

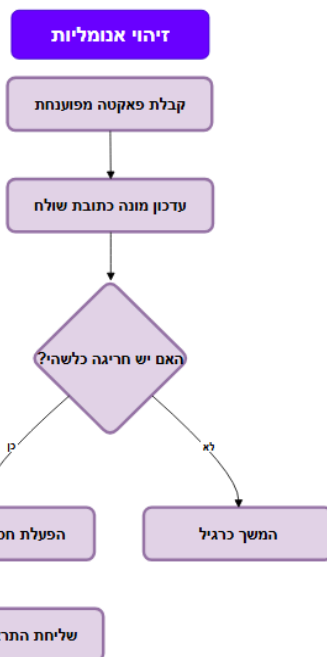
- תיאור הזרימה: התהליך מתחיל בלחיצה על כפתור שמירה בממשק. נפתח חלון בחירת נתיב במערכת ההפעלה. לאחר אישור הנתיב, התוכנה יוצרת קובץ חדש וכותבת אליו כותרת ראשית בפורמט תקני. מיד לאחר מכן, מופעלת לולאה העוברת על כל הפאקטות השמורות בזיכרון התצוגה, ממירה כל פאקטה לפורמט בינארי, וכותבת אותה לקובץ. בסיום, זרם הנתונים נסגר בבטחה ומוצגת הודעת הצלחה למשתמש.

5. סינון וחיפוש דינמי של תעבורה (סוכן קצה):

- תיאור הזרימה: התהליך מותנע ברגע שמוקלד טקסט בתיבת החיפוש בממשק המשתמש. מערכת ההפעלה מפעילה אירוע של שינוי טקסט, שקורא לפונקציית סינון. הפונקציה עוברת על כל רשימת הנתונים הגלוייה, ומשווה את הטקסט המוקלד לערכים של כל פאקטה (למשל סוג פרוטוקול). שורות שאין בהן התאמה מוסתרות מידיית מהמסך, ורק התוצאות הרלוונטיות נשארות גלויות בזמן אמת, מבלי לפגוע בהמשך קליטת הנתונים ברקע.

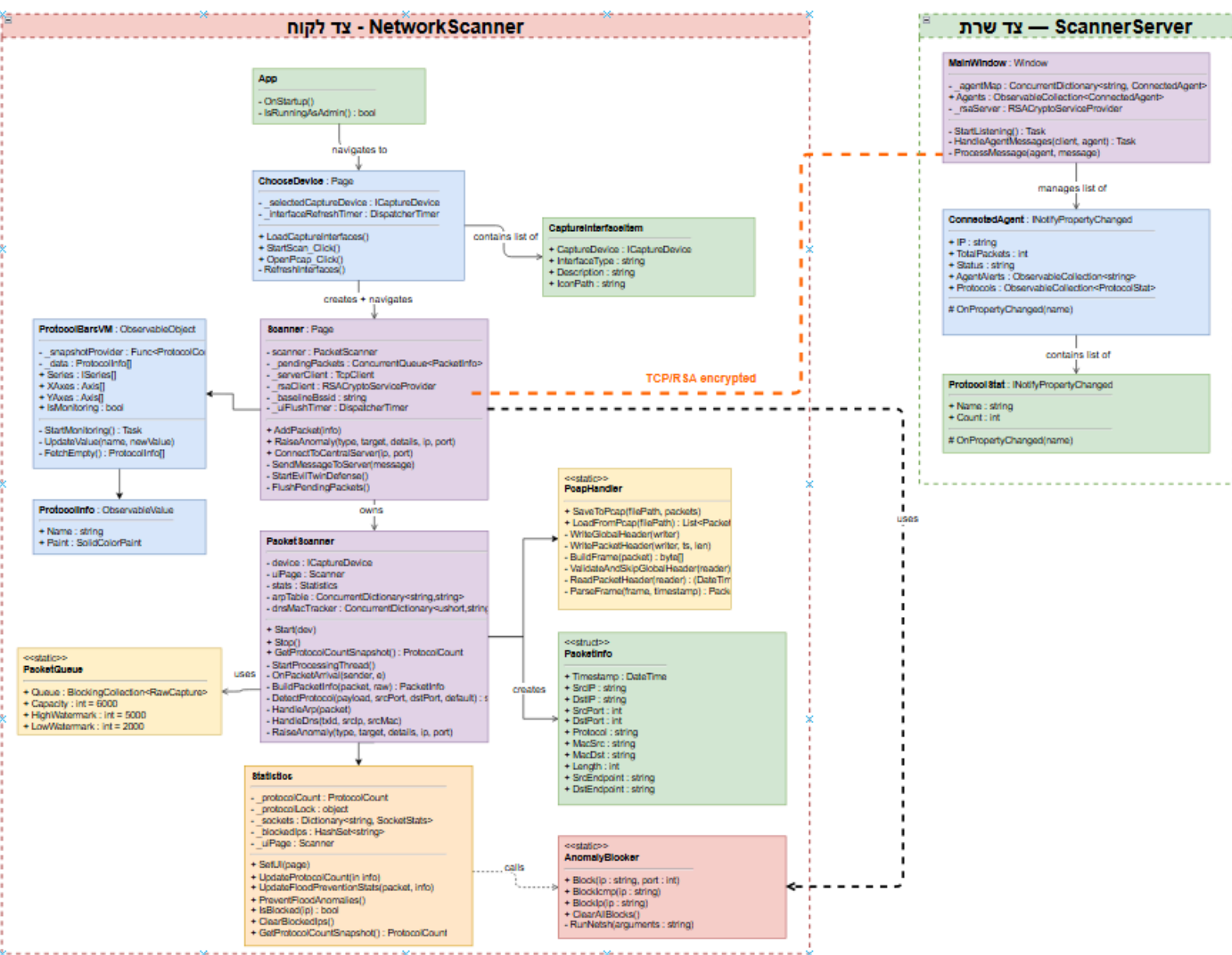
6. זיהוי אנומליות והגנה אקטיבית:

תהליך זה נועד לזהות ניסיונות תקיפה או עומס חריג ברשת, ולבלום אותם בזמן אמת.



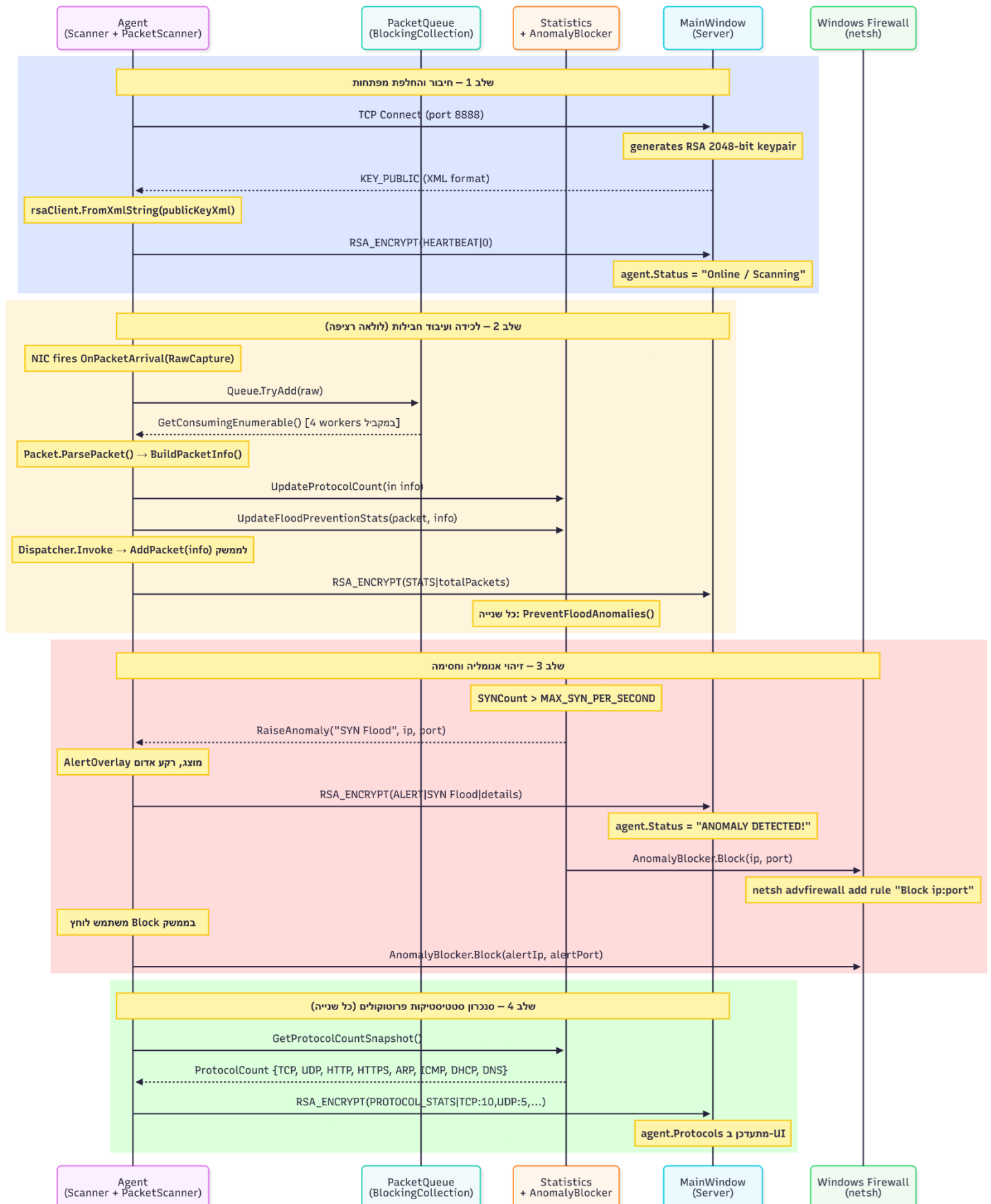
- תיאור הזרימה: לאחר שפאקטה מפוענחת, היא מועברת למנגנון סינון שבודק את כתובת השולח שלה. המערכת מנהלת מילון נתונים שסופר כמה בקשות התקבלו מכל כתובת. טיימר ייעודי בודק בכל שנייה האם כמות הבקשות מכתובת מסוימת חורגת מערך הסף המותר. במידה וזוהתה חריגה, מתבצעות שתי פעולות במקביל: הפעלת פקודת מערכת שמוסיפה את הכתובת התוקפת לרשימת החסימה של חומת האש (Windows Firewall), ושליחת הודעת התראה מוצפנת לשרת המרכזי כדי לעדכן את מנהל הרשת.

תרשים UML



קישור לצפייה: [Draw.io](https://draw.io)

Sequence Diagram



תיאור האלגוריתמים המרכזיים בפרויקט

ניסוח וניתוח של הבעיה האלגוריתמית

במערכת המבצעת האזנה לתעבורת רשת בזמן אמת, קצב חבילות הנתונים הנכנסות עשוי להגיע לאלפים בשנייה. האתגר המרכזי הוא זיהוי התנהגויות חריגות ומתקפות מניעת שירות, כגון הצפת בקשות מסוג SYN או ICMP, המגיעות מכתובת מקור בודדת או ממספר כתובות מוגדר.

הבעיה האלגוריתמית מוגדרת כך: יש לבצע מעקב רציף אחר כמות חבילות הנתונים מכל כתובת רשת בתוך חלון זמן נתון, לזהות חריגה מערך סף מקסימלי שהוגדר מראש, ולפעול לחסימת המקור. כל זאת חייב להתבצע בסיבוכיות זמן מינימלית כדי לא ליצור עומס או עיכוב בתהליך הלכידה הראשי, ובסיבוכיות מקום יעילה שתמנע צריכת זיכרון מוגזמת על ידי התוכנה.

תיאור אלגוריתמים קיימים לפתרון הבעיה

- אלגוריתם חלון נע (Sliding Window Log): אלגוריתם זה שומר חותמת זמן מדויקת עבור כל חבילת נתונים שהתקבלה ברשת. בעת הגעת חבילה חדשה, המערכת מוחקת מהזיכרון את כל חותמות הזמן הישנות שחורגות מחלון הזמן המוגדר, וסופרת את כמות החבילות שנותרו בזיכרון.
 - יתרון: דיוק מוחלט ומניעת נקודות עיוורות בכל רגע נתון.
 - חיסרון: צריכת זיכרון גבוהה מאוד התלויה ישירות במספר חבילות הנתונים הנכנסות, דבר העלול להוביל להאטה חמורה או לקריסת הזיכרון בזמן מתקפת הצפה רחבה.
- אלגוריתם חלון קבוע (Fixed Window Counter): אלגוריתם זה מחלק את ציר הזמן למקטעים קבועים ורציפים (למשל, חלונות קבועים של שנייה אחת). עבור כל כתובת רשת מנוהל מונה מספרי בודד המתעדכן בכל פעם שמזוהה חבילת נתונים מאותה הכתובת. בסיום המקטע, המונים מאופסים לחלוטין והספירה מתחילה מחדש.
 - יתרון: חסכוני מאוד במשאבי זיכרון ופשוט למימוש.
 - חיסרון: קיימת נקודה עיוורת בגבולות שבין החלונות. הצפה מסיבית שמתרחשת בדיוק בנקודת המעבר בין שני חלונות עוקבים עלולה להתחלק ביניהם ולא להיקלט כחריגה, למרות שבפועל חלון הזמן המצטבר הופר.

סקירת הפתרון הנבחר ונימוק הבחירה

בזכות אילוצי המשאבים של סוכן הקצה, נבחר שימוש באלגוריתם חלון קבוע בשילוב מבנה נתונים מילוני מהיר. המערכת מנהלת מילון בזיכרון שבו מפתח הגישה הוא כתובת הרשת של השולח, והערך הוא מונה מספרי פשוט.

- תהליך העדכון: עבור כל חבילת נתונים שמפוענחת, מתבצעת בדיקה ועדכון של המונה במילון בסיבוכיות זמן ממוצעת של $O(1)$
- תהליך הבדיקה והאיפוס: טיימר מערכת ייעודי פועל ברקע בכל שנייה אחת קבועה. הטיימר סורק את ערכי המילון, משווה אותם לערך הסף שנקבע בממשק המשתמש, ובמידה ומזוהה חריגה – מופעל מנגנון החסימה האקטיבי בחומת האש. מיד לאחר מכן המילון מנוקה ומאופס לקראת החלון הבא.

נימוק הבחירה ושליטת האלטרנטיבה:

הבחירה באלגוריתם זה על פני חלון נע נבעה מהצורך להגן על משאבי המחשב המקומי שבו מותקן הסוכן. מכיוון שהתוכנה מיועדת לרוץ כתהליך רקע, לא ניתן לאפשר מצב שבו מתקפת הצפה תגרום לצריכת זיכרון מסיבית עקב שמירת חותמות זמן של מיליוני פאקטות. שימוש במונים בתוך מילון מבטיח שצריכת הזיכרון תהיה תלויה אך ורק במספר הכתובות הייחודיות הפועלות ברשת באותה שנייה, ללא קשר לנפח ההצפה עצמו. פתרון זה מעניק חסינות ויציבות למערכת הניטור בזמן אמת.

ניתוח ביצועים

בבדיקות שבוצעו בסביבת רשת אמיתית, המערכת הצליחה לעמוד בעומסי תעבורה גבוהים ללא דיליי, ברמת ביצועים דומה לזו של Wireshark. צריכת הזיכרון נשארה יציבה גם תחת עומס, מאחר שהמילון מכיל לכל היותר כמות ערכים השווה למספר הכתובות הייחודיות ברשת ולא למספר הפאקטות. מנגנון האיפוס כל שנייה מונע הצטברות נתונים לאורך זמן. כך מתאפשר ניטור רציף ואמין גם ברשתות עמוסות, מבלי לפגוע בביצועי המחשב המקומי.

תיאור סביבת הפיתוח

פירוט כלי הפיתוח הדרושים לפיתוח המערכת:

- סביבת פיתוח משולבת (IDE): כתיבת הקוד וניהול הפרויקט בוצעו באמצעות Visual Studio. סביבה זו נבחרה בשל התאימות המלאה שלה לשפת C# ולטכנולוגיית WPF, הכוללת מעצב ממשק חזותי מובנה המקל על עיצוב חלונות התוכנה.
- ניהול חבילות וספריות חיצוניות: שילוב רכיבי התוכנה החיצוניים נעשה באמצעות מנהל החבילות NuGet. דרך תשתית זו הוספו לפרויקט הספריות SharpPcap ו-PacketDotNet האחראיות על הגישה לכרטיס הרשת ופענוח הפאקטות.
- תשתית הפיתוח: הפרויקט מתבסס על פלטפורמת NET. המודרנית, המספקת את כלי התשתית לניהול תהליכים מקביליים, תורים בטוחים ומנגנוני הצפנה מובנים.
- כלי לשרטוט ותיעוד הארכיטקטורה: לשם תכנון מסכי המערכת ויצירת תרשימי זרימת המידע נעשה שימוש ב draw.io.

פירוט הסביבה והכלים הנדרשים לבדיקות המערכת:

- כדי לוודא שהמערכת פועלת בצורה מדויקת, עומדת בעומסי תעבורה ומצליחה לבלום איומים, הוקמה סביבת בדיקות ייעודית הכוללת את הרכיבים הבאים:
- מערכת ההפעלה ומנהל התקן: הבדיקות והרצת המערכת בוצעו על גבי מערכת ההפעלה Windows. על מנת לאפשר לקוד לגשת ישירות לחומרת הרשת ולהאזין למידע במצב promiscuous, הותקן על מחשב הבדיקה דרייבר רשת בשם Npcap המקשר בין כרטיס הרשת לתוכנה.
- כלי חיצוני לאימות נתונים: לצורך בקרת איכות הלכידה, נעשה שימוש בתוכנת Wireshark במקביל להרצת הפרויקט. במהלך הבדיקה בוצעה השוואה בין הפאקטות שנלכדו והוצגו בממשק של סוכן הקצה לבין המידע שנלכד בתוכנת Wireshark, כדי לוודא שאין איבוד נתונים או שגיאות בפענוח הכתובות והפרוטוקולים.
- סימולטורים לשילוח מתקפות עומס (Flood): כדי לבחון את מנגנון זיהוי האנומליות והחסימה האקטיבית, נעשה שימוש בכלי מערכת מובנים (כמו פקודות ping ייעודיות בשורת הפקודה שנשלחו בקצב מהיר) וכן בסקריפטים חיצוניים להזרקת פאקטות מסוג SYN. הבדיקה הוכיחה כי המערכת מזהה את החרגה בזמן אמת, מייצרת חוק חסימה מתאים בתוך Windows Firewall, ומדווחת על כך מיד לשרת המרכזי.

תיאור פרוטוקול התקשורת

תיאור מילולי של פרוטוקול התקשורת

התקשורת במערכת בין סוכני הקצה לשרת המרכזי מתבצעת בשכבת האפליקציה על גבי פרוטוקול TCP בפורט 8888. הבחירה בפרוטוקול TCP מבטיחה הגעה אמינה, מסודרת ושלמה של נתוני הניטור וההתראות ללא איבוד פאקטות ברשת.

רכיבי המערכת מעבירים ביניהם הודעות במבנה טקסטואלי מוגדר הכולל מזהה סוג הודעה (Header) ולאחריו תוכן ההודעה (Payload), כאשר התוכן מופרד באמצעות תו מפריד קבוע. לפני השליחה ברשת, מחרוזת הטקסט מומרת למערך בתים (Byte Array). למעט הודעת החיבור הראשונית, כלל ההודעות הנשלחות מסוכני הקצה לשרת מוצפנות בצד הלקוח בעזרת מפתח ציבורי RSA ומפוענחות בצד השרת בעזרת המפתח הפרטי, דבר המגן על תעבורת הניהול מפני האזנות סתר.

פירוט כלל ההודעות הזורמות במערכת

פירוט ארבעת סוגי ההודעות המרכיבות את פרוטוקול הניהול של המערכת:

שם ההודעה	נשלחת מ- / אל-	מבנה השדות בהודעה ותוכן
PUBLIC_KEY	מהשרת המרכזי אל סוכן הקצה	הודעה לא מוצפנת המכילה את המפתח הציבורי של השרת במבנה של מחרוזת XML בתקן RSA. הודעה זו נשלחת באופן אוטומטי ישר לאחר שסוכן קצה חדש יוצר חיבור TCP ראשוני מול השרת.
HEARTBEAT	מסוכן הקצה אל השרת המרכזי	הודעה מוצפנת המודיעה לשרת שהסוכן פעיל ומבצע סריקה. במנה השדות: סוג הודעה כתובת IP של הסוכן סטטוס דוגמה: HEARTBEAT 192.168.1.50 Scanning
STATS	מסוכן הקצה אל השרת המרכזי	הודעה מוצפנת שנשלחת בכל שנייה ומכילה את נתוני נפח התעבורה שנלכדו על ידי הסוכן לצורך הצגת גרפים וסטטיסטיקות בשרת. מבנה השדות: סוג הודעה סך כל הפאקטות פאקטות TCP פאקטות UDP פאקטות ICMP דוגמה: STATS 1550 1200 300 50
ALERT	מסוכן הקצה אל השרת המרכזי	הודעה מוצפנת שנשלחת באופן מיידי ברגע שמנגנון זיהוי האנומליות של הסוכן מזהה חריגה מערך הסף המותר. מבנה השדות: סוג הודעה סוג המתקפה כתובת IP של התוקף חותמת זמן דוגמה: ALERT SYN Flood 10.0.0.4 14:32:05

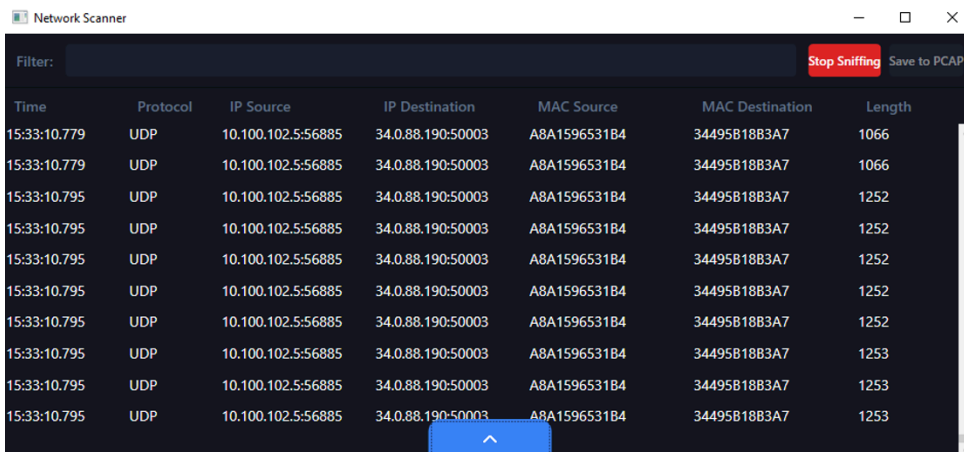
תיאור מסכי המערכת

המערכת מורכבת משתי אפליקציות גרפיות נפרדות שפותחו בטכנולוגיית WPF: אפליקציית סוכן הקצה המותקנת בתחנות העבודה, ואפליקציית השרת המרכזי המשמשת את מנהל הרשת. להלן פירוט המסכים, תפקידם והמידע המוצג בהם:

מסכי אפליקציית סוכן הקצה (Agent)

1. מסך הניטור והלכידה הראשי

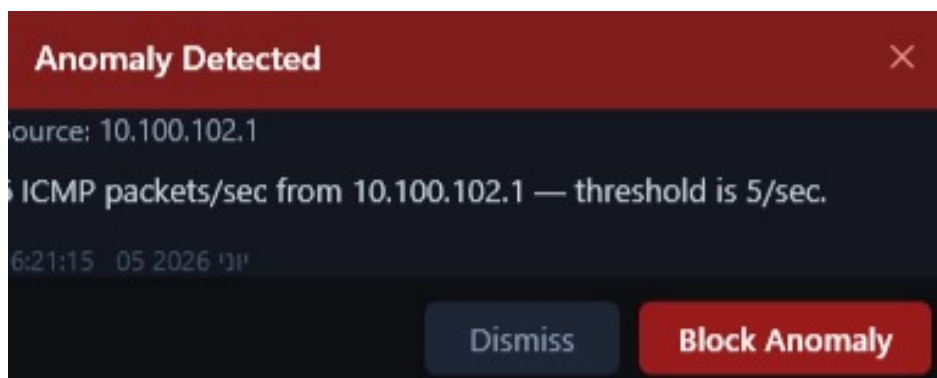
- תפקיד המסך: בחירת כרטיס הרשת להאזנה, הצגת תעבורת הרשת בזמן אמת, ביצוע סינונים וייצוא נתונים.
- תיאור המסך: בחלקו העליון של המסך קיימת תיבת בחירה ComboBox המציגה למשתמש את כל כרטיסי הרשת הפיזיים והווירטואליים הזמינים במחשב. במרכז המסך מוצגת טבלת נתונים חכמה DataGridView המתעדכנת באופן דינמי ומציגה שורה עבור כל פאקטה שנלכדה, כולל כתובת IP מקור, כתובת IP יעד, פורט וסוג הפרוטוקול. המסך כולל תיבת טקסט לחיפוש מהיר המאפשרת לסנן את הטבלה בזמן אמת, וכפתור ייעודי המפעיל את מנגנון השמירה לקובץ PCAP מקומי.
- תמונת המסך:



Time	Protocol	IP Source	IP Destination	MAC Source	MAC Destination	Length
15:33:10.779	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1066
15:33:10.779	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1066
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1253
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1253
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1253

2. חלונית התראת אבטחה (Pop-up)

- תפקיד המסך: הקפצת התראה גרפית מיידית למשתמש בעת זיהוי מתקפת עומס ברשת המקומית.
- תיאור המסך: חלונית זו אינה פתוחה באופן קבוע, אלא מופעלת כטריגר אוטומטי על ידי אלגוריתם זיהוי האנומליות. החלונית מעוצבת בצבעי אזהרה בולטים ומציגה למשתמש את פרטי האירוע: סוג המתקפה שזוהתה (למשל, הצפת SYN), כתובת ה IP של הגורם התוקף, והודעה על כך שבוצעה חסימה אקטיבית של הכתובת האש של מערכת ההפעלה.

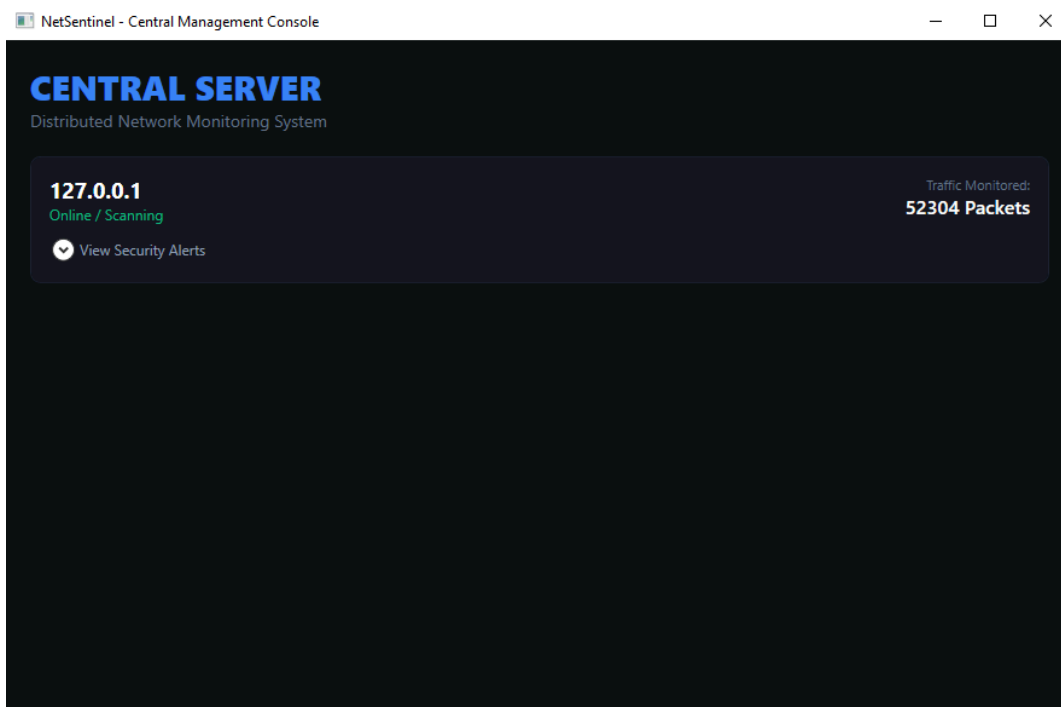


תמונת המסך:

מסכי אפליקציית השרת המרכזי (Central Server)

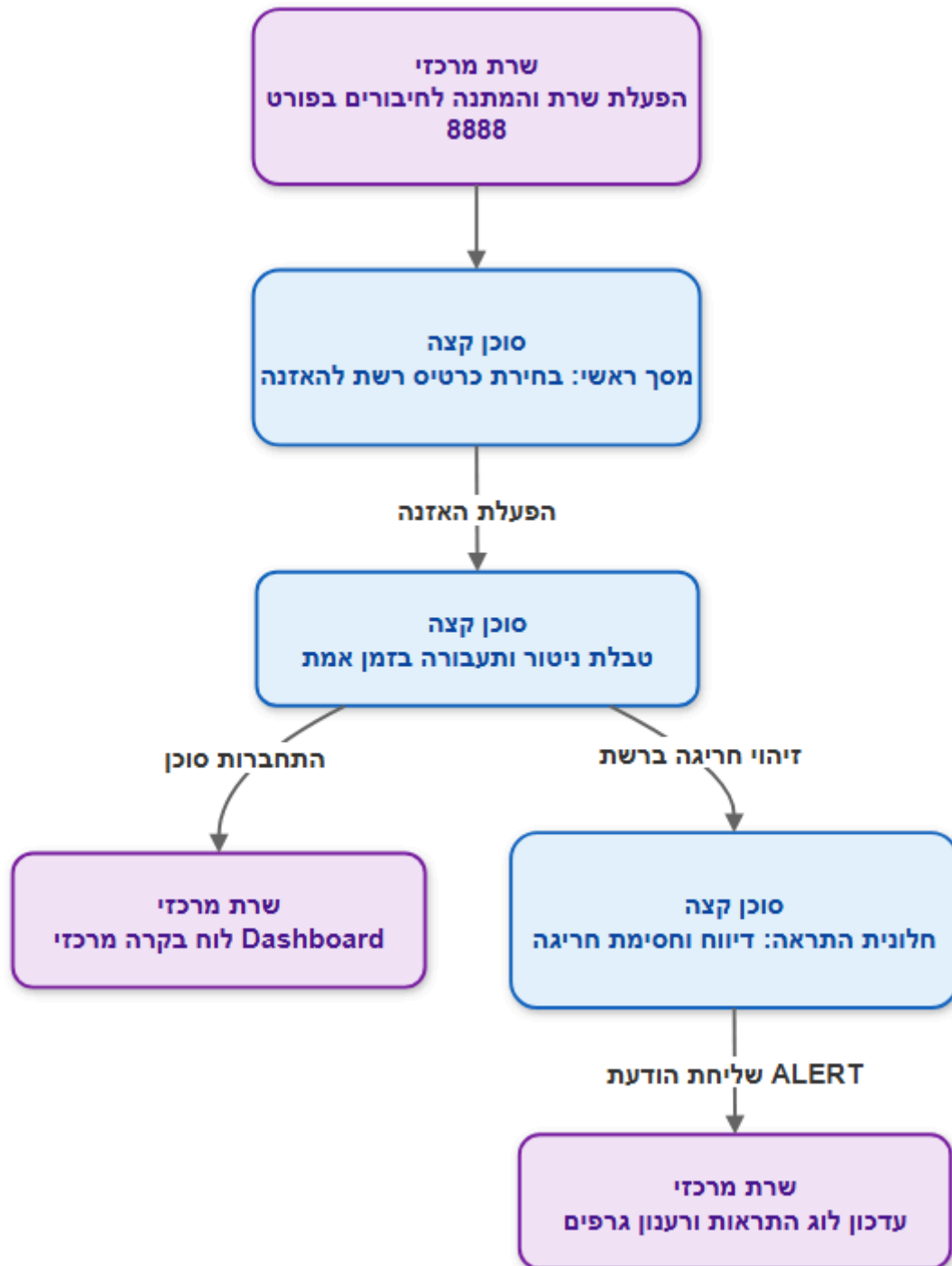
1. לוח הבקרה והניהול המרכזי (Dashboard)

- תפקיד המסך: ריכוז נתוני הניטור מכלל הסוכנים הפעילים ברשת, הצגת גרפים סטטיסטיים ותיעוד אירועי אבטחה מרכזיים.
- תיאור המסך: מסך זה מחולק לשלושה אזורי עבודה עיקריים:
 - רשימת סוכנים: תצוגה דינמית של כלל המחשבים המחוברים לשרת, המציגה את כתובת ה IP שלהם ואת סטטוס הפעילות שלהם (סורק או מנותק) על בסיס הודעות הדופק המתקבלות.
 - לוח סטטיסטיקה: רכיבים גרפיים המציגים בזמן אמת את נפח התעבורה המצטבר ואת התפלגות הפרוטוקולים ברשת (גרף עוגה או עמודות של TCP, UDP ו-ICMP).
 - לוג התראות מרכזי: טבלה המרכזת את כל הודעות ה ALERT שהתקבלו מסוכני הקצה השונים, ומאפשרת למנהל הרשת לראות איזה סוכן הותקף, מאיזו כתובת ובאיזו שעה.
- תמונת המסך:



תרשים מסכים והמעברים ביניהם (Screen Flow Diagram)

מכיוון שהמערכת מבוססת על ארכיטקטורת שרת לקוח מבוזרת, זרימת המסכים מתבצעת בשני צירים מקבילים בהתאם לפעולות המשתמש ואירועי הרשת.



תיאור מבני הנתונים במערכת

המערכת אינה עושה שימוש במסד נתונים רילציוני חיצוני, אלא מתבססת על שילוב של מבני נתונים מקומיים מהירים בזיכרון המחשב לצורך עיבוד וניהול הנתונים בזמן אמת, לצד שמירת המידע בקבצים בתצורה בינארית קבועה ותקנית. להלן פירוט מאגרי המידע ומבני הנתונים במערכת, בחלוקה לרמת השדות, הטיפוסים והתפקיד שלהם:

מבני נתונים מקומיים בזיכרון המערכת

1. מילון ניהול סוכנים בשרת (ConcurrentDictionary)

מבנה נתונים זה מנוהל בצד השרת ומטרתו לעקוב אחר כלל הסוכנים הפעילים ברשת. הבחירה בטיפוס זה נועדה לאפשר גישה, קריאה ועדכון של נתונים במקביל ממספר תהליכים שונים ללא חשש לקריסת התוכנה.

- מפתח ייחודי (Primary Key): כתובת IP של הסוכן (מחרוזת טקסט).

שם השדה	טיפוס הנתון (#C)	תיאור השדה	דוגמה לערך אפשרי
IpAddress	String	כתובת IP הייחודית של תחנת הקצה המשמשת כמפתח	"192.168.1.50"
Status	String	סטטוס הפעילות הנוכחי של הסוכן במערכת	"Scanning"
LastHeartbeat	DateTime	חותמת הזמן של הודעת הדופק האחרונה שהתקבלה	18:30:00 04/06/2026
TotalPackets	Long	סך כל הפאקטות שנלכדו על ידי הסוכן מתחילת הריצה	450230

2. תור פאקטות נכנסות בסוכן (ConcurrentQueue)

תור זה משמש כחוצץ זמני בצד הסוכן. פאקטות נכנסות נדחפות אליו מיד עם לכידתן ברקע על ידי הדרייבר, ונשלפות ממנו על ידי תהליך הממשק לצורך תצוגה וניתוח. מבנה זה מונע איבוד מידע בעת עומס תעבורה גבוה.

- מבנה השדות של אובייקט חבילת המידע (PacketInfo):

שם השדה	טיפוס הנתון (#C)	תיאור השדה	דוגמה לערך אפשרי
Id	Integer	מספר סידורי רץ של הפאקטה המשמש כמפתח ראשי	1402
Timestamp	String	זמן הלכידה המדויק של החבילה ברשת	"18:31:02.455"
SourceIp	String	כתובת הרשת של הגורם השולח	"10.0.0.1"
DestinationIp	String	כתובת הרשת של תחנת היעד	"142.250.186.78"
Protocol	String	סוג פרוטוקול התקשורת בשכבת התעבורה	"TCP"
Length	Integer	גודל חבילת המידע בבתים	64

3. מילון מונים לזיהוי אנומליות (Dictionary)

מבנה נתונים מהיר המשמש את אלגוריתם החלון הקבוע בצד הסוכן לצורך ספירת חבילות מידע נכנסות וזיהוי מתקפות הצפה. המילון מאופס לחלוטין בכל שנייה על ידי טיימר המערכת.

- מפתח ייחודי: כתובת IP של השולח (מחרוזת טקסט, לדוגמה: "172.16.0.5").
- ערך (Value): מספר שלם (Integer) המייצג את מונה החבילות שהתקבלו מאותה הכתובת בשנייה הנוכחית (לדוגמה: 1550).

אחסון מבוסס קבצים (קובץ PCAP)

כאשר המשתמש בוחר לייצא את תעבורת הרשת שנלכדה בתוכנה, המידע נשמר על הדיסק הקשיח בקובץ במבנה בינארי תקני התואם לפורמט PCAP העולמי. מבנה הקובץ מוגדר ברמת השדות והבתים בצורה הבאה:

1. כותרת גלובלית (Global Header)

נכתבת פעם אחת בלבד בתחילת הקובץ ומגדירה את מאפייני הלכידה הכלליים. גודלה קבוע ועומד על 24 בתים.

שם השדה	גודל בביתם	טיפוס / תפקיד	דוגמה לערך
Magic Number	4 בתים	זיהוי הפורמט וסדר הבתים בקובץ	0xa1b2c3d4
Version Major	2 בתים	מספר גרסה ראשי של הפורמט	2
Version Minor	2 בתים	מספר גרסה משני של הפורמט	4
SnapLen	4 בתים	אורך מקסימלי ללכידה מתוך כל פאקטה	65535
Network	4 בתים	סוג שכבת הקישוריות (למשל אתרנט)	1

2. כותרת חבילה (Packet Header)

נכתבת בנפרד עבור כל פאקטה ופאקטה השמורות בקובץ, וממוקמת מיד לפני תוכן הפאקטה הגולמי.
גודלה קבוע ועומד על 16 בתים.

שם השדה	גודל בבתים	טיפוס / תפקיד	דוגמה לערך
TsSec	4 בתים	זמן הלכידה בשניות מאז תאריך הבסיס	1717521104
TsUsec	4 בתים	חלק המיקרו-שניות של זמן הלכידה	455000
InclLen	4 בתים	מספר הבתים של הפאקטה שנשמרו בפועל בקובץ	64
OrigLen	4 בתים	האורך המקורי של הפאקטה כפי שעברה ברשת	64

סקירת חולשות ואיומים במערכת

שכבת האפליקציה

- **עבודה עם בסיס נתונים ומתקפות SQL Injection:** המערכת אינה משתמשת בבסיס נתונים חיצוני, אלא שומרת את כל המידע בזמן אמת בזיכרון המקומי של המחשב ובקובצי תעבורה. מכיוון שלא מתבצעת כתיבה או הרצה של שאילתות מול בסיס נתונים, המערכת חסינה לחלוטין מפני מתקפות הזרקת קוד מהסוג הזה.
- **עבודה עם אתרי אינטרנט (Web):** המערכת מבוססת על אפליקציות שולחניות שמיועדות לעבודה פנימית ברשת הארגונית, והיא אינה חשופה לאינטרנט דרך דפדפן או שרת אינטרנט. מבנה זה מקטין משמעותית את האפשרות לפריצות מבוססות רשת נפוצות.
- **תהליך ה-Login, אימות ווידוא:** חיבור הסוכנים לשרת המרכזי מתבצע על בסיס זיהוי כתובת הרשת של הסוכן. עם החיבור הראשוני, מתבצע תהליך לחיצת יד דיגיטלי שבו השרת מעביר לסוכן מפתח ציבורי ייעודי, ובכך מוודא שרק סוכנים מורשים שקיבלו את המפתח יוכלו להעביר נתונים וסטטיסטיקות לשרת המרכזי.
- **מתקפות אדם באמצע (MITM):** קיימת סכנה שתוקף יאזין לתעבורה שבין הסוכן לשרת המרכזי כדי לגנוב מידע על הרשת. כדי למנוע איום זה, כל הודעה ונתון שנשלחים מהסוכן לשרת עוברים הצפנה אסימטרית חזקה בעזרת מפתח RSA. גם אם תוקף יירט את חבילות המידע בדרך, הוא יראה מידע מוצפן וחסר משמעות, מכיוון שרק לשרת המרכזי יש את המפתח הפרטי הדרוש לפענוח המידע.
- **מתקפות מניעת שירות (DOS / DDOS):** השרת המרכזי עלול להיות חשוף למתקפות הצפה מצד גורמים עוינים שינסו להפיל אותו על ידי שליחת בקשות מרובות. כדי להתמודד עם איום זה, השרת מבוסס על ריבוי תהליכים ועל מבני נתונים בטוחים לשימוש מקבילי (כמו ConcurrentDictionary), המאפשרים לו לקלוט ולעבד אלפי פניות במקביל מבלי להינעל או לקרוס. בצד הסוכן, קיים מנגנון אקטיבי המזהה הצפות רשת מקומיות וחוסם את התוקף ישירות בחומת האש של מערכת ההפעלה כדי להגן על משאבי המחשב.
- **העלאת קבצים ופונקציית Hash:** המערכת מייצאת נתוני תעבורה לקובצי PCAP מקומיים. כדי להבטיח שקבצים אלו לא שונים, לא נפגמו ולא עברו שינוי זדוני בעת העברתם לגורם מקצועי אחר, ניתן להפיק עבור הקובץ חתימה דיגיטלית ייעודית באמצעות פונקציית Hash (כמו SHA256) המאפשרת למשתמש לאמת את שלמות המידע ולגלות כל שינוי בקובץ.

שכבת התעבורה

- **פרוטוקול TCP ולחיצת יד משולשת:** התקשורת בין הסוכנים לשרת מבוססת על פרוטוקול TCP. הקשר מתחיל בתהליך מובנה של לחיצת יד משולשת (החלפת הודעות SYN-ACK, SYN ו-ACK) המבטיח יצירת ערוץ אמין ויציב לפני העברת מידע. פרוטוקול זה מונע איבוד מידע ומבטיח שהודעות הניהול וההתראות יגיעו לשרת בדיוק בסדר שבו הן נשלחו.
- **הצפנה בשכבת התעבורה:** למרות שפרוטוקול TCP כשלעצמו מעביר מידע בצורה גלויה, ההגנה על הנתונים מתבצעת על ידי הצפנת התוכן עצמו בשכבת האפליקציה באמצעות מפתח RSA, רגע לפני שהמידע נדחף לתוך ערוץ התקשורת. בצורה זו מושגת אמינות תעבורה לצד אבטחה מקסימלית מפני האזנות בשכבת התעבורה.

הפעלת המערכת

- **חולשות קיימות בהפעלת המערכת וטיפול בהן:** בעת קלט של נתונים מצד המשתמש (כמו הזנת ערך סף לזיהוי מתקפה או הקלדת טקסט בתיבת הסינון), קיימת סכנה של חולשות מסוג הזרקת קוד או קריסת תוכנה בעקבות קלט לא תקין. כדי לטפל בחולשות אלו, בוצע אימות קלט קפדני בכל שדות המערכת. התוכנה מוודאת שערכי הסף המוקלדים הם מספרים חיוביים בלבד, ומנקה את מחרוזות החיפוש מתווים מיוחדים. בנוסף, הקוד כולו מוגן במנגנוני טיפול בשגיאות המונעים מהתוכנה לקרוס ומבטיחים סגירה בטוחה של משאבים (כמו ערוצי תקשורת וקבצים פתוחים) במקרה של תקלה לא צפויה בזמן הריצה.

מימוש הפרויקט

ארגון הקבצים

המערכת פותחה בארכיטקטורת שרת-לקוח (Client-Server) ומחולקת לשני פרויקטים מרכזיים. הפירוט מציג את סידור התיקיות והקבצים הלוגיים המרכיבים את הפרויקט:

-  **NetworkSecuritySystem**
 -  **NetworkScanner**
 -  **Icons**
 -  ChooseDevice.xaml
 -  ChooseDevice.xaml.cs
 -  Scanner.xaml
 -  Scanner.xaml.cs
 -  ProtocolBarsVM.cs
 -  PacketScanner.cs
 -  PacketQueue.cs
 -  Statistics.cs
 -  AnomalyBlocker.cs
 -  PcapHandler.cs
 -  **ScannerServer**
 -  MainWindow.xaml
 -  MainWindow.xaml.cs
 -  ConnectedAgent.cs

סקירת מודולים ומחלקות

מודולים ומחלקות מיובאים (ספריות חיצוניות)

המערכת משלבת מספר תשתיות וספריות קוד מובנות וחיצוניות לצורך ביצוע הפעולות הטכניות המורכבות ברשת וניהול האבטחה:

- **SharpPcap**: ספריית רשת חיצונית המאפשרת גישה ישירה לכרטיסי הרשת הפיזיים במחשב, ניהול התקני הלכידה ואיסוף חבילות המידע הגולמיות בזמן אמת.
- **PacketDotNet**: ספרייה חיצונית המשמשת לפענוח המבנה הפנימי של חבילות המידע שנלכדו וחילוץ השדות הרלוונטיים (כמו כתובות IP וסוג הפרוטוקול).
- **System.Net.Sockets**: תשתית מובנית של סביבת הפיתוח המשמשת ליצירת חיבורי רשת יציבים מבוססי פרוטוקול TCP לצורך העברת נתונים בין הסוכנים לשרת המרכזי.
- **System.Security.Cryptography**: ספריית האבטחה המובנית של המערכת, המספקת את כלי ההצפנה הדרושים למימוש אלגוריתם RSA, יצירת מפתחות ופענוח נתונים.
- **System.Diagnostics**: ספריית מערכת המאפשרת למערכת לפתוח ולהריץ תהליכי רקע (Processes), המשמשת בעיקר לצורך הפעלת פקודות חומת האש (Windows Firewall).
- **System.Collections.Concurrent**: אוסף של מבני נתונים מיוחדים (כמו ConcurrentQueue ו-ConcurrentDictionary) הבטוחים לשימוש בסביבה של ריבוי תהליכים במקביל (Multi-threading), ומונעים התנגשויות בזיכרון.
- **System.Threading / System.Threading.Tasks**: תשתית המשמשת לפתיחה וניהול של תהליכונים רקע (Threads), המאפשרת למערכת להאזין לרשת, לנתח נתונים ולקבל ממשק משתמש רספונסיבי במקביל.

טבלת ריכוז מחלקות הפרויקט

צד במערכת	מחלקה (Class)	תפקיד מרכזי
סוכן קצה (Client)	ChooseDevice	ניהול מסך הבית, סריקה, סינון ורענון מחזורי של כרטיסי הרשת הפיזיים במערכת.
סוכן קצה (Client)	CaptureInterfaceItem	אריזת נתוני כרטיס הרשת ושיוך אייקונים גרפיים לצורך תצוגה נקייה במסך הבחירה.
סוכן קצה (Client)	PcapHandler	טעינה ושמירה עצמאית של קובצי תעבורה בינאריים בתקן PCAP לדיסק (מצב Offline).
סוכן קצה (Client)	PacketQueue	חוצץ (Buffer) מאובטח מרובה-תהליכונים לאחסון זמני של פאקטות ומניעת הצפת הזיכרון.
סוכן קצה (Client)	PacketScanner	מנוע הניתוח המרכזי המנהל את הלכידה (SharpPcap) וחלוקת עבודה ל-4 תהליכונים במקביל.
סוכן קצה (Client)	ProtocolBarsVM	מתווך (ViewModel) המחשב וממין את נתוני הפרוטוקולים לטובת האנימציה של הגרפים.
סוכן קצה (Client)	ProtocolInfo	ישות המייצגת עמודה בגרף ומעדכנת את התצוגה הגרפית אוטומטית בעת שינוי ערך.
סוכן קצה (Client)	Statistics	ריכוז מונים, בדיקת קצבי הזרמה לשנייה והפעלת מנגנוני הגנה אוטומטיים מפני הצפות.
סוכן קצה (Client)	AnomalyBlocker	אכיפה דינמית ויצירת חוקי חסימה מיידיים בחומת האש של מערכת ההפעלה (Windows Firewall).
סוכן קצה (Client)	Scanner	מסך התצוגה הראשי, הרצת הגנת ה-Evil Twin האלחוטית וניהול הלקוח המוצפן (RSA) מול השרת.
שרת מרכזי (Server)	MainWindow	האזנה אסינכרונית לסוכנים, ניהול מפתחות ההצפנה (RSA) וניתוב הנתונים ללוח הבקרה.
שרת מרכזי (Server)	ConnectedAgent	מודל נתונים המייצג את מצבו, התראותיו ומוני הפרוטוקולים של כל סוכן קצה מחובר.
שרת מרכזי (Server)	ProtocolStat	ישות המייצגת מונה של פרוטוקול בודד עבור סוכן ספציפי לצורך הצגתו בטבלאות השרת.

פירוט המחלקות - צד לקוח (מחשב קצה)

המחלקה: PcapHandler

- **תפקיד המחלקה:** ניהול, טעינה ושמירה של קובצי תעבורת רשת בינאריים בתקן PCAP הרשמי. המחלקה מבצעת קריאה וכתיבה של כותרות הפרוטוקולים ברמת הבית (Byte-level) באופן עצמאי לחלוטין, ומאפשרת למערכת לנתח קובצי תעבורה שהוקלטו מראש.

● תכונות במחלקה:

- **PcapMagicNumber** (קבוע מסוג uint): ערך בינארי קבוע (0xa1b2c3d4) המשמש לחתימת הקובץ ומאפשר לוודא כי הקובץ הנטען הוא קובץ PCAP תקין.
- **UnixEpoch** (סטטי מסוג DateTime): נקודת הייחוס העולמית בזמן (1/1/1970) המשמשת כבסיס לחישוב זמני הגעת החבילות במיקרו-שניות בתוך הקובץ.
- **MinFrameSize** (קבוע מסוג int): הגודל המינימלי האפשרי של מסגרת (34 בתים, המורכבים מ-14 בתים של כותרת Ethernet ו-20 בתים של כותרת IPv4) המשמש לסינון חבילות פגומות.

● פעולות במחלקה:

○ SaveToPcap

- **טענת כניסה:** מחרוזת המייצגת את נתיב הקובץ לשמירה (string filePath), ורשימה של אובייקטים המכילים מידע על החבילות שנסרקו (List<PacketInfo> packets).
- **טענת יציאה:** הפעולה פותחת ערוץ כתיבה לקובץ (FileStream), מייצרת את הכותרת הגלובלית של הפורמט, וכותבת לתוך הדיסק את המידע הבינארי המלא של כל חבילה. הפעולה אינה מחזירה ערך (void).

○ LoadFromPcap

- **טענת כניסה:** מחרוזת המייצגת נתיב לקובץ PCAP קיים בדיסק (string filePath).
- **טענת יציאה:** הפעולה קוראת את הקובץ במצב בינארי, מאמתת את תקינותו, מפרקת את המבנה של כל חבילה (שדות מקור, יעד, פורטים ופרוטוקול) ומחזירה רשימה מלאה ומפוענחת של החבילות מסוג List<PacketInfo>.

המחלקה: PacketQueue

- **תפקיד המחלקה:** מחלקה סטטית המשמשת כחוצץ (Buffer) דינמי ומאובטח, המאפשר אחסון זמני של חבילות מידע המגיעות בזמן אמת מכרטיס הרשת. המחלקה אחראית לניהול קצב זרימת הנתונים ומניעת הצפת זיכרון העבודה או קפיאה של ממשק המשתמש (UI) כאשר קצב הגעת החבילות גבוה מקצב העיבוד.
- **תכונות במחלקה:**
 - **Capacity** (קבוע מסוג int): מגדיר את קיבולת התור המקסימלית (6,000 חבילות מידע) כדי למנוע צריכת זיכרון בלתי מוגבלת במערכת.
 - **HighWatermark** (קבוע מסוג int): נקודת סף עליונה (5,000 חבילות מידע) המייצגת את רמת העומס המקסימלית המומלצת בתוך התור.
 - **LowWatermark** (קבוע מסוג int): נקודת יעד תפעולית תחתונה (2,000 חבילות מידע) המייצגת מצב עבודה אופטימלי ויציב של התור.
 - **Queue** (סטטי מסוג BlockingCollection<RawCapture>): מבנה נתונים מקבילי ומתקדם מבית Microsoft, הבטוח לשימוש בסביבה מרובת תהליכים (Thread-Safe). הוא מאפשר תיאום מושלם בין תהליכון הלכידה (המפיק) לבין תהליכוני הניתוח (הצרכנים) ללא התנגשויות בזיכרון הרשת.
- **פעולות במחלקה:** מחלקה זו מנהלת את הנתונים ישירות דרך תכונת ה-Queue המובנית שלה, המחשבת ומנהלת את הוספת ושליפת האיברים בצורה אוטומטית ומאובטחת, ולכן היא אינה דורשת פונקציות תפעוליות ידניות נוספות.

המחלקה: PacketScanner

- **תפקיד המחלקה:** מנוע הניתוח, הפענוח והסריקה המרכזי של מודול תחנת הקצה. המחלקה מנהלת את ההאזנה לכרטיס הרשת הפיזי, מפעילה מערך תהליכונים מקביליים לעיבוד החבילות שנלכדו, ומבצעת פענוח עמוק לשכבות הרשת (Ethernet, IP, TCP/UDP) כדי לזהות אנומליות ושינויי כתובות בזמן אמת.

● תכונות במחלקה:

- **device** (מסוג ICaptureDevice): משתנה השומר את הרפרנס להתקן הרשת הפיזי הנבחר לצורך איסוף התעבורה.
- **uiPage** (מסוג Scanner): רפרנס לדף ממשק המשתמש הגרפי (UI) המאפשר עדכון דינמי של הנתונים על המסך והקפצת התראות.
- **allowedProtocols** (מערך מסוג string[]): רשימת הפרוטוקולים המאושרים לניתוח במערכת (TCP, UDP, ICMP, ARP, DNS, DHCP, HTTP, HTTPS).
- **stats** (מסוג Statistics): מופע של מחלקת הסטטיסטיקה המשמש לניהול מונים ובדיקת קצבי הזרמת הנתונים.
- **arpTable** (מסוג ConcurrentDictionary<string, string>): מילון מקבילי (Thread-Safe) הממפה בין כתובות IP לכתובות MAC כדי לזהות שינויים חשודים ברשת המקומית.
- **dnsMacTracker** (מסוג ConcurrentDictionary<ushort, string>): מילון מקבילי העוקב אחר מספרי עסקאות (Transaction IDs) של שאילות DNS כדי לאמת את זהות המשיב.
- **WorkerCount** (קבוע מסוג int): מגדיר את כמות תהליכוני הרקע הפועלים במקביל (4 תהליכונים) לחלוקת עומס העיבוד.

● פעולות במחלקה:

- **SetUI**
 - **טענת כניסה:** אובייקט מסוג דף הממשק הגרפי (Scanner page).
 - **טענת יציאה:** השיוך נשמר במחלקה המאפשר תקשורת ישירה אל המסך. הפעולה אינה מחזירה ערך (void).
- **Start**
 - **טענת כניסה:** אובייקט ממשק רשת פיזי פעיל (ICaptureDevice dev).
 - **טענת יציאה:** פותחת את כרטיס הרשת במצב האזנה כולל (Promiscuous), מאתחלת את תהליכוני העיבוד ותהליכון מניעת ההצפות, נרשמת לאירוע הגעת הפאקטות ומתחילה את הלכידה בפועל. הפעולה אינה מחזירה ערך (void).

StartProcessingThread ○

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** מריצה 4 משימות רקע במקביל (Tasks). כל משימה שולפת חבילות מידע מהתור באופן רציף, מפענחת אותן, בודקת חסימות ומעדכנת את רכיבי המערכת. הפעולה פרטית ואינה מחזירה ערך (void).

OnPacketArrival ○

- **טענת כניסה:** אובייקט השולח ונתוני הפאקטה הגולמית שנלכדה (object sender, PacketCapture e).
- **טענת יציאה:** פונקציית אירוע הנקראת אוטומטית בכל הגעת חבילה מהרשת, השולפת את המידע ומנסה לדחוף אותו בבטחה אל תוך התור. הפעולה פרטית ואינה מחזירה ערך (void).

BuildPacketInfo ○

- **טענת כניסה:** אובייקט חבילה מפורק (Packet packet) ואובייקט לכידה גולמי (RawCapture raw).
- **טענת יציאה:** מפענחת באופן עמוק את שכבות הפריים בעזרת ספריית PacketDotNet, מחלצת כתובות מקור/יעד, פורטים וזמנים, ומחזירה אובייקט מובנה ומאורגן מסוג PacketInfo. הפעולה פרטית.

DetectProtocol ○

- **טענת כניסה:** מערך בתים של תוכן החבילה (byte[] payload), פורט מקור (int srcPort), פורט יעד (int dstPort) ומחרוזת ברירת מחדל (string defaultProtocol).
- **טענת יציאה:** מנתחת את תוכן הבתים הראשונים (כגון זיהוי פקודות HTTP או פרוטוקול הצפנת HTTPS) או מספרי פורטים ייעודיים, ומחזירה את שם הפרוטוקול המדויק שזוהה כמחרוזת (string). הפעולה פרטית.

Stop ○

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** עוצרת את הלכידה הפיזית, סוגרת את רכיב הרשת, ומנקה את רשימות החסימה הדינמיות במערכת. הפעולה אינה מחזירה ערך (void).

HandleArp ○

- **טענת כניסה:** אובייקט חבילת מידע מהרשת (Packet packet).
- **טענת יציאה:** מבודדת חבילות מסוג ARP, משווה את כתובות ה-IP וה-MAC של השולח מול מילון ה-arpTable, ומפעילה התרעה על חריגה במידה וזוהה שינוי כתובת פיזית ל-IP קיים. הפעולה פרטית ואינה מחזירה ערך (void).

HandleDns ○

- **טענת כניסה:** מזהה העסקה של השאילתה (ushort txId), כתובת ה-IP של השולח (string sourceIp), וכתובת ה-MAC שלו (string sourceMac).
- **טענת יציאה:** מנהלת מעקב אחר מספרי עסקאות של שאילתות DNS; במידה ומתקבל מענה מאותה עסקה אך מכתובת פיזית (MAC) שונה מזו שביקשה או שענתה במקור, היא מזהה חריגת רשת ומפעילה התרעה. הפעולה פרטית ואינה מחזירה ערך (void).

RaiseAnomaly ○

- **טענת כניסה:** סוג החריגה (string type), היעד (string target), פירוט השינוי (string details), כתובת ה-IP (string ip) והפורט (int port).
- **טענת יציאה:** פונה ל-Dispatcher של ממשק המשתמש ומזריקה את פרטי החריגה ישירות אל מסך הניהול הגרפי. הפעולה פרטית ואינה מחזירה ערך (void).

המחלקה: AnomalyBlocker

- **תפקיד המחלקה:** רכיב אכיפת המדיניות הדינמי במערכת. המחלקה אחראית על יצירה, ניהול ומחיקה של חוקי חסימה דינמיים בחומת האש של מערכת ההפעלה (Windows Firewall) בזמן אמת, במטרה לחסום באופן מיידי תעבורה מכתובות רשת המפגנות התנהגות חריגה או מפרות את מדיניות הרשת.
- **תכונות במחלקה:** המחלקה מעוצבת כמחלקה תפעולית סטטית המבצעת קריאות מערכת ישירות ואינה שומרת תכונות מצב (Stateless).

- **פעולות במחלקה:**

- **Block**

- **טענת כניסה:** מחרוזת טקסט המייצגת את כתובת ה-IP לחסימה (string ip), ומספר שלם המייצג את פורט היעד לתקשורת (int port).
- **טענת יציאה:** הפעולה בודקת האם ערך הפורט גדול מ-0. במידה וכן, היא מייצרת חוק חסימה ממוקד לפרוטוקול TCP בפורט ובכתובת המבוקשים (כמו במקרה של זיהוי הצפת חבילות סנכרון). במידה והפורט אינו מוגדר (שווה ל-0), היא חוסמת את כלל התעבורה מאותה הכתובת על ידי קריאה לפעולה BlockIp. הפעולה אינה מחזירה ערך (void).

- **BlockIcmp**

- **טענת כניסה:** מחרוזת טקסט המייצגת את כתובת ה-IP של מקור התעבורה החריגה (string ip).
- **טענת יציאה:** הפעולה מנסחת ומריצה פקודת מערכת המגדירה חוק חסימה ייעודי לפרוטוקול ICMP (חבילות Ping) עבור כתובת ה-IP שנמסרה, במטרה למנוע הצפת רוחב פס. הפעולה אינה מחזירה ערך (void).

- **BlockIp**

- **טענת כניסה:** מחרוזת טקסט המייצגת את כתובת ה-IP של מקור התעבורה החריגה (string ip).
- **טענת יציאה:** הפעולה מנסחת ומריצה פקודת מערכת גורפת החוסמת לחלוטין כל סוג של תעבורת רשת נכנסת (protocol=any) מכתובת ה-IP המבוקשת (משמשת בעיקר לחסימה הרמטית בעת זיהוי שינויי כתובות או התחזות ברשת). הפעולה אינה מחזירה ערך (void).

- **ClearAllBlocks**

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** הפעולה מבצעת ניקוי ואיפוס מלא של חוקי חומת האש שנוצרו על ידי התוכנה, על ידי הרצת פקודת מחיקה גורפת עבור כל חוק המתחיל בתחילית "Block". הפעולה אינה מחזירה ערך (void).

RunNetsh ○

- **טענת כניסה:** מחרוזת טקסט המכילה את הארגומנטים והפרמטרים של פקודת ה-Firewall המבוקשת (string arguments).
- **טענת יציאה:** הפעולה משתמשת בתשתית Process.Start כדי להריץ ברמת מערכת ההפעלה את כלי הניהול netsh. הפעולה מגדירה ריצה סמויה ללא פתיחת חלון קונסול שחור למשתמש (= CreateNoWindow) ומפעילה את התהליך בפועל. הפעולה פרטית, פנימית ואינה מחזירה ערך (void).

המחלקה: Statistics

- **תפקיד המחלקה:** ריכוז הנתונים הכמותיים של תעבורת הרשת בזמן אמת, ניהול המונים עבור כל פרוטוקול בנפרד, ומעקב רציף אחר קצבי חבילות נכנסות לשנייה כדי לזהות חריגות נפח (הצפות) ולהפעיל מנגנוני הגנה אוטומטיים במערכת.

● תכונות במחלקה:

- `protocolCount_` (מסוג הסטרוקטורה ProtocolCount): משתנה פנימי המרכז את המונים המצטברים עבור כל אחד מפרוטוקולי הרשת הנתמכים (TCP, UDP, HTTP, HTTPS, ARP, ICMP, DHCP, DNS).
- `protocolLock_` (מסוג object): אובייקט חסימה המשמש לסינכרון הגישה (Locking) למוני הפרוטוקולים, המונע התנגשויות נתונים בריצה מקבילית.
- `sockets_` (מסוג Dictionary<string, SocketStats>): מילון המרכז את נתוני המונים הזמניים עבור כל ערוץ תקשורת (Socket) פעיל, על פי מפתח ייחודי של IP ופורט.
- `blockedIps_` (מסוג HashSet<string>): אוסף מהיר וייחודי המתעד את כתובות ה-IP שהפרו את מדיניות הקצב ונמצאות כרגע בתהליך הגבלה בחומת האש.
- `uiPage_` (מסוג Scanner): משתנה המאחסן רפרנס לדף הממשק הגרפי כדי לאפשר למחלקת הלוגיקה להקפיץ התראות ויזואליות למשתמש בעת זיהוי חריגה.

● פעולות במחלקה:

- `SetUI`
 - **טענת כניסה:** אובייקט מסוג דף הממשק הגרפי (Scanner page).
 - **טענת יציאה:** הפעולה משייכת את הרפרנס למשתנה המקומי כדי לאפשר שליחת התראות למסך. הפעולה אינה מחזירה ערך (void).
- `UpdateProtocolCount`
 - **טענת כניסה:** אובייקט נתוני פאקטה מפוענחת באמצעות פרמטר ייעול לקריאה בלבד (in PacketInfo info).
 - **טענת יציאה:** הפעולה נועלת את אובייקט הסינכרון, מנתחת את סוג הפרוטוקול באמצעות מבנה switch, ומקדמת ב-1 את המונה המתאים בתוך הסטרוקטורה. הפעולה אינה מחזירה ערך (void).

UpdateFloodPreventionStats ○

- **טענת כניסה:** אובייקט חבילה מפורק (Packet packet) ואובייקט נתוני פאקטה (PacketInfo info).
- **טענת יציאה:** הפעולה מחלצת את כתובת ה-IP ופורט המקור, בודקת האם מדובר בחבילת סנכרון (TCP SYN) או בחבילת בקרה (ICMP). לאחר מכן, היא מעדכנת את מוני הקצב הייעודיים של אותו מקור בתוך מילון ה-sockets_. הפעולה אינה מחזירה ערך (void).

PreventFloodanomalies ○

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** לולאת רקע אינסופית המיועדת להרצה בתהליכון נפרד (Thread). הלולאה משתרעת בהשהיה של שנייה אחת (1,000 מילי-שניות), סורקת את כל ערוצי התקשורת הפעילים במילון, ובודקת האם הם חרגו מערכי הסף המקסימליים המותרים. במידה וזוהתה חריגה, היא רושמת את הכתובת כחסומה, מקפיצה חלון התראה בממשק, וקוראת למחלקת AnomalyBlocker לצורך חסימה. בסיום, היא מאפסת את המונים לשנייה הבאה. הפעולה אינה מחזירה ערך (void).

IsBlocked ○

- **טענת כניסה:** מחרוזת המייצגת כתובת רשת לבדיקה (string ip).
- **טענת יציאה:** הפעולה בודקת האם הכתובת קיימת באוסף החסימות המקומי ומחזירה ערך בוליאני – אמת (true) אם הכתובת חסומה, ושקר (false) אם היא מורשית.

ClearBlockedIps ○

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** הפעולה מנקה ומאפסת לחלוטין את אוסף הכתובות החסומות המקומי (blockedIps_). הפעולה אינה מחזירה ערך (void).

GetProtocolCountSnapshot ○

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** הפעולה נועלת בבטחה את אובייקט הסינכרון ומחזירה העתק (Snapshot) של המונים הנוכחיים של כלל הפרוטוקולים במערכת, במבנה של סטרוקטורה מסוג ProtocolCount.

המחלקה: ProtocolBarsVM

- **תפקיד המחלקה:** מחלקת תיווך ארכיטקטונית מסוג ViewModel (בתבנית MVVM). היא אחראית לקשר בין מחלקת הסטטיסטיקה של הרשת לבין רכיבי הגרפים הויזואלי בממשק המשתמש. המחלקה דוגמת נתונים, מחשבת מיקומים דינמיים, ומבצעת מיון בזמן אמת המייצר אפקט אנימציה של "מרוץ גרפים" (Bar Racing) בהתאם לפרוטוקול השולט ברשת.

● תכונות במחלקה:

- **snapshotProvider_** (מסוג <Func<Statistics.ProtocolCount>): נציג (Delegate) פונקציונלי המאפשר ל-VM למשוך העתק עדכני (Snapshot) של מוני הפרוטוקולים ישירות ממחלקת הסטטיסטיקה מבלי לייצר תלות קשיחה ביניהן.
- **data_** (מערך מסוג ProtocolInfo[]): רשימה המכילה את נתוני הפרוטוקולים השונים, הערכים הדינמיים שלהם והצבעים הייעודיים שיוצגו בגרף.
- **Series** (מערך ממשק מסוג ISeries[]): מחזיק את סדרות הנתונים (מסוג RowSeries) המוזרקות ישירות לרכיב התצוגה הגרפי במסך.
- **IsMonitoring** (מסוג bool): משתנה בוליאני השולט על פעילות לולאת הניטור והעדכון ברקע.

● פעולות במחלקה:

○ StartMonitoring

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** פעולת רקע אסינכרונית (Task) המריצה לולאה אינסופית כל עוד תכונת הניטור פעילה. היא דוגמת את נתוני הסטטיסטיקה כל 250 מילי-שניות, מעדכנת את הערכים, וממיינת מחדש את המערך כדי להניע את האנימציה במסך.

○ UpdateValue

- **טענת כניסה:** מחרוזת המייצגת את שם הפרוטוקול (string name) ומספר שלם המייצג את ערכו החדש (int newValue).
- **טענת יציאה:** הפעולה מחפשת במערך הנתונים את האובייקט המתאים ומעדכנת את ערכו הפנימי. הפעולה פרטית ואינה מחזירה ערך (void).

○ FetchEmpty

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** פעולת עזר סטטית המאתחלת מערך ראשוני מאופס של כל שמונת הפרוטוקולים הנתמכים, ומשייכת לכל אחד מהם את צבע התצוגה שלו. מחזירה מערך מסוג ProtocolInfo[].

המחלקה: ProtocolInfo

- **תפקיד המחלקה:** מחלקת ישות (Entity Class) היורשת מהמחלקה ObservableValue. היא משמשת כמבנה נתונים חכם המייצג עמודת פרוטוקול בודדת, ומסוגלת להודיע אוטומטית לרכיבי הגרף על כל שינוי בערך המספרי שלה כדי לעורר שינוי ויזואלי מיידי.
- **תכונות במחלקה:**
 - **Name** (מסוג string): שם הפרוטוקול (למשל "TCP" או "HTTPS") שיוצג כתווית על גבי הגרף.
 - **Paint** (מסוג SolidColorPaint): אובייקט גרפי הקובע את צבע המילוי הייחודי של העמודה בגרף.

המחלקה: Scanner

- **תפקיד המחלקה:** מחלקת התצוגה והניהול המרכזית של ממשק המשתמש (View) באפליקציה. המחלקה אחראית על הזרמת נתוני החבילות למסך, וניהול ערוץ תקשורת מאובטח ומוצפן אסימטרית (RSA) לשליחת נתונים והתראות חריגה לשרת המרכזי.
- **תכונות במחלקה:**
 - **scanner** (מסוג PacketScanner): מופע של מנוע הסריקה המשמש לחיבור שכבת הלוגיקה והאזנה לכרטיס הרשת ישירות מול הממשק הגרפי.
 - **serverClient_** (מסוג TcpClient): רכיב רשת המשמש לניהול וייצוב חיבור ה-TCP מול תשתית הניהול המרכזית של השרת.
 - **rsaClient_** (מסוג RSACryptoServiceProvider): תשתית הצפנה אסימטרית המשמשת לטעינת המפתח הציבורי של השרת וביצוע הצפנה גלובלית לכל הודעה היוצאת לרשת.
 - **baselineBssid_** (מסוג string): שומר את כתובת ה-MAC הפיזית האמיתית של נקודת הגישה האלחוטית (BSSID) בעת תחילת הריצה, לצורך השוואה ומניעת מניפולציות רשת.
 - **pendingPackets_** (מסוג ConcurrentQueue<PacketInfo>): תור ייעודי מהיר המרכז חבילות ממתינות כדי להזריק אותן למסך במקצים מרוכזים (Flushing) למניעת עומס על ה-UI Thread.
- **פעולות במחלקה:**
 - **ConnectToCentralServer**
 - **טענת כניסה:** מחרוזת המייצגת את כתובת ה-IP של השרת המרכזי (string serverIp) ומספר שלם של פורט ההאזנה (int port).
 - **טענת יציאה:** פעולה אסינכרונית היוצרת סוקט תקשורת מול השרת, קוראת בצורה מאובטחת את המפתח הציבורי שלו במבנה XML, ומגדירה את רכיב ה-RSA לצורך תחילת עבודה מוצפנת. הפעולה אינה מחזירה ערך (void).
 - **SendMessageToServer**
 - **טענת כניסה:** מחרוזת טקסט המכילה את תוכן ההודעה הגולמית או הסטטיסטיקה שיש להעביר (string message).
 - **טענת יציאה:** הפעולה הופכת את מחרוזת הטקסט למערך בתים, מצפינה אותו בעזרת המפתח הציבורי של השרת (אלגוריתם RSA), ממירה את התוצאה המוצפנת למחרוזת Base64 בטוחה, ומשגרת אותה על גבי ערוץ ה-TCP. הפעולה אינה מחזירה ערך (void).
 - **StartEvilTwinDefense**
 - **טענת כניסה:** אין פרמטרים.
 - **טענת יציאה:** הפעולה דוגמת את נתוני ה-BSSID וה-SSID הנוכחיים של כרטיס הרשת האלחוטי באמצעות כלי המערכת, קובעת אותם כקו הבסיס (Baseline), ומפעילה טיימר מחזורי (DispatcherTimer) הרץ כל 5 שניות. הטיימר מוודא שזהות החומרה של הנתב לא השתנתה בחשאי תחת אותו שם רשת. הפעולה אינה מחזירה ערך (void).

GetNetshValue ○

- **טענת כניסה:** מחרוזת טקסט המייצגת את שדה המפתח לחיפוש, כגון "BSSID" או "string key" (SSID).
- **טענת יציאה:** מריצה תהליך סמוי של כלי המערכת netsh wlan show interfaces, מנתחת את פלט הטקסט שחזר שורה אחר שורה, ומחלצת ומחזירה את הערך המדויק התואם למפתח המבוקש כמחרוזת (string). הפעולה פרטית.

RaiseAnomaly ○

- **טענת כניסה:** סוג החריגה (string type), רכיב היעד המושפע (string target), פירוט טכני של האירוע (string details), כתובת ה-IP החשודה (string ip) ומספר הפורט המעורב (int port).
- **טענת יציאה:** מייצרת באופן מיידי הודעת התראה מובנית ומעבירה אותה ברקע לשרת המרכזי. במקביל, היא משנה את רקע דף האפליקציה לצבע אדום ומקפיצה מסך חסימה (Alert Overlay) המציג את פרטי החריגה ומאפשר למנהל הרשת לבצע חסימה מידית בחומת האש. הפעולה אינה מחזירה ערך (void).

המחלקה: ChooseDevice

- **תפקיד המחלקה:** מחלקת הניהול והתצוגה של מסך בחירת ממשק התקשורת (View). המחלקה אחראית על סריקה דינמית של כרטיסי הרשת הזמינים במערכת, סינון מתאמים וירטואליים שאינם רלוונטיים, הצגת ההתקנים הפיזיים בצורה אינטואיטיבית למשתמש, וניתוב האפליקציה למסך הסריקה הראשי או טעינת קובצי תעבורה היסטוריים.
- **תכונות במחלקה:**
 - **selectedCaptureDevice_** (מסוג CaptureDevice): שומר את רכיב ממשק הרשת הפיזי שנבחר על ידי המשתמש מתוך הרשימה לצורך העברתו למנוע הסריקה.
 - **interfaceRefreshTimer_** (מסוג DispatcherTimer): טיימר מערכת הרץ ברקע ומפעיל בכל 5 שניות רענון אוטומטי של רשימת כרטיסי הרשת כדי לזהות חיבור או ניתוק של מתאמים (כמו כרטיסי רשת אלחוטיים בחיבור USB).
- **פעולות במחלקה:**
 - **LoadCaptureInterfaces**
 - **טענת כניסה:** אין פרמטרים.
 - **טענת יציאה:** הפעולה משתמשת בספריית SharpPcap כדי למשוך את כל רכיבי הרשת הזמינים. היא מפעילה לולאת סינון המעלימה מתאמים וירטואליים (כגון Bluetooth, Virtual, Miniport), מתאימה לכל כרטיס רשת את האייקון הוויזואלי שלו (Wi-Fi או Ethernet) ומזריקה את הרשימה המעודכנת למסך. הפעולה פרטית ואינה מחזירה ערך (void).
 - **StartScan_Click**
 - **טענת כניסה:** אובייקט השולח ופרטי אירוע הלחיצה (object sender, RoutedEventArgs).
 - **טענת יציאה:** פונקציית אירוע הנדלקת בעת לחיצה על כפתור התחלת הסריקה. היא מוודאת שנבחר התקן תקין, שומרת אותו, ומנווטת את המשתמש למסך הסריקה הראשי (Scanner) תוך העברת אובייקט ה-Device כפרמטר. הפעולה אינה מחזירה ערך (void).
 - **RefreshInterfaces**
 - **טענת כניסה:** אובייקט השולח ופרטי אירוע הטיימר (object? sender, EventArgs).
 - **טענת יציאה:** פונקציית אירוע המופעלת בצורה מחזורית על ידי הטיימר. היא מפעילה אנימציה גרפית של סיבוב כפתור הרענון וקוראת לפונקציה LoadCaptureInterfaces לעדכן כרטיסי הרשת. הפעולה פרטית ואינה מחזירה ערך (void).

OpenPcap_Click ○

- **טענת כניסה:** אובייקט השולח ופרטי אירוע הלחיצה (object sender, RoutedEventArgs e).
- **טענת יציאה:** פונקציית אירוע הפותחת חלונת מערכת רשמית לבחירת קבצים (OpenFileDialog) ומאפשרת למשתמש לבחור קובץ PCAP מהמחשב. בעת בחירת קובץ, היא שולחת אותו למחלקת העזר PcapHandler לצורך טעינה ופענוח, ומנווטת את המשתמש למסך התצוגה הראשי עם רשימת החבילות ההיסטוריות. הפעולה אינה מחזירה ערך (void).

המחלקה: CaptureInterfaceItem

- **תפקיד המחלקה:** מחלקת ישות פנימית (Entity Class) המשמשת כמבנה נתונים ייעודי ועטוף (Wrapper) עבור ממשקי הרשת. היא מחברת בין אובייקט כרטיס הרשת הגולמי של SharpPcap לבין תכונות גרפיות ותיאוריות מותאמות אישית, ובכך מאפשרת לבצע אחיזת נתונים (Data Binding) נקייה ישירות מול רכיב ה-ListView במסך.
- **תכונות במחלקה:**
 - **CaptureDevice** (מסוג ICaptureDevice?): שומר רפרנס ישיר להתקן הלכידה האמיתי של ספריית SharpPcap.
 - **InterfaceType** (מסוג string): מחרוזת המגדירה את סוג הממשק בצורה קריאה למשתמש (למשל "Ethernet", "Wi-Fi" או "Loopback").
 - **Description** (מסוג string): מחרוזת המכילה את התיאור הטכני הרשמי של יצרן כרטיס הרשת כפי שהוא מזוהה במערכת ההפעלה.
 - **IconPath** (מסוג string): נתיב יחסי לקובץ התמונה והאייקון המתאים (למשל Icons/Wifi.png), המשמש להצגה חזותית דינמית של סוג הרשת בממשק.

פירוט המחלקות - צד שרת (מחשב ראשי)

המחלקה: MainWindow (בצד השרת)

- **תפקיד המחלקה:** מחלקת הניהול, הבקרה והתצוגה המרכזית של צד השרת (& View Logic). המחלקה אחראית על פתיחת סוקט האזנה רשתי, קליטת סוכני קצה (Agents) חדשים, ניהול מנגנון ההצפנה האסימטרי (RSA) ברמת השרת הכולל הפצת מפתחות ציבוריים ופענוח הודעות, וניתוב דינמי של המידע המשודר (התרעות, מונים, קצבים) ישירות ללוח הבקרה (Dashboard) של מנהל הרשת.

● תכונות במחלקה:

- **agentMap_** (מסוג ConcurrentDictionary<string, <ConnectedAgent>): מילון מהיר ומאובטח מרובה-תהליכונים, הממפה בין כתובת ה-IP של סוכן הקצה לבין אובייקט הייצוג שלו, המאפשר גישה ושליפת נתונים מיידיית בזמן אמת.
- **Agents** (מסוג ObservableCollection<ConnectedAgent>): אוסף דינמי של סוכנים המחוברים לשרת, המקושר ב-Data Binding ישירות לרכיב התצוגה במסך (AgentsList), ומעדכן את הממשק אוטומטית בכל שינוי סטטוס.
- **rsaServer_** (מסוג RSACryptoServiceProvider): רכיב האבטחה המרכזי של השרת. הוא מייצר אוטומטית זוג מפתחות (ציבורי ופרטי) בגודל 2048 סיביות (Bits) עם עליית השרת, ואחראי על פענוח כלל המידע המוצפן המגיע מהסוכנים.

● פעולות במחלקה:

○ StartListening

- **טענת כניסה:** אין פרמטרים.
- **טענת יציאה:** פעולה אסינכרונית (Task) המאתחלת רכיב TcpListener על פורט 8888 ומפעילה לולאת האזנה אינסופית. בכל פעם שסוכן קצה מתחבר, היא מחלצת את כתובת ה-IP שלו, מייצרת עבורו ישות במערכת במידה ואינו קיים, ומזניקה תהליכון עצמאי לטיפול בו. הפעולה אינה מחזירה ערך (void).

○ HandleAgentMessages

- **טענת כניסה:** אובייקט ערוץ התקשורת הפיזי מול הלקוח (TcpClient client) ואובייקט נתוני הסוכן המקומי (ConnectedAgent agent).
 ■ **טענת יציאה:** פעולה אסינכרונית המנהלת את ערוץ התקשורת הישיר. בשלב הראשון, היא מייצאת את המפתח הציבורי של השרת במבנה XML ומשגרת אותו בצורה גלויה לסוכן. בשלב השני, היא נכנסת ללולאת קריאה רציפה, קולטת הודעות מוצפנות (בפורמט Base64), מפענחת אותן באמצעות המפתח הפרטי (rsaServer.Decrypt_), ומעבירה את הטקסט הגלוי לעיבוד. בעת ניתוק, היא מעדכנת את סטטוס הסוכן במסך. הפעולה אינה מחזירה ערך (void).

○ ProcessMessage

- **טענת כניסה:** אובייקט הסוכן הרלוונטי (ConnectedAgent agent) ומחרוזת הטקסט המפוענחת והגלויה שהגיעה מהשרת (string message).
 ■ **טענת יציאה:** הפעולה מפרקת את מחרוזת המידע לפי תו המפריד (|) ומנתחת את סוג הפקודה (Type) באמצעות מבנה switch:
 ■ **ALERT:** גורם להזרקת התרעת אבטחה מפורטת לראש רשימת האירועים של הסוכן ומשנה את מצבו ל-"ANOMALY! DETECTED".
 ■ **STATS:** מעדכן את סך כל החבילות הכללי שנסרק על ידי הסוכן.
 ■ **HEARTBEAT:** מעדכן את סטטוס הסוכן ל-"Online / Scanning" ומאשר שהקשר תקין.
 ■ **PROTOCOL_STATS:** מפרק מחרוזת מונים מורכבת (למשל TCP: 10, UDP: 5) ומעדכן או מייצר דינמית את רשימת מוני הפרוטוקולים של אותו סוכן לצורך הצגה בטבלאות. הפעולה אינה מחזירה ערך (void).

המחלקה: ConnectedAgent

- **תפקיד המחלקה:** מחלקת ישות ומודל נתונים דינמית (Entity/Model Class) המייצגת סוכן קצה בודד (Client Agent) המנוהל על ידי השרת המרכזי. המחלקה יורשת ומממשת את הממשק INotifyPropertyChanged כדי לאפשר אחיזת נתונים (Data Binding) דו-כיוונית מול ממשק המשתמש הגרפי, ומבטיחה שכל עדכון סטטוס או זיהוי חריגה שהתקבלו ברשת ישתקפו במסך הניהול באופן מיידי.

● תכונות במחלקה:

- **IP** (מסוג string): כתובת ה-IP הייחודית של תחנת הקצה המחוברת, המשמשת כמזהה החד-ערכי שלה בשרת.
- **TotalPackets** (מסוג int): מונה המציג את סך כל חבילות התעבורה הגולמיות שנסרקו על ידי הסוכן מתחילת פעולתו.
- **Status** (מסוג string): מחרוזת המייצגת את מצב העבודה הנוכחי של הסוכן בזמן אמת (למשל: "Connected", "Online / Scanning", "Disconnected" או המצב הקריטי: "ANOMALY DETECTED").
- **AgentAlerts** (מסוג ObservableCollection<string>): אוסף דינמי מנוהל של מחרוזות טקסט המתעד ומציג כרונולוגית את כל התראות החריגה ואנומליות האבטחה שנשלחו מהסוכן הספציפי הזה.
- **Protocols** (מסוג ObservableCollection<ProtocolStat>): אוסף דינמי המרכז את מוני הפרוטוקולים השונים שנלכדו בתחנה, ומאפשר להציג בשרת טבלה מפורטת של הרכב התעבורה עבור כל לקוח בנפרד.

● פעולות במחלקה:

○ OnPropertyChanged

- **טענת כניסה:** מחרוזת המייצגת את שם התכונה שהשתנתה (string name) באמצעות מאפיין מערכת אוטומטי (CallerMemberName).
- **טענת יציאה:** הפעולה מדליקה את אירוע ה-PropertyChanged ומדווחת למסך ה-WPF שהמידע התעדכן ושיש לרענן את התצוגה הוויזואלית. הפעולה מוגנת ואינה מחזירה ערך (void).

המחלקה: ProtocolStat

- **תפקיד המחלקה:** מחלקת ישות ועזר פנימית המשמשת כמבנה נתונים חכם עבור רשומת פרוטוקול בודדת בשרת. היא מאפשרת לפרק את מחרוזות הסטטיסטיקה המורכבות המגיעות מהרשת ולייצג כל פרוטוקול כאובייקט עצמאי המקושר חזותית לרכיבי התצוגה בלוח הבקרה.
- **תכונות במחלקה:**
 - Name (מסוג string): שם פרוטוקול התקשורת המפוענח (למשל: TCP, UDP, ARP, DNS וכו').
 - Count (מסוג int): כמות חבילות המידע המצטברת השייכת לאותו פרוטוקול ספציפי.
- **פעולות במחלקה:**
 - **OnPropertyChanged**
 - **טענת כניסה:** מחרוזת המייצגת את שם התכונה שהשתנתה (string name).
 - **טענת יציאה:** מפעילה את אירוע העדכון הגרפי עבור הרשומה הבודדת בטבלה. הפעולה מוגנת ואינה מחזירה ערך (void).

פירוט בעיות אלגוריתמיות ומנגנונים מיוחדים

1. אלגוריתם ניהול, סנכרון ועיבוד תור חבילות מקבילי (Concurrent Packet Queue)

א. הסבר על היכולת והאתגר האלגוריתמי

האתגר האלגוריתמי המרכזי בעיצוב מערכת להאזנה וניתוח תעבורת רשת בזמן אמת הוא פער המהירויות (Producer-Consumer Problem). כרטיס הרשת מסוגל להזרים אלפי חבילות מידע בשנייה (היצרן - Producer), בעוד שפענוח החבילה, חילוץ הפרוטוקולים והזרקת הנתונים לממשק הגרפי דורשים זמן עיבוד ארוך יותר (הצרכן - Consumer). אם ננסה לעבד כל חבילה ברגע הגעתה על אותו תהליכון, המערכת תפספס חבילות מידע קריטיות ותקפא לחלוטין.

כדי לפתור את הבעיה הזו, יישמתי אלגוריתם המבוסס על תור אסינכרוני חוסם ומאובטח מרובה-תהליכונים (Concurrent Blocking Queue). המנגנון פועל בצורה הבאה:

1. **קליטה מהירה (Fire and Forget):** פונקציית האירוע `OnPacketArrival` רק תופסת את הפאקטה הגולמית מהכרטיס ודוחפת אותה מיד לתור הציבורי `PacketQueue` – פעולה זו לוקחת שבריר מילישנייה.

```
private void OnPacketArrival(object sender, PacketCapture e)
{
    // השגת החבילה הגולמית מהאירוע של כרטיס הרשת
    var raw = e.GetPacket();

    // ניסיון הוספה מהיר לתור המקבילי. אם התור מלא, תודפס שגיאה
    if (!PacketQueue.Queue.TryAdd(raw))
        Console.WriteLine("Queue full - packet dropped.");
}
```

2. מנגנון הצרכנים המקבילי (Multi-Worker Execution): המערכת מריצה באופן

קבוע 4 משימות רקע עצמאיות במקביל (WorkerCount = 4). משימות אלו משתמשות במתודה GetConsumingEnumerable(), אשר יודעת לחסום את התהליכון בצורה יעילה כשהתור ריק, ולשלוח חבילה בצורה בטוחה (Thread-Safe) ברגע שהיא מגיעה, ללא התנגשויות זיכרון בין התהליכונים.

3. **עיבוד ועדכון אסינכרוני:** החבילה נשלחת לפירסור וחלוקה סטטיסטית, ורק אם היא רלוונטית, היא מוזרקת לממשק המשתמש בצורה מבוקרת, מה שמבטיח ריצה חלקה לחלוטין.

```
private const int WorkerCount = 4;
```

1 reference

```
private void StartProcessingThread()
```

```
{
    // לולאה המייצרת ומפעילה את כמות תהליכונים העבודה שהוגדרה
    for (int i = 0; i < WorkerCount; i++)
    {
        Task.Run(() =>
        {
            // לולאה חכמה השולפת חבילות בצורה מאובטחת וממתינה אוטומטית אם התור ריק
            foreach (var raw in PacketQueue.Queue.GetConsumingEnumerable())
            {
                try
                {
                    // שלב הפענוח והפירסור של שכבות התקשורת
                    var packet = Packet.ParsePacket(raw.LinkLayerType, raw.Data);
                    var info = BuildPacketInfo(packet, raw);

                    // בדיקה מול רכיב החסימות הדינמי
                    if (stats.IsBlocked(info.SrcIP)) continue;

                    // עדכון המונים, פילוח הסטטיסטיקות ובדיקת אנומליות ברשת
                    stats.UpdateFloodPreventionStats(packet, info);
                    stats.UpdateProtocolCount(in info);
                    HandleArp(packet);

                    // הזרקת המידע המפוענח לממשק הגרפי במידה והפרוטוקול נתמך
                    if (info.Protocol != "UNSUPPORTED")
                    {
                        Application.Current.Dispatcher.Invoke(() =>
                        {
                            uiPage?.AddPacket(info);
                        });
                    }
                }
                catch (Exception ex)
                {
                    Console.WriteLine("Error processing packet: " + ex.Message);
                }
            }
        });
    }
}
```

2. אלגוריתם זיהוי וחילוץ פרוטוקולים בשכבת האפליקציה (Parsing)

האתגר האלגוריתמי המרכזי בניתוח חבילות רשת הוא זיהוי פרוטוקולים ברמת שכבת האפליקציה (שכבה 7 במודול OSI). בעוד שספריות פירסור סטנדרטיות (כמו PacketDotNet) מספקות זיהוי מובנה ומלא עד לשכבת התעבורה (TCP/UDP), הן אינן מפענחות באופן אוטומטי את תוכן המידע הדינמי (Payload) הנמצא בתוך החבילה.

כדי לפתור בעיה זו ולבצע פילוח סטטיסטי מדויק של פרוטוקולי הגלישה והניהול המרכזיים ברשת (כגון HTTP, HTTPS, DNS, DHCP), פותח אלגוריתם המבוסס על זיהוי משולב של פורטים ייעודיים (Well-Known Ports) וסריקת חתימות בינאריות (Byte-Level Signatures) בתוך מערך הבתים של הפאקטה:

1. **חילוץ ה-Payload:** האלגוריתם מבודד את שכבת התעבורה (TCP או UDP) ושולף מתוכה מערך בתים גולמי המייצג את תוכן ההודעה.

```
var arp = packet.Extract<ArpPacket>();
if (arp != null) protocol = "ARP";

if (packet.Extract<IcmpV4Packet>() != null ||
    packet.Extract<IcmpV6Packet>() != null)
    protocol = "ICMP";

// שלח Payload- וחילוץ הנתונים ו TCP בדיקה האם מדובר בחבילת
var tcp = packet.Extract<TcpPacket>();
if (tcp != null)
{
    srcPort = tcp.SourcePort; // שמירת פורט המקור
    dstPort = tcp.DestinationPort; // שמירת פורט היעד
    protocol = DetectProtocol(tcp.PayloadData, srcPort, dstPort, "TCP"); // לפענח Payload-העברת ה
}

// שלח Payload- וחילוץ הנתונים ו UDP בדיקה האם מדובר בחבילת
var udp = packet.Extract<UdpPacket>();
if (udp != null)
{
    srcPort = udp.SourcePort; // שמירת פורט המקור
    dstPort = udp.DestinationPort; // שמירת פורט היעד
    protocol = DetectProtocol(udp.PayloadData, srcPort, dstPort, "UDP"); // לפענח Payload-העברת ה

    // לוגיקה ייעודית עבור חבילות מסוג DNS
    if (protocol == "DNS" && srcPort == 53 && udp.PayloadData.Length >= 2)
    {
        ushort transactionID = (ushort)((udp.PayloadData[0] << 8) | udp.PayloadData[1]);
        HandleDns(transactionID, srcIP, srcMac);
    }
}
```

2. **אימות חתימה דינמי (Deep Packet Inspection):** בשונה מהסתמכות פשוטה על מספרי פורטים (שניתן לזיוף בקלות), האלגוריתם חוקר את הבתים הראשונים במידע. לדוגמה, בעת זיהוי פורט 80 (HTTP), האלגוריתם מוודא שבתחילת מערך הבתים מופיעות מחרוזות טקסט של פקודות רשת רשמיות כגון "GET", "POST", או "HTTP". בעת בדיקת פורט 443 (HTTPS), האלגוריתם מחפש את הבית 0x16 (המייצג את תחילת לחיצת היד של פרוטוקול ההצפנה - TLS Handshake).

```
private string DetectProtocol(byte[] payload, int srcPort, int dstPort, string defaultProtocol)
{
    if (payload == null || payload.Length == 0)
        return "TCP";

    // לפי חתימת בתים בינארית של HTTPS שלב אימות החתימה הדינמי: זיהוי פרוטוקול
    if (payload.Length >= 3 && payload[0] == 0x16 && payload[1] == 0x03)
        return "HTTPS";

    // לפי מילות מפתח בבתים הראשונים HTTP שלב אימות החתימה הדינמי: זיהוי פרוטוקול
    if (payload.Length >= 4)
    {
        string startStr = System.Text.Encoding.ASCII.GetString(payload, 0, 4).ToUpper();
        if (startStr.StartsWith("GET") || startStr.StartsWith("POST") ||
            startStr.StartsWith("HEAD") || startStr.StartsWith("HTTP"))
            return "HTTP";
    }

    // במידה ולא נמצאה חתימה ייחודית בבתים, סיווג הפרוטוקול יתבצע על פי מספרי פורטים נפוצים
    int port = (srcPort <= 1023) ? srcPort : dstPort;
    return port switch
    {
        53 => "DNS",
        67 or 68 => "DHCP",
        80 => "HTTP (Port)",
        443 => "Unknown (443)",
        _ => defaultProtocol
    };
}
```

3. **סיווג אסינכרוני:** לאחר שפונקציית הזיהוי מחזירה את שם הפרוטוקול הסופי (למשל "HTTPS" או "ARP"), האלגוריתם מוודא שהפרוטוקול נתמך במערכת. לאחר מכן, האובייקט המלא PacketInfo נבנה ומוחזר לחתיכת הרקע (Background Thread) שמנהלת את הריצה האסינכרונית. משם, הנתון מועבר למחלקת הסטטיסטיקה כדי לעדכן את המונים המצטברים בזיכרון בבטחה, ובמקביל מוזרק בצורה אסינכרונית ומבוקרת לתוך מסך התצוגה הגרפי של המשתמש בעזרת מנגנון ה-Dispatcher.

```
// בדיקה האם הפרוטוקול שפוענח מותר ונתמך על ידי רשימת הסינון של המערכת
if (!allowedProtocols.Contains(protocol))
    protocol = "UNSUPPORTED";

// (Local Time) השגת חותמת הזמן וסידור אזור הזמן המקומי
var ts = raw.Timeval.Date;
if (ts.Kind == DateTimeKind.Unspecified)
    ts = DateTime.SpecifyKind(ts, DateTimeKind.Utc).ToLocalTime();
else
    ts = ts.ToLocalTime();

// UI-לצורך עדכון מונים והזרקה ל (PacketInfo) בנייה והחזרה של ישות המידע המלאה
return new PacketInfo
{
    Timestamp = ts,
    SrcIP = srcIP,
    DstIP = dstIP,
    Length = raw.Data.Length,
    Protocol = protocol,
    SrcPort = srcPort,
    DstPort = dstPort,
    MacSrc = srcMac,
    MacDst = dstMac
};
```

מסמך בדיקות

מס' בדיקה	רכיב נבדק	תיאור הבדיקה	קלט (פעולת הבדיקה)	תוצאה צפויה	תוצאה בפועל (Pass/Fail)
1	ChooseDevice	זיהוי וסינון דינמי של כרטיסי רשת.	הרצת התוכנה ופתיחת מסך הבית.	המערכת מציגה רק מתאמים פיזיים פעילים (Wi-Fi/Ethernet) עם אייקון מתאים, ומסננת רשתות וירטואליות.	עבר בהצלחה
2	PcapHandler	טעינת קובץ תעבורה היסטורי מהדיסק.	לחיצה על "Open" ובחירת קובץ pcap. קיים.	הקובץ נקרא ברמת הבית, מפורסר בהצלחה, והחבילות מוזרקות ישירות לטבלת התצוגה במצב Offline.	עבר בהצלחה
3	PacketQueue / PacketScanner	עמידה בעומס ועיבוד מקבילי בזמן אמת.	בחירת כרטיס רשת פעיל ולחיצה על "Start Scan".	התור קולט חבילות במהירות, 4 תהליכונים הרקע שולפים ומפענחים אותן במקביל ללא קפיאה של הממשק הגרפי.	עבר בהצלחה
4	ProtocolBarsVM	עדכון גרף הפרוטוקולים הדינמי.	הזרמת חבילות מסוגים שונים (למשל גלישה באתר HTTPS).	הגרף דוגם את המונים כל 250 מילי-שניות, ממין מחדש את העמודות ומציג "מרוץ גרפים" באנימציה חלקה.	עבר בהצלחה
5	(Scanner UI)	תגובת ממשק המשתמש בעת זיהוי חריגה.	הפעלת פונקציות זיהוי חריגה בעקבות זיהוי חריגת רשת או הצפה.	רקע המסך משתנה מיד לצבע אדום, מוקפץ חלון חסימה (Alert Overlay) המציג את פרטי האירוע, והתעבורה נחסמת.	עבר בהצלחה
6	Scanner / MainWindow	תקשורת מוצפנת אסימטרית מול השרת.	שליחת התראה או סטטיסטיקה מלוח הלקוח לשרת המרכזי.	הלקוח מצפין את המידע ב-RSA בעזרת המפתח הציבורי של השרת, השרת מפענח בהצלחה בעזרת המפתח הפרטי ומציג את הטקסט.	עבר בהצלחה
7	AnomalyBlocker	אכיפת מדיניות חסימה מול חומת האש.	קריאה לפונקציות החסימה עבור כתובת IP חורגת.	התוכנה מריצה ברקע פקודת netsh advfirewall חוק חסימה רשמי ב-Windows Firewall עבור אותה כתובת.	עבר בהצלחה

מדריך למשתמש

עץ קבצים עיקריים

—  NetworkSecuritySystem	— פתרון המערכת המלא (Solution)
—  NetworkScanner	— פרויקט סוכן הקצה והניטור (Client)
—  Icons	— משאבים גרפיים לממשק המשתמש
—  ChooseDevice.xaml	— ממשק עיצוב ויזואלי למסך הבית
—  ChooseDevice.xaml.cs	— לוגיקת סריקה וסינון כרטיסי רשת פיזיים
—  Scanner.xaml	— ממשק עיצוב ויזואלי למסך הניטור הראשי
—  Scanner.xaml.cs	— לוגיקת מסך הניטור, הגנת רשת והתחברות לשרת
—  ProtocolBarsVM.cs	— מתווך נתונים האחראי על מיון ואנימציות הגרפים
—  PacketScanner.cs	— מנוע הלכידה המרכזי ופירסור פרוטוקולים עמוק
—  PacketQueue.cs	— חוצץ זיכרון אסינכרוני ומאובטח מרובה-תהליכונים
—  Statistics.cs	— ניהול מוני פאקטות ובדיקת קצבי הזרמה לשנייה
—  AnomalyBlocker.cs	— רכיב אכיפה וחסימת כתובות ב-Windows Firewall
—  PcapHandler.cs	— מנגנון טעינה ושמירה של קובצי תעבורה לדיסק
—  ScannerServer	— פרויקט השרת והניהול המרכזי (Server)
—  MainWindow.xaml	— ממשק עיצוב ויזואלי ללוח הבקרה של השרת
—  MainWindow.xaml.cs	— האזנה אסינכרונית לסוכנים, הצפנה וניתוב מידע
—  ConnectedAgent.cs	— מודל נתונים המייצג סוכן ומעדכן את המסך בזמן אמת

התקנת המערכת

שלב א': התקנת מנהל התקן ללכידת חבילות רשת (Npcap)

ברירת המחדל של מערכת ההפעלה Windows חוסמת את הגישה למידע הגולמי שעובר בכרטיס הרשת. המערכת שלך משתמשת בספריית SharpPcap אשר דורשת התקנה של רכיב מערכת בשם Npcap. רכיב זה מעביר את כרטיס הרשת למצב האזנה מורחב (Promiscuous Mode).

1. יש להוריד את התקנת הגרסה העדכנית ביותר של הרכיב מהאתר הרשמי:
 ○ קישור להורדה: [Npcap Official Website \(https://npcap.com\)](https://npcap.com)
2. יש להריץ את קובץ ההתקנה (npcap-X.XX.exe) כמנהל מערכת.
3. דגש חשוב במהלך ההתקנה: בשלב סימון Vn (הגדרות ההתקנה), יש לוודא שמסומנת האפשרות: "Support raw 802.11 traffic (and monitor mode) for wireless adapters" (תמיכה בתעבורת אלחוטית גולמית עבור כרטיסי Wi-Fi).
4. לסיים את ההתקנה (אין צורך להפעיל מחדש את המחשב).

שלב ב': התקנת סביבת הריצה של NET 8.0

התוכנה בנויה על גבי התשתית החדשה ביותר של מיקרוסופט (NET 8.0). במידה ומחשב הניטור אינו כולל סביבה זו, יש להתקינה:

- קישור להורדה: [NET 8.0 Runtime Download](#)
- יש לבצע הורדה תחת העמודה של NET Desktop Runtime (התואם למערכת ההפעלה Windows שלך, לרוב x64).

סוגי משתמשים ואופן הפעלת המערכת

הגדרת משתמשי המערכת והרשאות נדרשות

המערכת מיועדת לשימוש על ידי מנהל רשת, ארכיטקט אבטחה או אנליסט בסייבר. משתמשי הקצה הפשוטים בארגון אינם מפעילים את התוכנה. מנהל המערכת מריץ את אפליקציית השרת המרכזית בעמדת הבקרה שלו, ומפעיל את אפליקציית הסוכן (הלקוח) בעמדות הניטור ברשת כדי לאסוף נתונים ולשמור על אבטחת התחנות בזמן אמת.

מדריך הפעלה מלא שלב אחר שלב

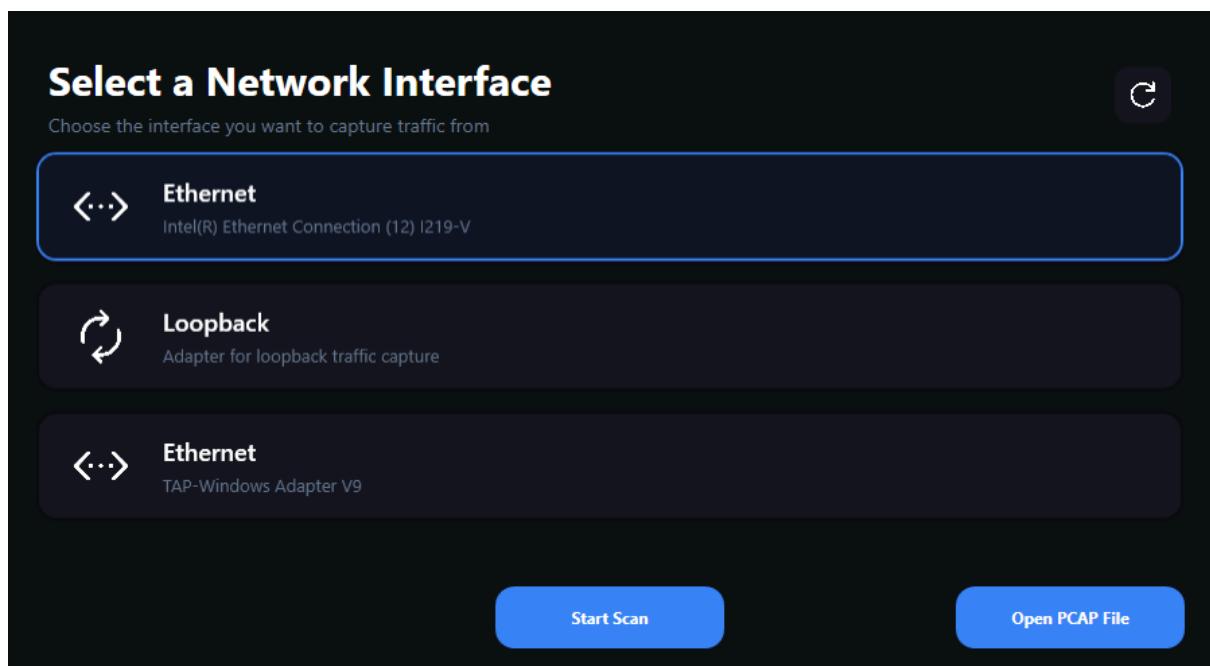
שלב 1: הפעלת השרת המרכזי

1. כניסה לתיקיית קובצי הריצה של השרת: יש להיכנס אל נתיב מערכת הקבצים של רכיב הניהול ולהפעיל את הקובץ  `ScannerServer.exe`
2. המגננה הראשונית והאזנה: עם פתיחת מסך לוח הבקרה הראשי של השרת, הטבלה המרכזית תופיע כשהיא ריקה, תוך הצגת הודעה מערכת המציינת כי מתבצעת המתנה אקטיבית להתחברות של סוכני קצה מרוחקים. בשלב זה, השרת מתחיל להאזין באופן אוטומטי בפורט הרשת 8888 ומכין את רכיב ההצפנה לקבלת הדיווחים.

CENTRAL SERVER
Distributed Network Monitoring System

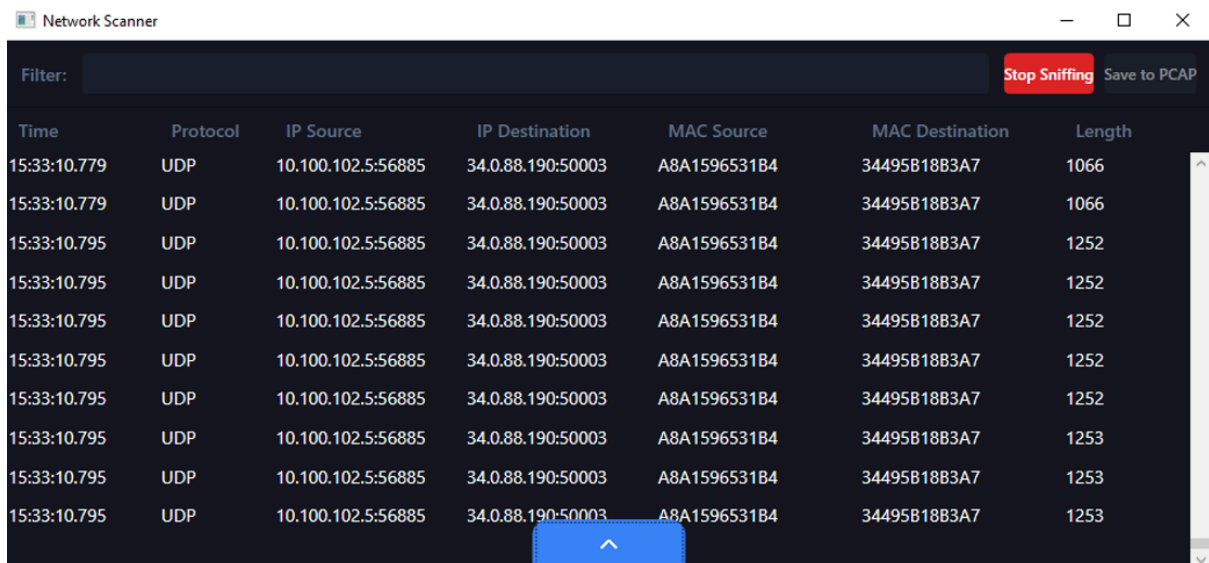
שלב 2: הפעלת הסוכן ובחירת כרטיס רשת

1. כניסה לתיקיית קובצי הריצה של השרת: יש להיכנס אל נתיב מערכת הקבצים של רכיב הניהול ולהפעיל את הקובץ  NetworkScanner.exe
2. לבחור באפשרות "הפעל כמנהל מערכת" (Run as Administrator).
(במידה ויבוקש, על מנת שלתוכנה תהיה הרשאה לגשת לחומת האש של Windows)
3. זיהוי רכיבי החומרה: עם עליית האפליקציה, יוצג מסך הבית ובו רשימה דינמית ומסונגת של כלל כרטיסי הרשת הפיזיים הפעילים בתחנה (כגון מתאמי Wi-Fi או Ethernet) המלווים באייקונים חזותיים תואמים.
4. בחירת ערוץ הלכידה: יש ללחוץ על כרטיס הרשת הרלוונטי שדרכו מתבצעת תעבורת האינטרנט הפעילה של התחנה באותו הרגע.
5. תחילת איסוף הנתונים: יש ללחוץ על כפתור "Start Scan" על מנת להזניק את מנוע הניטור ותהליכוני הלכידה בזמן אמת.
6. חלופת ניתוח מקומי (Offline Mode): במידה ונדרש ניתוח של קובץ לכידה היסטורי, יש ללחוץ על כפתור "Open PCAP" ולבחור את הקובץ המבוקש מתוך מערכת הקבצים המקומית.



שלב 3: מסך הניטור הראשי והגרפים

1. **יצירת קשר רשתי מאובטח:** לאחר בחירת כרטיס הרשת, מתבצע מעבר אוטומטי אל מסך הניטור המרכזי של הסוכן. בשלב זה, האפליקציה מייצרת חיבור רשת מוצפן (מבוסס RSA) מול השרת המרכזי ומשדרת אות חיים (Heartbeat) לצורך אימות.
2. **מעקב תעבורה בזמן אמת:** במרכז המסך מוצגת טבלה דינמית המתעדכנת בקצב מהיר ומציגה את כלל חבילות המידע הנלכדות ברשת, כולל כתובות IP של מקור ויעד, סוג הפרוטוקול המפוענח, פורטים פעילים וכתובות פיזיות (MAC).
3. **תצוגה סטטיסטית חזותית:** בצד הממשק מוצג גרף עמודות נע ומתעדכן, המציג באנימציה רציפה את התפלגות הפרוטוקולים הפעילים ביותר ברשת באותו חלון זמן (לדוגמה: אחוזי תעבורת HTTPS לעומת DNS ו-DHCP).
4. **סינון ומיקוד מידע:** ניתן להקליד מילות מפתח ספציפיות (כמו כתובת IP מסוימת) בתיבת הסינון העליונה על מנת לבצע פילוח מיידי של השורות המוצגות.
5. **עצירה ושמירת נתונים:** לצורך הפסקת איסוף החבילות, יש ללחוץ על כפתור "Stop Sniffing". לאחר מכן, מתאפשרת לחיצה על כפתור "Save to PCAP" לצורך ייצוא ושמירה של כלל החבילות שנלכדו לקובץ תקני בדיסק.



The screenshot shows the 'Network Scanner' application window. At the top, there is a 'Filter:' input field and two buttons: 'Stop Sniffing' (red) and 'Save to PCAP' (grey). Below this is a table with the following columns: Time, Protocol, IP Source, IP Destination, MAC Source, MAC Destination, and Length. The table contains 10 rows of data, all showing UDP traffic between 10.100.102.5 and 34.0.88.190. A blue button with an upward arrow is visible at the bottom center of the table area.

Time	Protocol	IP Source	IP Destination	MAC Source	MAC Destination	Length
15:33:10.779	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1066
15:33:10.779	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1066
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1252
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1253
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1253
15:33:10.795	UDP	10.100.102.5:56885	34.0.88.190:50003	A8A1596531B4	34495B18B3A7	1253

סיכום אישי ורפלקציה על תהליך הפיתוח

תיאור תהליך העבודה, האתגרים ודרכי הפתרון

פיתוח מערכת הניטור והגנת הרשת הציב אתגרים הנדסיים מורכבים, אשר דרשו גישור על הפער שבין למידה תיאורטית לבין יישום מעשי בסביבת ריצה אמיתית. התהליך החל מאפיון המודולים השונים והגדרת פרוטוקולי התקשורת, והמשיך דרך התמודדות ממושכת עם עקרונות תקשורת נמוכי-רמה (Low-Level Networking) בסביבת פיתוח המנוהלת ב-C#.

- האתגר האלגוריתמי בעיבוד החבילות: האתגר המשמעותי ביותר בתחילת הדרך היה פער המהירויות שבין קצב הגעת חבילות המידע מכרטיס הרשת לבין קצב העיבוד שלהן בממשק הגרפי (Producer-Consumer Problem). חבילות רבות נטו ללכת לאיבוד, והממשק נטה לקפוא תחת עומס. הפתרון הושג על ידי פיתוח חוצץ זיכרון עצמאי ומאובטח (PacketQueue) המבוסס על BlockingCollection, בשילוב מערך מבוזר של ארבעה תהליכי רקע מקביליים (Worker Threads). ארכיטקטורה זו אפשרה לקלוט נתונים במהירות עצומה ללא חסימת ה-UI Thread.

תהליך הלמידה - תובנות, טכנולוגיות וכלים חדשים

מהלך העבודה על הפרויקט היווה מקפצה מקצועית משמעותית וחשף כלים ומתודולוגיות עבודה מתקדמות, אשר לא נכללו בתוכנית הלימודים הרגילה ושולבו באופן עצמאי לחלוטין:

- **טכנולוגיות הצפנה אסימטריות (RSA):** לצורך אבטחת ערוץ התקשורת שבין הסוכנים לשרת המרכזי, נלמד לעומק מנגנון ההצפנה האסימטרי באמצעות מחלקות ה-CryptoServiceProvider של .NET. נרכשה הבנה מעשית בניהול מפתחות, הפצה בטוחה של מפתח ציבורי, ופענוח נתונים מוצפנים בצד השרת בלבד כדי למנוע האזנות סתר ברשת.
- **אינטגרציה מול מערכת ההפעלה (Windows Firewall Interactivity):** פיתוח מודול האכיפה והחסימה הדינמי דרש לימוד של תהליכי מערכת (System Processes) ופקודות רשת מתקדמות של netsh. היכולת לגרום לאפליקציה שפותחה ב-WPF להשפיע באופן אקטיבי על מדיניות חומת האש של Windows היוותה אבן דרך קריטית בהבנת ממשקי מערכת הפעלה.
- **ארכיטקטורת MVVM ו-WPF Data Binding:** נרכשו מיומנויות מתקדמות בעיצוב ממשקי משתמש מורכבים, שימוש במנגנוני INotifyPropertyChanged לעדכון אוטומטי של רכיבי UI, ועבודה עם ספריות גרפים דינמיות המציגות שינויי זיכרון אסינכרוניים ללא צורך בריענון ידני.

3. ראייה לאחור, חקר עצמי וסיכום (Reflection)

במבט לאחור על כל השנה הזו ועל התהליך של הפיתוח, אני יכול להגיד בביטחון שהצלחתי להגיע לכל המטרות שהצבתי לעצמי בהתחלה. המערכת עובדת מעולה, היא יציבה ומאובטחת, וממש נהנתי לראות אותה פועלת ומתפתחת.

בחרתי בנושא הזה של סייבר ואבטחת מידע בעיקר כי זה תחום שמאוד עניין וסקרן אותי עוד לפני שהתחלתי את הפרויקט, רציתי להעמיק בתחום הרשתות. לאורך כל הדרך, למרות שהיו לא מעט קשיים ודברים שהסתבכתי איתם, באמת נהנתי מהעבודה ומהלמידה. הייתה לי תחושת סיפוק בכל פעם שהצלחתי לפתור באג או כשהצלחתי לגרום לחלק מסוים בקוד לעבוד בדיוק כמו שרציתי.

כמובן שהדרך לא הייתה חלקה לגמרי. הבעיה המשמעותית ביותר שנתקלתי בה הייתה צוואר בקבוק בהצגת החבילות. כשהרשת הייתה עמוסה, כמות הפאקטות שהגיעו הייתה גדולה בהרבה ממה שהממשק הגרפי יכול היה לעכל, ונוצר דילי שהלך וגדל עם הזמן. החבילות שהוצגו על המסך היו ישנות יותר ויותר ביחס למה שבאמת קרה ברשת באותו רגע, מה שהפך את הניטור בזמן אמת לחסר משמעות. בהתחלה ניסיתי לפתור את זה בדרכים פשוטות יותר, אבל הבנתי שצריך לחשוב על זה מחדש ברמה הארכיטקטונית. הפתרון הסופי היה הפרדה מוחלטת בין תהליך הלכידה לבין תהליך ההצגה באמצעות התור המקבילי, ביחד עם טיימר שמרענן את הממשק בקבוצות כל 50 מילישניות במקום לצייר כל פאקטה בנפרד. בנוסף, נתקלתי בבעיות בהצגה הנכונה של הסטטיסטיקות, שדרשו כוונן של אופן הסנכרון בין המונים למסך.

הפרויקט הזה נתן לי המון. למדתי ממנו לעומק איך רשתות עובדות, איך לנהל כמה תהליכונים במקביל כדי שהתוכנה לא תיתקע, ואיך להשתמש בהצפנות. להתמודד עם אתגרים כמו בניית התור המאובטח, הסינון של החבילות והצגת הגרפים בזמן אמת, לא היה פשוט בכלל אבל זה בדיוק מה שפיתח אותי, לימד אותי איך לחקור וללמוד דברים לבד, ונתן לי כלים אמיתיים שיעזרו לי מאוד בהמשך הדרך בעולם התוכנה.

ביבליוגרפיה

בטח. הנה הביבליוגרפיה מחדש בסגנון טבעי יותר:

ספריות קוד פתוח

SharpPcap היא ספריית הקוד הפתוח ששימשה ללכידת החבילות הגולמיות מכרטיס הרשת. דרכה נפתח הכרטיס במצב Promiscuous ונרשם האירוע שמופעל בכל הגעת חבילה.

<https://github.com/dotpcap/sharppcap>

PacketDotNet שימשה לפירסור שכבות הפרוטוקול מתוך החבילה הגולמית, כלומר חילוץ כתובות IP, MAC, פורטים וסוג הפרוטוקול לאובייקטים שניתן לעבוד איתם בקוד.

<https://github.com/antmicro/Packet.Net>

Wireshark שימש ככלי עזר לאימות תוצאות הלכידה במהלך הפיתוח, בעיקר להשוואה בין מה ש-NetSentinel לכד לבין מה שהופיע ב-Wireshark. <https://github.com/wireshark/wireshark>

תיעוד רשמי של Microsoft

תיעוד BlockingCollection שימש להבנת אופן השימוש בתור החסום המאובטח, בפרט המתודה GetConsumingEnumerable שמאפשרת לתהליכונים העיבוד לחכות אוטומטית כשהתור ריק.

<https://learn.microsoft.com/en-us/dotnet/api/system.collections.concurrent.blockingcollection>

תיעוד RSACryptoServiceProvider שימש ליישום מנגנון ההצפנה האסימטרית, כולל יצירת זוג מפתחות, ייצוא המפתח הציבורי בפורמט XML והצפנת ההודעות בצד הסוכן.

<https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.rsacryptoserviceprovider>

תיעוד ConcurrentDictionary שימש למימוש מספר מבני נתונים מקביליים במערכת, בפרט טבלת ה-ARP למעקב אחר כתובות MAC ומילון הסוכנים המחוברים בשרת.

<https://learn.microsoft.com/en-us/dotnet/api/system.collections.concurrent.concurrentdictionary-2>

תיעוד netsh שימש להבנת תחביר הפקודות ליצירה ומחיקה של חוקי חסימה ב-Windows Firewall בזמן ריצה.

<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/netsh>

תיעוד Task-based Asynchronous Pattern שימש להבנת אופן הרצת תהליכוני רקע מקביליים עם Task.Run ולניהול נכון של המשאבים.

<https://learn.microsoft.com/en-us/dotnet/standard/asynchronous-programming-patterns/task-based-asynchronous-pattern-tap>

RFC

RFC 791 הוא התקן של פרוטוקול IPv4. השתמשתי בו להבנת מבנה הכותרת לצורך פירסור ידני של חבילות בתוך PcapHandler. <https://www.rfc-editor.org/rfc/rfc791>

RFC 793 הוא התקן של פרוטוקול TCP. השתמשתי בו להבנת מנגנון ה-SYN Handshake, שהוא הבסיס לזיהוי מתקפות SYN Flood במערכת. <https://www.rfc-editor.org/rfc/rfc793>

RFC 826 הוא התקן של פרוטוקול ARP. השתמשתי בו להבנת מבנה הודעות ARP לצורך זיהוי ARP Spoofing. <https://www.rfc-editor.org/rfc/rfc826>

RFC 1035 הוא התקן של פרוטוקול DNS. השתמשתי בו להבנת מבנה הודעת DNS, בפרט חילוץ Transaction ID לצורך זיהוי <https://www.rfc-editor.org/rfc/rfc1035> DNS Spoofing.

RFC 5246 הוא התקן של TLS 1.2. השתמשתי בו להבנת חתימת ה-TLS Handshake, שהיא הבסיס לזיהוי תעבורת HTTPS לפי הבית הראשון של ה-Payload. <https://www.rfc-editor.org/rfc/rfc5246>

קהילות פיתוח

לשם פתרון אתגרים הנדסיים מורכבים במערכת, חקירת תקלות קוד וגיבוש ארכיטקטורת הרשת, בוצע שימוש נרחב בפורומים מקצועיים ובמאגרי ידע גלובליים המשותפים לקהילת המפתחים בקוד פתוח. הפלטפורמות המרכזיות ששימשו כבסיס ידע תומך הן GitHub, Stack Overflow, Reddit.

נספחים

קישור ל-GitHub של הפרויקט: github.com/Noamkamri/NetworkAnalyzer

קישור לתרשים UML ועוד תרשימי זרימה: [Draw.io](https://draw.io)

קבצי קוד - צד לקוח NetworkScanner

PacketScanner.cs

```
using System;
using System.Collections.Concurrent;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using SharpPcap;
using PacketDotNet;
using System.Windows;
using System.Net.Sockets;

namespace NetworkScanner
{
    public class PacketScanner
    {
        public struct PacketInfo
        {
            public DateTime Timestamp { get; set; }
            public string SrcIP { get; set; }
            public string DstIP { get; set; }
            public int Length { get; set; }
            public string Protocol { get; set; }
            public int SrcPort { get; set; }
            public int DstPort { get; set; }
            public string MacSrc { get; set; }
            public string MacDst { get; set; }

            public string SrcEndpoint => $"{SrcIP}:{SrcPort}";
            public string DstEndpoint => $"{DstIP}:{DstPort}";
        }

        private ICaptureDevice device;
        private Scanner uiPage;
        private string[] allowedProtocols = Array.Empty<string>();
        private Statistics stats;
        ConcurrentDictionary<string, string> arpTable = new();

        // --- DNS Spoofing Variables ---
        private ConcurrentDictionary<ushort, string> dnsMacTracker =
new();
```

```

private DateTime _lastDnsAlertTime = DateTime.MinValue;
private readonly object _dnsAlertLock = new object();

public void SetUI(Scanner page)
{
    uiPage = page;
}

public void Start(ICaptureDevice dev)
{
    allowedProtocols = new[] { "TCP", "UDP", "ICMP", "ARP",
"DNS", "DHCP", "HTTP", "HTTPS" };
    device = dev;
    stats = new Statistics();

    // Wire the Scanner page into Statistics so flood alerts use
the overlay
    stats.SetUI(uiPage);

    device.Open(DeviceModes.Promiscuous, 10);

    StartProcessingThread();

    Thread floodPreventionThread =
        new Thread(() => stats.PreventFloodAnomalies())
        {
            IsBackground = true
        };
    floodPreventionThread.Start();

    device.OnPacketArrival += OnPacketArrival;
    device.StartCapture();
}

private const int WorkerCount = 4;

private void StartProcessingThread()
{
    // לולאה המייצרת ומפעילה את כמות תהליכונים העבודה שהוגדרה
    for (int i = 0; i < WorkerCount; i++)
    {
        Task.Run(() =>
        {
            // לולאה חכמה השולפת חבילות בצורה מאובטחת וממתינה
            foreach (var raw in

```

אוטומטית אם התור ריק

```

PacketQueue.Queue.GetConsumingEnumerable()
{
    try
    {
        // שלב הפענוח והפירסור של שכבות התקשורת
        var packet =
Packet.ParsePacket(raw.LinkLayerType, raw.Data);
        var info = BuildPacketInfo(packet, raw);

        // בדיקה מול רכיב החסימות הדינמי
        if (stats.IsBlocked(info.SrcIP)) continue;

        // עדכון המונים, פילוח הסטטיסטיקות ובדיקת אנומליות
ברשת
        stats.UpdateFloodPreventionStats(packet,
info);

        stats.UpdateProtocolCount(in info);
        HandleArp(packet);

        // הזרקת המידע המפוענח לממשק הגרפי במידה
והפרוטוקול נתמך
        if (info.Protocol != "UNSUPPORTED")
        {
            Application.Current.Dispatcher.Invoke(()
=>
            {
                uiPage?.AddPacket(info);
            });
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error processing packet:
" + ex.Message);
        }
    }
});
}

private void OnPacketArrival(object sender, PacketCapture e)
{
    // השגת החבילה הגולמית מהאירוע של כרטיס הרשת
    var raw = e.GetPacket();

    // ניסיון הוספה מהיר לתור המקבילי. אם התור מלא, תודפס שגיאה

```

```

        if (!PacketQueue.Queue.TryAdd(raw))
            Console.WriteLine("Queue full -- packet dropped.");
    }

    private PacketInfo BuildPacketInfo(Packet packet, RawCapture
raw)
    {
        string srcIP = "";
        string dstIP = "";
        string srcMac = "";
        string dstMac = "";
        int srcPort = 0;
        int dstPort = 0;
        string protocol = "";

        var ip = packet.Extract<IPPacket>();
        if (ip != null)
        {
            srcIP = ip.SourceAddress.ToString();
            dstIP = ip.DestinationAddress.ToString();
        }

        var eth = packet.Extract<EthernetPacket>();
        if (eth != null)
        {
            srcMac = eth.SourceHardwareAddress.ToString();
            dstMac = eth.DestinationHardwareAddress.ToString();
        }

        var arp = packet.Extract<ArpPacket>();
        if (arp != null) protocol = "ARP";

        if (packet.Extract<IcmpV4Packet>() != null ||
            packet.Extract<IcmpV6Packet>() != null)
            protocol = "ICMP";

        // שלה Payload-וחילוך הנתונים זה TCP בדיקה האם מדובר בחבילת
        var tcp = packet.Extract<TcpPacket>();
        if (tcp != null)
        {
            srcPort = tcp.SourcePort;
            // שמירת פורט המקור
            dstPort = tcp.DestinationPort;
            // שמירת פורט היעד
            protocol = DetectProtocol(tcp.PayloadData, srcPort,
dstPort, "TCP"); // לפענוח Payload-העברת ה
    }

```

```

{

שלה Payload-וחילוף הנתונים זה UDP // בדיקה האם מדובר בחבילת
var udp = packet.Extract<UdpPacket>();
if (udp != null)
{
    srcPort = udp.SourcePort;
    // שמירת פורט המקור
    dstPort = udp.DestinationPort;
    // שמירת פורט היעד
    protocol = DetectProtocol(udp.PayloadData, srcPort,
dstPort, "UDP"); // לפענוח Payload-העברת ה

DNS // לוגיקה ייעודית עבור חבילות מסוג
if (protocol == "DNS" && srcPort == 53 &&
udp.PayloadData.Length >= 2)
{
    ushort transactionID = (ushort)((udp.PayloadData[0]
<< 8) | udp.PayloadData[1]);
    HandleDns(transactionID, srcIP, srcMac);
}
}

// בדיקה האם הפרוטוקול שפוענח מותר ונתמך על ידי רשימת הסינון של
המערכת
if (!allowedProtocols.Contains(protocol))
    protocol = "UNSUPPORTED";

// השגת חותמת הזמן וסידור אזור הזמן המקומי (Local Time)
var ts = raw.Timeval.Date;
if (ts.Kind == DateTimeKind.Unspecified)
    ts = DateTime.SpecifyKind(ts,
DateTimeKind.Utc).ToLocalTime();
else
    ts = ts.ToLocalTime();

לצורך עדכון מונים (PacketInfo) בנייה והחזרה של ישות המידע המלאה ל
UI-והזרקה ל
return new PacketInfo
{
    Timestamp = ts,
    SrcIP = srcIP,
    DstIP = dstIP,
    Length = raw.Data.Length,
    Protocol = protocol,
    SrcPort = srcPort,

```

```

        DstPort = dstPort,
        MacSrc = srcMac,
        MacDst = dstMac
    };
}

private string DetectProtocol(byte[] payload, int srcPort, int
dstPort, string defaultProtocol)
{
    if (payload == null || payload.Length == 0)
        return "TCP";

    // חתימת בתיים HTTPS של אימות החתימה הדינמי: זיהוי פרוטוקול
    של TLS בינארית של
    if (payload.Length >= 3 && payload[0] == 0x16 && payload[1]
== 0x03)
        return "HTTPS";

    // מילות מפתח בתיים HTTP של אימות החתימה הדינמי: זיהוי פרוטוקול
    הראשוניים
    if (payload.Length >= 4)
    {
        string startStr =
System.Text.Encoding.ASCII.GetString(payload, 0, 4).ToUpper();
        if (startStr.StartsWith("GET") ||
startStr.StartsWith("POST") ||
            startStr.StartsWith("HEAD") ||
startStr.StartsWith("HTTP"))
            return "HTTP";
    }

    // במידה ולא נמצאה חתימה ייחודית בתיים, סיווג הפרוטוקול יתבצע על פי
    מספרי פורטים נפוצים
    int port = (srcPort <= 1023) ? srcPort : dstPort;
    return port switch
    {
        53 => "DNS",
        67 or 68 => "DHCP",
        80 => "HTTP (Port)",
        443 => "Unknown (443)",
        _ => defaultProtocol
    };
}

public void Stop()
{

```

```

        if (device != null && device.Started)
        {
            device.StopCapture();
            device.Close();
            stats?.ClearBlockedIps();
            AnomalyBlocker.ClearAllBlocks();
        }
    }

    public Statistics.ProtocolCount GetProtocolCountSnapshot()
        => stats != null ? stats.GetProtocolCountSnapshot() : new
Statistics.ProtocolCount();

    void HandleArp(Packet packet)
    {
        var eth = packet as EthernetPacket;
        if (eth == null) return;

        var arp = eth.PayloadPacket as ArpPacket;
        if (arp == null) return;

        string ip = arp.SenderProtocolAddress.ToString();
        string mac = arp.SenderHardwareAddress.ToString();

        string existingMac = arpTable.GetOrAdd(ip, mac);
        if (existingMac != mac)
            RaiseAnomaly("ARP Spoofing", ip, $"MAC changed from
{existingMac} to {mac}", ip, 0);
    }

    void HandleDns(ushort txId, string sourceIp, string sourceMac)
    {
        if (dnsMacTracker.TryGetValue(txId, out string firstMac))
        {
            if (firstMac != sourceMac)
            {
                lock (_dnsAlertLock)
                {
                    if ((DateTime.Now -
_lastDnsAlertTime).TotalSeconds > 4)
                    {
                        _lastDnsAlertTime = DateTime.Now;
                        RaiseAnomaly("DNS Spoofing", sourceIp,
                            $"MAC conflict on transaction ID
{txId:X4} -- expected {firstMac}, got {sourceMac}.",
                            sourceIp, 53);
                    }
                }
            }
        }
    }

```

```

        }
    }
    dnsMacTracker.TryRemove(txId, out _);
}
else
{
    dnsMacTracker.TryAdd(txId, sourceMac);
    Task.Run(async () =>
    {
        await Task.Delay(5000);
        dnsMacTracker.TryRemove(txId, out _);
    });
}
}

// All anomaly types funnel through here → Scanner overlay
private void RaiseAnomaly(string type, string target, string
details, string ip = "", int port = 0)
{
    Application.Current.Dispatcher.Invoke(() =>
    {
        uiPage?.RaiseAnomaly(type, target, details, ip, port);
    });
}
}
}

```

Scanner.xaml.cs

```
using SharpPcap;
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Net.Sockets;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Threading;
using System.Security.Cryptography;
using System.IO;
using System.Threading.Tasks;

namespace NetworkScanner
{
    public partial class Scanner : Page
    {
        private PacketScanner scanner;

        private ScrollViewer _packetsScrollViewer;
        private bool _followPackets = true;
        private int _ignoreScrollEvents = 0;
        private bool _isSniffing = true;

        private const int MaxDisplayedPackets = 20000;
        private static readonly TimeSpan UIFlushInterval =
TimeSpan.FromMilliseconds(50);

        private readonly ConcurrentQueue<PacketScanner.PacketInfo>
_pendingPackets = new();
        private DispatcherTimer _uiFlushTimer;
        private DispatcherTimer _statsSyncTimer;

        private int _totalScannedPackets = 0;

        private ICaptureDevice _currentDevice;

        // --- Class Members for Evil Twin Defense ---
        private DispatcherTimer _securityTimer;
        private string _baselineBssid = "";
        private string _baselineSsid = "";
    }
}
```

```

// Last anomalyer info
private string _alertIp = "";
private int _alertPort = 0;
private string _alertType = "";

/// Central server communication
private TcpClient _serverClient;
private StreamWriter _serverWriter;
private RSACryptoServiceProvider _rsaClient;

private bool hasPreviousData = false;
public ObservableCollection<PacketScanner.PacketInfo> Packets {
get; }
    = new ObservableCollection<PacketScanner.PacketInfo>();

public ProtocolBarsVM ProtocolVM { get; private set; }

public Scanner(ICaptureDevice device)
{
    InitializeComponent();

    PacketsGrid.ItemsSource = Packets;

    _currentDevice = device;

    scanner = new PacketScanner();
    scanner.SetUI(this);
    scanner.Start(device);

    StartEvilTwinDefense();

    _uiFlushTimer = new
DispatcherTimer(DispatcherPriority.Background)
    {
        Interval = UIFlushInterval
    };
    _uiFlushTimer.Tick += FlushPendingPackets;
    _uiFlushTimer.Start();

    ProtocolVM = new
ProtocolBarsVM(scanner.GetProtocolCountSnapshot);
    DataContext = this;

    Loaded += Scanner_Loaded;
    Unloaded += Scanner_Unloaded;

```

```

// Connect to the central management server
ConnectToCentralServer("127.0.0.1", 8888);

// Timer for Protocol Stats
_statsSyncTimer = new
DispatcherTimer(DispatcherPriority.Background)
{
    Interval = TimeSpan.FromSeconds(1)
};

_statsSyncTimer.Tick += (s, e) =>
{
    if (!_isSniffing) return;
    var counts = scanner.GetProtocolCountSnapshot();
    string statsMsg =
$"PROTOCOL_STATS|TCP:{counts.TCP},UDP:{counts.UDP},HTTP:{counts.HTTP},HT
TPS:{counts.HTTPS},ARP:{counts.ARP},ICMP:{counts.ICMP},DHCP:{counts.DHCP
},DNS:{counts.DNS}";
    Task.Run(() => SendMessageToServer(statsMsg));
};
_statsSyncTimer.Start();
}

public Scanner(List<PacketScanner.PacketInfo> packets)
{
    InitializeComponent();

    PacketsGrid.ItemsSource = Packets;

    foreach (var packet in packets)
        Packets.Add(packet);

    StopButton.IsEnabled = false;
    StopButton.Content = "Stopped";
    StopButton.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#1E2535"));

    SavePcapButton.IsEnabled = true;

    ProtocolVM = new ProtocolBarsVM(() => new
Statistics.ProtocolCount());
    DataContext = this;

    Loaded += Scanner_Loaded;
    Unloaded += Scanner_Unloaded;
}

```

```
//
// EVIL TWIN DETECTION
//

private string GetNetshValue(string key)
{
    try
    {
        var startInfo = new System.Diagnostics.ProcessStartInfo
        {
            FileName = "netsh",
            Arguments = "wlan show interfaces",
            RedirectStandardOutput = true,
            UseShellExecute = false,
            CreateNoWindow = true
        };

        using (var process =
System.Diagnostics.Process.Start(startInfo))
        {
            string output = process.StandardOutput.ReadToEnd();
            foreach (string line in output.Split(new[] {
Environment.NewLine }, StringSplitOptions.None))
            {
                if (line.Contains(key) && (key != "SSID" ||
!line.Contains("BSSID")))
                {
                    int colonIndex = line.IndexOf(':');
                    if (colonIndex != -1)
                        return line.Substring(colonIndex +
1).Trim();
                }
            }
        }
        catch { }
        return "";
    }
}

private string GetCurrentBssid() =>
GetNetshValue("BSSID").ToUpper();
private string GetCurrentSsid() => GetNetshValue("SSID");
```

```

private void StartEvilTwinDefense()
{
    _baselineBssid = GetCurrentBssid();
    _baselineSsid = GetCurrentSsid();

    if (string.IsNullOrEmpty(_baselineBssid)) return;

    _securityTimer = new DispatcherTimer { Interval =
TimeSpan.FromSeconds(5) };
    _securityTimer.Tick += (s, e) =>
    {
        string currentMac = GetCurrentBssid();
        string currentSsid = GetCurrentSsid();

        if (string.IsNullOrEmpty(currentMac)) return;

        if (currentSsid == _baselineSsid && currentMac !=
_baselineBssid)
        {
            RaiseAnomaly("Evil Twin Detected", "Wi-Fi
Integrity",
                        $"You are still on '{currentSsid}', but the
hardware identity changed from {_baselineBssid} to {currentMac}!");
        }
    };
    _securityTimer.Start();
}

//

```

```

// INLINE SECURITY ALERT
//

```

```

public void RaiseAnomaly(string type, string target, string
details, string ip = "", int port = 0)
{
    // Fire and forget the network transmission
    Task.Run(() => SendMessageToServer($"ALERT|{type}|{target} -
{details}"));

    Dispatcher.Invoke(() =>
    {
        this.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#7F1D1D"));
    });
}

```

```

        _alertIp = ip;
        _alertPort = port;
        _alertType = "Anomaly Detected";

        AlertTypeLabel.Text = "Anomaly Detected";
        AlertTargetLabel.Text = target;
        AlertDetailsLabel.Text = details;
        AlertTimeLabel.Text = DateTime.Now.ToString("HH:mm:ss
dd MMM yyyy");

        AlertOverlay.Visibility = Visibility.Visible;
    });
}

public void ConnectToCentralServer(string serverIp, int port)
{
    Task.Run(async () =>
    {
        try
        {
            _serverClient = new TcpClient(serverIp, port);
            NetworkStream stream = _serverClient.GetStream();

            StreamReader reader = new StreamReader(stream,
Encoding.UTF8);
            _serverWriter = new StreamWriter(stream,
Encoding.UTF8) { AutoFlush = true };

            _rsaClient = new RSACryptoServiceProvider(2048);

            // 1. Receive Public Key from Server
            string publicKeyXml = await reader.ReadLineAsync();

            if (!string.IsNullOrEmpty(publicKeyXml))
            {
                // Load the server's lock into our RSA client
                _rsaClient.FromXmlString(publicKeyXml);

                // 2. Send an initial encrypted "Hello"
                SendMessageToServer("HEARTBEAT|0");
            }
        }
        catch { /* Server is offline */ }
    });
}

```

```

private void SendMessageToServer(string message)
{
    try
    {
        if (_serverWriter != null && _serverClient.Connected &&
_rsaClient != null)
        {
            // 1. Convert to bytes
            byte[] dataToEncrypt =
Encoding.UTF8.GetBytes(message);

            // 2. Encrypt
            byte[] encryptedData =
_rsaClient.Encrypt(dataToEncrypt, false);

            // 3. Convert to Base64
            string base64Encrypted =
Convert.ToBase64String(encryptedData);

            // 4. Send
            _serverWriter.WriteLine(base64Encrypted);
        }
    }
    catch { /* Handle disconnection */ }
}

private void AlertClose_Click(object sender, RoutedEventArgs e)
{
    AlertOverlay.Visibility = Visibility.Collapsed;
    this.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#0D0F12"));
}

private void AlertBlock_Click(object sender, RoutedEventArgs e)
{
    if (!string.IsNullOrEmpty(_alertIp))
    {
        if (_alertType.Contains("ICMP"))
            AnomalyBlocker.BlockIcmp(_alertIp);
        else if (_alertType.Contains("ARP") ||
_alertType.Contains("Evil Twin"))
            AnomalyBlocker.BlockIp(_alertIp);
        else
            AnomalyBlocker.Block(_alertIp, _alertPort);
    }
}

```

```

        AlertOverlay.Visibility = Visibility.Collapsed;
        this.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#0D0F12"));
    }

    //

```

```

    // EXISTING FEATURES (PCAP, UI, Scrolling)
    //

```

```

public void AddPacket(PacketScanner.PacketInfo packet)
{
    _pendingPackets.Enqueue(packet);
}

private void FlushPendingPackets(object sender, EventArgs e)
{
    const int maxPerFlush = 500;
    int count = 0;
    while (count < maxPerFlush && _pendingPackets.TryDequeue(out
var packet))
    {
        Packets.Add(packet);
        count++;
        _totalScannedPackets++;
    }
    int excess = Packets.Count - MaxDisplayedPackets;
    for (int i = 0; i < excess; i++) Packets.RemoveAt(0);
    if (count > 0 && _followPackets) ForceScrollToBottom();

    if (count > 0)
    {
        Task.Run(() =>
SendMessageToServer($"STATS|{_totalScannedPackets}"));
    }
}

private void Scanner_Loaded(object sender, RoutedEventArgs e)
{
    _packetsScrollViewer ??=
FindVisualChild<ScrollViewer>(PacketsGrid);
    if (_packetsScrollViewer != null)
_packetsScrollViewer.ScrollChanged += PacketsScrollViewer_ScrollChanged;
    _followPackets = true;
    ForceScrollToBottom();
}

```

```

    }

    private void Scanner_Unloaded(object sender, RoutedEventArgs e)
    {
        if (_packetsScrollViewer != null)
        _packetsScrollViewer.ScrollChanged -= PacketsScrollViewer_ScrollChanged;
        if (ProtocolVM != null) ProtocolVM.IsMonitoring = false;
        _uiFlushTimer?.Stop();
        _securityTimer?.Stop();
        _statsSyncTimer?.Stop();
    }

    private void StopButton_Click(object sender, RoutedEventArgs e)
    {
        if (_isSniffing)
        {
            // PacketScanner-עוצר את ההסנפה במחלקת ה
            // שנה StopCapture, אם הפונקציה שלך נקראת בשם אחר כמו)
            (בהתאם)

            scanner.Stop();

            _isSniffing = false;
            hasPreviousData = true;

            StopButton.Content = "Start Sniffing";
            StopButton.Background = new
            SolidColorBrush((Color)ColorConverter.ConvertFromString("#10B981")); //
            צבע ירוק

            // מדליק את האפשרות לשמור קובץ
            SavePcapButton.IsEnabled = true;
        }
        else
        {
            if (hasPreviousData)
            {
                // מעלים את הכפתור הראשי ומציג את שתי האפשרויות
                StopButton.Visibility = Visibility.Collapsed;
                spResumeOptions.Visibility = Visibility.Visible;
            }
            else
            {
                // מפעיל מחדש אם איכשהו הגענו לפה בלי נתונים
                scanner.Start(_currentDevice);
                _isSniffing = true;
                StopButton.Content = "Stop Sniffing";
            }
        }
    }

```

```

        StopButton.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#DC2626")); //
חזרה לאדום
        SavePcapButton.IsEnabled = false;
    }
}

private void BtnContinue_Click(object sender, RoutedEventArgs e)
{
    // מחזיר את התצוגה של הכפתור הראשי
    spResumeOptions.Visibility = Visibility.Collapsed;
    StopButton.Visibility = Visibility.Visible;

    StopButton.Content = "Stop Sniffing";
    StopButton.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#DC2626"));

    _isSniffing = true;
    SavePcapButton.IsEnabled = false;

    // ממשיך את ההסנפה בלי למחוק כלום מהמסך
    scanner.Start(_currentDevice);
}

private void BtnNewSession_Click(object sender, RoutedEventArgs
e)
{
    // מחזיר את התצוגה של הכפתור הראשי
    spResumeOptions.Visibility = Visibility.Collapsed;
    StopButton.Visibility = Visibility.Visible;

    StopButton.Content = "Stop Sniffing";
    StopButton.Background = new
SolidColorBrush((Color)ColorConverter.ConvertFromString("#DC2626"));

    _isSniffing = true;
    SavePcapButton.IsEnabled = false;

    // מאפס את כל הנתונים במסך ומוחק סטטיסטיקות
    Packets.Clear();
    _totalScannedPackets = 0;
    // שמאפסת סטטיסטיקות, קרא לה פה *scanner-אם יש לך פונקציה ב*

    // מתחיל הסנפה נקייה לגמרי
    scanner.Start(_currentDevice);
}

```

```

    }

    private void SavePcapButton_Click(object sender, RoutedEventArgs
e)
    {
        var packetsToSave = new List<PacketScanner.PacketInfo>();
        foreach (var item in PacketsGrid.Items)
        {
            if (item is PacketScanner.PacketInfo info)
packetsToSave.Add(info);
        }
        if (packetsToSave.Count == 0) return;
        string desktopPath =
Environment.GetFolderPath(Environment.SpecialFolder.Desktop);
        string fileName =
$"Capture_{DateTime.Now:yyyyMMdd_HH:mm:ss}.pcap";
        string fullPath = System.IO.Path.Combine(desktopPath,
fileName);
        try
        {
            PcapHandler.SaveToPcap(fullPath, packetsToSave);
            MessageBox.Show($"Saved to: {fullPath}", "Success",
MessageBoxButton.OK, MessageBoxImage.Information);
        }
        catch (Exception ex) { MessageBox.Show(ex.Message); }
    }

    private void PacketsScrollViewer_ScrollChanged(object sender,
ScrollChangedEventArgs e)
    {
        if (_packetsScrollViewer == null) return;
        bool atBottom = _packetsScrollViewer.ScrollableHeight <= 0
|| _packetsScrollViewer.VerticalOffset >=
_packetsScrollViewer.ScrollableHeight - 12;
        ScrollToBottomButton.Visibility = atBottom ?
Visibility.Collapsed : Visibility.Visible;
        if (e.ExtentHeightChange == 0 && e.ViewportHeightChange == 0
&& e.VerticalChange != 0) _followPackets = false;
        if (atBottom && userScroll_check(e)) _followPackets = true;
    }
    private bool userScroll_check(ScrollChangedEventArgs e) =>
e.ExtentHeightChange != 0 || e.VerticalChange != 0;

    private void ScrollToBottomButton_Click(object sender,
RoutedEventArgs e) { _followPackets = true; ForceScrollToBottom(); }
    private void ForceScrollToBottom() { _ignoreScrollEvents = 6;

```

```

_packetsScrollViewer?.ScrollToEnd(); }

    private static T FindVisualChild<T>(DependencyObject obj) where
T : DependencyObject
    {
        for (int i = 0; i < VisualTreeHelper.GetChildrenCount(obj);
i++)
        {
            var child = VisualTreeHelper.GetChild(obj, i);
            if (child is T t) return t;
            var childOfChild = FindVisualChild<T>(child);
            if (childOfChild != null) return childOfChild;
        }
        return null;
    }

    private void CheckFilter(object sender, RoutedEventArgs e)
    {
        string filterText = FilterTextBox.Text.Trim().ToLower();
        PacketsGrid.Items.Filter = item => {
            if (item is PacketScanner.PacketInfo p) return
string.IsNullOrEmpty(filterText) ||
p.Protocol.ToLower().Contains(filterText) ||
p.SrcIP.Contains(filterText) || p.DstIP.Contains(filterText);
            return false;
        };
    }
}
}

```

Statistics.cs

```
using PacketDotNet;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using static NetworkScanner.PacketScanner;

namespace NetworkScanner
{
    public class Statistics
    {
        private const int MAX_SYN_PER_SECOND =
Constants.MAX_SYN_PER_SECOND;
        private const int MAX_ICMP_PER_SECOND =
Constants.MAX_ICMP_PER_SECOND;

        private ProtocolCount _protocolCount;
        private readonly object _protocolLock = new();

        private readonly Dictionary<string, SocketStats> _sockets =
new();
        private readonly HashSet<string> _blockedIps = new();

        // Reference to the Scanner page so we can call RaiseAnomaly on
it
        private Scanner _uiPage;
        public void SetUI(Scanner page) => _uiPage = page;

        private struct SocketStats
        {
            public string Ip { get; set; }
            public string Protocol { get; set; }
            public ushort Port { get; set; }
            public int SYNCount { get; set; }
            public int ICMPCount { get; set; }
        }

        private static string MakeSocketKey(string ip, ushort port) =>
$"{ip}:{port}";

        public struct ProtocolCount
        {
            public int TCP, UDP, HTTP, HTTPS, ARP, ICMP, DHCP, DNS;
        }
    }
}
```

```

public void UpdateProtocolCount(in PacketInfo info)
{
    lock (_protocolLock)
    {
        switch (info.Protocol)
        {
            case "TCP": _protocolCount.TCP++; break;
            case "UDP": _protocolCount.UDP++; break;
            case "HTTP": _protocolCount.HTTP++; break;
            case "HTTPS": _protocolCount.HTTPS++; break;
            case "ARP": _protocolCount.ARP++; break;
            case "ICMP": _protocolCount.ICMP++; break;
            case "DHCP": _protocolCount.DHCP++; break;
            case "DNS": _protocolCount.DNS++; break;
        }
    }
}

public void UpdateFloodPreventionStats(Packet packet, PacketInfo
info)
{
    lock (_sockets)
    {
        string ip = info.SrcIP;
        ushort port = (ushort)info.SrcPort;
        string key = MakeSocketKey(ip, port);

        var tcp = packet.Extract<TcpPacket>();
        bool isSyn = tcp != null && tcp.Synchronize &&
!tcp.Acknowledgment;
        bool isIcmp = info.Protocol == "ICMP";

        if (_sockets.TryGetValue(key, out var socket))
        {
            if (isIcmp) { socket.Protocol = "ICMP";
socket.ICMPCount++; }
            else if (isSyn) { socket.Protocol = "TCP";
socket.SYNCount++; }
            _sockets[key] = socket;
        }
        else
        {
            _sockets[key] = new SocketStats
            {
                Ip = ip,

```

```

        Port = port,
        Protocol = isIcmp ? "ICMP" : (isSyn ? "TCP" :
info.Protocol),

        SYNCount = isSyn ? 1 : 0,
        ICMPCount = isIcmp ? 1 : 0
    };
    }
}

public void PreventFloodAnomalies()
{
    while (true)
    {
        Thread.Sleep(1000);

        lock (_sockets)
        {
            foreach (var key in _sockets.Keys.ToArray())
            {
                var socket = _sockets[key];

                if (socket.SYNCount > MAX_SYN_PER_SECOND &&
!_blockedIps.Contains(socket.Ip))
                {
                    _blockedIps.Add(socket.Ip);
                    _uiPage?.RaiseAnomaly(
                        "SYN Flood Detected",
                        $"Source: {socket.Ip}:{socket.Port}",
                        $"{socket.SYNCount} SYN packets/sec from
{socket.Ip} on port {socket.Port} -- threshold is
{MAX_SYN_PER_SECOND}/sec.",
                        socket.Ip, socket.Port
                    );
                    AnomalyBlocker.Block(socket.Ip,
socket.Port);
                }

                if (socket.ICMPCount > MAX_ICMP_PER_SECOND &&
!_blockedIps.Contains(socket.Ip))
                {
                    _blockedIps.Add(socket.Ip);
                    _uiPage?.RaiseAnomaly(
                        "ICMP Flood Detected",
                        $"Source: {socket.Ip}",
                        $"{socket.ICMPCount} ICMP packets/sec

```

```

from {socket.Ip} -- threshold is {MAX_ICMP_PER_SECOND}/sec.",
    socket.Ip, 0
);
AnomalyBlocker.BlockIcmp(socket.Ip);
}

socket.SYNCount = 0;
socket.ICMPCount = 0;
_sockets[key] = socket;
}
}
}

public bool IsBlocked(string ip) => _blockedIps.Contains(ip);

public void ClearBlockedIps() => _blockedIps.Clear();

public ProtocolCount GetProtocolCountSnapshot()
{
    lock (_protocolLock)
        return _protocolCount;
}
}
}

```

AnomalyBlocker.cs

```
using System.Diagnostics;

namespace NetworkScanner
{
    class AnomalyBlocker
    {
        // Blocks a TCP socket (SYN flood, DNS spoofing)
        // Input: ip, port
        public static void Block(string ip, int port)
        {
            if (port > 0)
            {
                // TCP rule with specific port (e.g. SYN flood)
                RunNetsh(
                    $"advfirewall firewall add rule " +
                    $"name=\"Block {ip}:{port}\" " +
                    $"dir=in action=block protocol=TCP localport={port}
remoteip={ip}"
                );
            }
            else
            {
                // No port -- block all traffic from this IP (ARP
spoofing, Evil Twin)
                BlockIp(ip);
            }
        }

        // Blocks all ICMP traffic from a specific IP (ICMP flood)
        public static void BlockIcmp(string ip)
        {
            RunNetsh(
                $"advfirewall firewall add rule " +
                $"name=\"Block ICMP {ip}\" " +
                $"dir=in action=block protocol=ICMP remoteip={ip}"
            );
        }

        // Blocks ALL traffic from a specific IP (ARP spoofing, Evil
Twin, catch-all)
        public static void BlockIp(string ip)
        {
            RunNetsh(
                $"advfirewall firewall add rule " +
```

```

        $"name=\"Block IP {ip}\" " +
        $"dir=in action=block protocol=any remoteip={ip}"
    );
}

// Removes all firewall rules created by our program
public static void ClearAllBlocks()
{
    // Delete by name prefix -- matches "Block *", "Block ICMP
    *, "Block IP *"
    foreach (var prefix in new[] { "Block " })
    {
        RunNetsh($"advfirewall firewall delete rule
name=\"{prefix}\"");
    }
}

private static void RunNetsh(string arguments)
{
    Process.Start(new ProcessStartInfo
    {
        FileName = "netsh",
        Arguments = arguments,
        UseShellExecute = false,
        CreateNoWindow = true
    });
}
}
}

```

PacketQueue.cs

```
using SharpPcap;
using System.Collections.Concurrent;

namespace NetworkScanner
{
    public static class PacketQueue
    {
        // יותר קטן = פחות "פאקטות ישנות" שיוצגו באיחור
        public const int Capacity = 6000;

        // real time-כשעוברים את זה - ננקה ישנות כדי לחזור ל
        public const int HighWatermark = 5000;
        public const int LowWatermark = 2000;

        public static BlockingCollection<RawCapture> Queue =
            new BlockingCollection<RawCapture>(boundedCapacity:
Capacity);
    }
}
```

ProtocolBarsVM.c

```
using CommunityToolkit.Mvvm.ComponentModel;
using LiveChartsCore;
using LiveChartsCore.Defaults; // REQUIRED for ObservableValue
using LiveChartsCore.Kernel;
using LiveChartsCore.Measure;
using LiveChartsCore.SkiaSharpView;
using LiveChartsCore.SkiaSharpView.Painting;
using SkiaSharp;

namespace NetworkScanner
{
    // 1. Inherit from ObservableValue so the bar width animates
    // automatically
    public class ProtocolInfo : ObservableValue
    {
        public ProtocolInfo(string name, int value, SolidColorPaint
paint)
        {
            Name = name;
            Paint = paint;
            Value = value; // Updates the ObservableValue base
        }

        public string Name { get; set; }
        public SolidColorPaint Paint { get; set; }
    }

    public partial class ProtocolBarsVM : ObservableObject
    {
        private readonly Func<Statistics.ProtocolCount>
_snapshotProvider;
        private ProtocolInfo[] _data;

        public ProtocolBarsVM(Func<Statistics.ProtocolCount>
snapshotProvider)
        {
            _snapshotProvider = snapshotProvider;
            _data = FetchEmpty();

            // 2. Create the Series ONCE. We will not recreate it, just
            // update its Values.
            var rowSeries = new RowSeries<ProtocolInfo>
            {
                Values = _data,
            }
        }
    }
}
```

```

        MaxBarWidth = 35,
        Padding = 5,
        DataLabelsPaint = new SolidColorBrush(SKColors.White),
        DataLabelsPosition = DataLabelsPosition.End,

        // MAPPING: This creates the racing effect.
        // X = Value (Length), Y = Index (Rank in the list)
        Mapping = (item, index) => new Coordinate(item.Value ??
0, index),

        DataLabelsFormatter = p =>
        {
            var item = (ProtocolInfo)p.Context.DataSource;
            return $"{item.Name} ({item.Value})";
        }
    };

    // 3. Apply Colors individually
    rowSeries.PointMeasured += point =>
    {
        if (point.Visual is null || point.Model is null) return;
        point.Visual.Fill = point.Model.Paint;
    };

    Series = new ISeries[] { rowSeries };

    XAxes = new[] { new Axis { MinLimit = 0, IsVisible = false }
};

    YAxes = new[] { new Axis { IsVisible = false } }; // We hide
Y labels because bar labels show the name

    _ = StartMonitoring();
}

public bool IsMonitoring { get; set; } = true;
public ISeries[] Series { get; set; }
public Axis[] XAxes { get; set; }
public Axis[] YAxes { get; set; }

private async Task StartMonitoring()
{
    await Task.Delay(1000);

    while (IsMonitoring)
    {
        // A. Get Data from your Statistics class

```

```

var s = _snapshotProvider();

// B. Update existing objects (Do not create new ones!)
//     Note: Using the EXACT field names from your
Statistics.cs

//MessageBox.Show(s.UDP.ToString());

UpdateValue("TCP", s.TCP);
UpdateValue("UDP", s.UDP);
UpdateValue("HTTP", s.HTTP);
UpdateValue("HTTPS", s.HTTPS);
UpdateValue("ARP", s.ARP);
UpdateValue("ICMP", s.ICMP);
UpdateValue("DHCP", s.DHCP);
UpdateValue("DNS", s.DNS);

// C. SORT the array
//     Because we use "Mapping", changing the index
triggers the animation.
_data = _data.OrderBy(x => x.Value).ToArray();

// D. Tell the chart the order changed
Series[0].Values = _data;

await Task.Delay(250);
}
}

private void UpdateValue(string name, int newValue)
{
    // Find the specific protocol object and update its value
    var item = _data.FirstOrDefault(x => x.Name == name);
    if (item != null)
    {
        item.Value = newValue;
    }
}

// Helper to initialize the list with 0 values and correct
colors
private static ProtocolInfo[] FetchEmpty() => new[]
{
    new ProtocolInfo("TCP", 0, GetPaintByName("TCP")),
    new ProtocolInfo("UDP", 0, GetPaintByName("UDP")),
    new ProtocolInfo("ICMP", 0, GetPaintByName("ICMP")),

```

```

        new ProtocolInfo("ARP", 0, GetPaintByName("ARP")),
        new ProtocolInfo("DNS", 0, GetPaintByName("DNS")),
        new ProtocolInfo("DHCP", 0, GetPaintByName("DHCP")),
        new ProtocolInfo("HTTP", 0, GetPaintByName("HTTP")),
        new ProtocolInfo("HTTPS", 0, GetPaintByName("HTTPS")),
    };

    private static SolidColorPaint GetPaintByName(string name) =>
name switch
    {
        "TCP" => new SolidColorPaint(new SKColor(0x42, 0x7C, 0xE6)),
// Blue
        "UDP" => new SolidColorPaint(new SKColor(0x2E, 0xB8, 0x7C)),
// Green
        "ICMP" => new SolidColorPaint(new SKColor(0xE2, 0x5D,
0x5D)), // Red
        "ARP" => new SolidColorPaint(new SKColor(0x9B, 0x6B, 0xE9)),
// Purple
        "DNS" => new SolidColorPaint(new SKColor(0xE0, 0x4C, 0xAA)),
// Pink
        "DHCP" => new SolidColorPaint(new SKColor(0x3B, 0xB1,
0xB9)), // Cyan
        "HTTP" => new SolidColorPaint(new SKColor(0xF2, 0xA1,
0x2A)), // Orange
        "HTTPS" => new SolidColorPaint(new SKColor(0xD6, 0xB5,
0x2A)), // Yellow
        _ => new SolidColorPaint(SKColors.Gray)
    };
    }
}

```

PcapHandler.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Net;
using System.Net.NetworkInformation;
using static NetworkScanner.PacketScanner;

namespace NetworkScanner
{
    public static class PcapHandler
    {
        // ——— Constants

        private const uint PcapMagicNumber = 0xa1b2c3d4;
        private const short PcapMajorVersion = 2;
        private const short PcapMinorVersion = 4;
        private const int PcapSnapLen = 65535;
        private const int PcapNetworkEthernet = 1;

        private const ushort EtherTypeIPv4 = 0x0800;
        private const int EthernetHeaderSize = 14;
        private const int IPv4HeaderSize = 20;
        private const int MinFrameSize = EthernetHeaderSize +
IPv4HeaderSize;

        private const byte ProtocolTCP = 6;
        private const byte ProtocolUDP = 17;
        private const byte DefaultTTL = 64;
        private const int DefaultLength = 60;

        private static readonly DateTime UnixEpoch = new DateTime(1970,
1, 1, 0, 0, 0, DateTimeKind.Utc);

        // ——— Public API

        public static void SaveToPcap(string filePath, List<PacketInfo>
packets)
        {
            using var fs = new FileStream(filePath,
```

```

        FileMode.Create);
        using var writer = new BinaryWriter(fs);

        WriteGlobalHeader(writer);

        foreach (var packet in packets)
        {
            byte[] frame = BuildFrame(packet);
            WritePacketHeader(writer, packet.Timestamp,
frame.Length);
            writer.Write(frame);
        }
    }

    public static List<PacketInfo> LoadFromPcap(string filePath)
    {
        var packets = new List<PacketInfo>();

        using var fs = new FileStream(filePath, FileMode.Open,
FileAccess.Read);
        using var reader = new BinaryReader(fs);

        ValidateAndSkipGlobalHeader(reader);

        while (fs.Position < fs.Length)
        {
            (DateTime timestamp, int savedLength) =
ReadPacketHeader(reader);

            byte[] frame = reader.ReadBytes(savedLength);
            if (frame.Length < MinFrameSize) continue;

            PacketInfo? packet = ParseFrame(frame, timestamp);
            if (packet.HasValue)
                packets.Add(packet.Value);
        }

        return packets;
    }

    // — Save Helpers

```

```

private static void WriteGlobalHeader(BinaryWriter writer)
{

```

```

        writer.Write(PcapMagicNumber);
        writer.Write(PcapMajorVersion);
        writer.Write(PcapMinorVersion);
        writer.Write(0); // Reserved
        writer.Write(0); // Reserved
        writer.Write(PcapSnapLen);
        writer.Write(PcapNetworkEthernet);
    }

    private static void WritePacketHeader(BinaryWriter writer,
    DateTime timestamp, int frameLength)
    {
        long ticks          = timestamp.ToUniversalTime().Ticks -
    UnixEpoch.Ticks;
        int  seconds        = (int)(ticks / TimeSpan.TicksPerSecond);
        int  microseconds = (int)((ticks % TimeSpan.TicksPerSecond)
    / 10);

        writer.Write(seconds);
        writer.Write(microseconds);
        writer.Write(frameLength); // Saved length
        writer.Write(frameLength); // Original length
    }

    private static byte[] BuildFrame(PacketInfo p)
    {
        using var ms = new MemoryStream();
        using var bw = new BinaryWriter(ms);

        // Ethernet header (14 bytes)
        bw.Write(ParseMacAddress(p.MacDst));
        bw.Write(ParseMacAddress(p.MacSrc));
        bw.Write(new byte[] { 0x08, 0x00 }); // EtherType: IPv4

        // IPv4 header (20 bytes)
        short length = (short)(p.Length > 0 ? p.Length :
    DefaultLength);
        byte proto = p.Protocol.ToUpper().Contains("TCP") ?
    ProtocolTCP : ProtocolUDP;

        bw.Write((byte)0x45); // Version=4, IHL=5
        bw.Write((byte)0x00); // DSCP/ECN
        bw.Write(IPAddress.HostToNetworkOrder(length));
        bw.Write((short)0); // Identification
        bw.Write((short)0); // Flags + Fragment Offset
        bw.Write(DefaultTTL);
    }

```

```

        bw.Write(proto);
        bw.Write((short)0);    // Checksum (not calculated)
        bw.Write(GetIPv4Bytes(p.SrcIP));
        bw.Write(GetIPv4Bytes(p.DstIP));

        // Transport header -- ports (4 bytes)
        bw.Write(IPAddress.HostToNetworkOrder((short)p.SrcPort));
        bw.Write(IPAddress.HostToNetworkOrder((short)p.DstPort));

        // Pad to original packet length if needed
        int padding = p.Length - (int)ms.Length;
        if (padding > 0) bw.Write(new byte[padding]);

        return ms.ToArray();
    }

    // ——— Load Helpers

```

```

private static void ValidateAndSkipGlobalHeader(BinaryReader
reader)
{
    uint magic = reader.ReadUInt32();
    if (magic != PcapMagicNumber)
        throw new InvalidDataException($"Invalid pcap file.
Expected magic 0x{PcapMagicNumber:X8}, got 0x{magic:X8}.");

    reader.ReadInt16(); // Major version
    reader.ReadInt16(); // Minor version
    reader.ReadInt32(); // Reserved
    reader.ReadInt32(); // Reserved
    reader.ReadInt32(); // SnapLen
    reader.ReadInt32(); // Network type
}

private static (DateTime timestamp, int savedLength)
ReadPacketHeader(BinaryReader reader)
{
    int seconds      = reader.ReadInt32();
    int microseconds = reader.ReadInt32();
    int savedLength  = reader.ReadInt32();
    reader.ReadInt32(); // Original length (not needed)

    var timestamp = UnixEpoch
        .AddTicks((long)seconds * TimeSpan.TicksPerSecond)

```

```

        .AddTicks((long)microseconds * 10);

    return (timestamp, savedLength);
}

private static PacketInfo? ParseFrame(byte[] frame, DateTime
timestamp)
{
    try
    {
        int offset = 0;

        // Ethernet header (14 bytes)
        string macDst = FormatMac(frame, offset); offset += 6;
        string macSrc = FormatMac(frame, offset); offset += 6;
        ushort etherType = ReadUInt16BigEndian(frame, offset);
offset += 2;

        if (etherType != EtherTypeIPv4) return null;

        // IPv4 header (20 bytes)
        offset += 1; // Version + IHL
        offset += 1; // DSCP/ECN

        short totalLength =
IPAddress.NetworkToHostOrder(BitConverter.ToInt16(frame, offset));
offset += 2;

        offset += 2; // Identification
        offset += 2; // Flags + Fragment Offset
        offset += 1; // TTL

        byte protocol = frame[offset]; offset += 1;
        string protocolName = protocol == ProtocolTCP ? "TCP"
                                : protocol == ProtocolUDP ? "UDP"
                                : $"PROTO_{protocol}";

        offset += 2; // Checksum

        string srcIP = ReadIPv4(frame, offset); offset += 4;
        string dstIP = ReadIPv4(frame, offset); offset += 4;

        // Transport ports (4 bytes)
        int srcPort = 0, dstPort = 0;
        if (frame.Length >= offset + 4)
        {
            srcPort =

```

```
(ushort)IPAddress.NetworkToHostOrder(BitConverter.ToInt16(frame,
offset)); offset += 2;
        dstPort =
(ushort)IPAddress.NetworkToHostOrder(BitConverter.ToInt16(frame,
offset));
    }

    return new PacketInfo
    {
        Timestamp = timestamp,
        MacSrc     = macSrc,
        MacDst     = macDst,
        SrcIP      = srcIP,
        DstIP      = dstIP,
        SrcPort    = srcPort,
        DstPort    = dstPort,
        Protocol   = protocolName,
        Length     = totalLength
    };
}
catch
{
    return null; // Skip malformed packets
}
}

// — Utility
```

```
/// <summary>Parses a colon-separated MAC string into 6
bytes.</summary>
private static byte[] ParseMacAddress(string mac)
    => PhysicalAddress.Parse(mac.Replace(":",
"-")).GetAddressBytes();

/// <summary>Formats 6 bytes starting at offset as a
colon-separated MAC string.</summary>
private static string FormatMac(byte[] frame, int offset)
    => string.Join(":", frame.Skip(offset).Take(6).Select(b =>
b.ToString("X2")));

/// <summary>Returns the 4 IPv4 bytes of an IP string, or
0.0.0.0 if invalid.</summary>
private static byte[] GetIPv4Bytes(string ip)
{
```

```

        if (IPAddress.TryParse(ip, out IPAddress addr) &&
            addr.AddressFamily ==
System.Net.Sockets.AddressFamily.InterNetwork)
            return addr.GetAddressBytes();

        return new byte[] { 0, 0, 0, 0 };
    }

    /// <summary>Reads 4 bytes from frame at offset as an IPv4
address string.</summary>
    private static string ReadIPv4(byte[] frame, int offset)
        => new IPAddress(new[] { frame[offset], frame[offset + 1],
frame[offset + 2], frame[offset + 3] }).ToString();

    /// <summary>Reads a big-endian UInt16 from two bytes.</summary>
    private static ushort ReadUInt16BigEndian(byte[] data, int
offset)
        => (ushort)((data[offset] << 8) | data[offset + 1]);
    }
}

```

ChooseDevice.xaml.cs

```
using SharpPcap;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media.Animation;
using System.Windows.Threading;
using Microsoft.Win32;
using System.IO;

namespace NetworkScanner
{
    public partial class ChooseDevice : Page
    {
        private ICaptureDevice? _selectedCaptureDevice;
        private DispatcherTimer _interfaceRefreshTimer;

        public class CaptureInterfaceItem
        {
            public ICaptureDevice? CaptureDevice { get; set; }
            public string InterfaceType { get; set; } = string.Empty;
            public string Description { get; set; } = string.Empty;
            public string IconPath { get; set; } = string.Empty;
        }

        public ChooseDevice()
        {
            InitializeComponent();

            LoadCaptureInterfaces();

            DeviceList.SelectionChanged += OnDeviceSelectionChanged;

            _interfaceRefreshTimer = new DispatcherTimer
            {
                Interval = TimeSpan.FromSeconds(5)
            };

            _interfaceRefreshTimer.Tick += RefreshInterfaces;
            _interfaceRefreshTimer.Start();
        }

        private void LoadCaptureInterfaces()
```

```

{
    ICaptureDevice? previouslySelectedDevice = null;

    if (DeviceList.SelectedItem is CaptureInterfaceItem
selectedItem)
    {
        previouslySelectedDevice = selectedItem.CaptureDevice;
    }

    var captureDevices = CaptureDeviceList.New();
    var interfaceItems = new List<CaptureInterfaceItem>();

    foreach (var captureDevice in captureDevices)
    {
        string description = captureDevice.Description ??
string.Empty;
        string deviceName = captureDevice.Name ?? string.Empty;

        if (description.Contains("Miniport") ||
            description.Contains("Monitor") ||
            description.Contains("Virtual") ||
            description.Contains("Filter") ||
            description.Contains("QoS") ||
            description.Contains("Bluetooth"))
        {
            continue;
        }

        string iconPath = "Icons/Ethernet.png";
        string interfaceType = "Ethernet";

        if (description.Contains("wireless",
StringComparison.OrdinalIgnoreCase) ||
            description.Contains("wifi",
StringComparison.OrdinalIgnoreCase) ||
            deviceName.Contains("wifi",
StringComparison.OrdinalIgnoreCase) ||
            deviceName.Contains("wi-fi",
StringComparison.OrdinalIgnoreCase))
        {
            iconPath = "Icons/Wifi.png";
            interfaceType = "Wi-Fi";
        }
        else if (description.Contains("loopback",
StringComparison.OrdinalIgnoreCase))
        {

```

```

        iconPath = "Icons/Loopback.png";
        interfaceType = "Loopback";
    }

    interfaceItems.Add(new CaptureInterfaceItem
    {
        CaptureDevice = captureDevice,
        InterfaceType = interfaceType,
        Description = description,
        IconPath = iconPath
    });
}

DeviceList.ItemsSource = interfaceItems;

if (previouslySelectedDevice != null)
{
    DeviceList.SelectedItem = interfaceItems.FirstOrDefault(
        i => i.CaptureDevice?.Name ==
previouslySelectedDevice.Name
    );
}

private void StartScan_Click(object sender, RoutedEventArgs e)
{
    if (DeviceList.SelectedItem is not CaptureInterfaceItem
selectedInterface ||
        selectedInterface.CaptureDevice == null)
    {
        MessageBox.Show("Please select a valid capture
interface.");
        return;
    }

    _selectedCaptureDevice = selectedInterface.CaptureDevice;
    NavigationService.Navigate(new
Scanner(_selectedCaptureDevice));
}

private void RefreshInterfaces(object? sender, EventArgs e)
{
    if (FindResource("RefreshSpinStoryboard") is Storyboard
storyboard)
    {
        storyboard.Begin();
    }
}

```

```

    }

    LoadCaptureInterfaces();
}

private void OnDeviceSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    StartScanButton.IsEnabled = DeviceList.SelectedItem != null;
}

private void OpenPcap_Click(object sender, RoutedEventArgs e)
{
    OpenFileDialog openFileDialog = new OpenFileDialog
    {
        InitialDirectory =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
        Filter = "PCAP files
(*.pcap;*.pcapng)|*.pcap;*.pcapng|All files (*.*)|*.*",
        RestoreDirectory = true
    };

    if (openFileDialog.ShowDialog() == true)
    {
        var packets =
PcapHandler.LoadFromPcap(openFileDialog.FileName);
        NavigationService.Navigate(new Scanner(packets));
    }
}
}
}

```

קבצי קוד - צד שרת ScannerServer

MainWindow.xaml.cs

```
using System;
using System.Collections.Concurrent;
using System.Collections.ObjectModel;
using System.Linq;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Security.Cryptography;
using System.IO;

namespace ScannerServer
{
    public partial class MainWindow : Window
    {
        // Thread-safe dictionary to quickly find agents by their IP
        private ConcurrentDictionary<string, ConnectedAgent> _agentMap =
        new ConcurrentDictionary<string, ConnectedAgent>();
        public ObservableCollection<ConnectedAgent> Agents { get; set; }
        = new ObservableCollection<ConnectedAgent>();

        // של השרת שמחזיק את המפתח הפרטי והפומבי RSA-אובייקט ה
        private RSACryptoServiceProvider _rsaServer;

        public MainWindow()
        {
            InitializeComponent();
            AgentsList.ItemsSource = Agents;

            // ייצור מפתח 2048 ביט חדש כשהשרת עולה
            _rsaServer = new RSACryptoServiceProvider(2048);

            Task.Run(() => StartListening());
        }

        private async Task StartListening()
        {
            TcpListener listener = new TcpListener(IPAddress.Any, 8888);
```

```

listener.Start();

while (true)
{
    TcpClient client = await
listener.AcceptTcpClientAsync();
    string agentIp =
((IPEndPoint)client.Client.RemoteEndPoint).Address.ToString();

    // Get existing agent or create a new one
    var agent = _agentMap.GetOrAdd(agentIp, (ip) => {
        var newAgent = new ConnectedAgent { IP = ip };
        Dispatcher.Invoke(() => Agents.Add(newAgent));
        return newAgent;
    });

    _ = HandleAgentMessages(client, agent);
}

private async Task HandleAgentMessages(TcpClient client,
ConnectedAgent agent)
{
    try
    {
        using (var stream = client.GetStream())
        using (var reader = new StreamReader(stream,
Encoding.UTF8))
        using (var writer = new StreamWriter(stream,
Encoding.UTF8) { AutoFlush = true })
        {
            // 1. שליחת המפתח הפומבי ללקוח (false = פומבי)
            string publicKeyXml = _rsaServer.ToXmlString(false);
            await writer.WriteLineAsync(publicKeyXml);

            // 2. לולאת האזנה להודעות מוצפנות מהלקוח
            while (true)
            {
                // קריאת ההודעה המוצפנת (פורמט Base64)
                string encryptedBase64 = await
reader.ReadLineAsync();
                if (encryptedBase64 == null) break; // הלקוח
התנתק

                try
                {

```

```

        // 3. פענוח ההודעה
        byte[] encryptedBytes =
Convert.FromBase64String(encryptedBase64);
        byte[] decryptedBytes =
_rsaServer.Decrypt(encryptedBytes, false);
        string decryptedMessage =
Encoding.UTF8.GetString(decryptedBytes);

        // 4. עיבוד ההודעה הגלויה
        ProcessMessage(agent, decryptedMessage);
    }
    catch (Exception ex)
    {
        Dispatcher.Invoke(() =>
agent.AgentAlerts.Insert(0, $"[!] Decryption Error: {ex.Message}"));
    }
}

}
}
catch { }
finally
{
    Dispatcher.Invoke(() => agent.Status = "Disconnected");
    client.Close();
}
}

private void ProcessMessage(ConnectedAgent agent, string
message)
{
    var parts = message.Split('|');
    if (parts.Length < 2) return;

    string type = parts[0];
    string value = parts[1];

    Dispatcher.Invoke(() =>
    {
        switch (type)
        {
            case "ALERT":
                if (parts.Length >= 3)
                {
                    agent.AgentAlerts.Insert(0,
$" [{DateTime.Now:HH:mm}] {value}: {parts[2]}");
                    agent.Status = "ANOMALY DETECTED!"; // שינינו

```

לקראת התיק ל-Anomaly ל-Attack מ

```

    }
    break;

    case "STATS":
        if (int.TryParse(value, out int count))
            agent.TotalPackets = count;
        break;

    case "HEARTBEAT":
        agent.Status = "Online / Scanning";
        break;

    case "PROTOCOL_STATS":
        // Format: PROTOCOL_STATS|TCP:10,UDP:5,HTTP:2...
        var protocolData = value.Split(',');
        foreach (var p in protocolData)
        {
            var kv = p.Split(':');
            if (kv.Length == 2)
            {
                string protoName = kv[0];
                if (int.TryParse(kv[1], out int pCount))
                {
                    var existing =
agent.Protocols.FirstOrDefault(x => x.Name == protoName);
                    if (existing != null)
                        existing.Count = pCount;
                    else
                        agent.Protocols.Add(new
ProtocolStat { Name = protoName, Count = pCount });
                }
            }
        }
        break;
    }
}
});
}
}
}

```

ConnectedAgent.cs

```
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace ScannerServer
{
    // (והכמות שלו TCP מחלקה חדשה שמייצגת פרוטוקול בודד) (למשל)
    public class ProtocolStat : INotifyPropertyChanged
    {
        private string _name;
        private int _count;

        public string Name { get => _name; set { _name = value;
OnPropertyChanged(); } }
        public int Count { get => _count; set { _count = value;
OnPropertyChanged(); } }

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name
= null) => PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
    }

    public class ConnectedAgent : INotifyPropertyChanged
    {
        private string _ip;
        private int _totalPackets;
        private string _status = "Connected";

        public string IP { get => _ip; set { _ip = value;
OnPropertyChanged(); } }
        public int TotalPackets { get => _totalPackets; set {
_totalPackets = value; OnPropertyChanged(); } }
        public string Status { get => _status; set { _status = value;
OnPropertyChanged(); } }

        public ObservableCollection<string> AgentAlerts { get; set; } =
new ObservableCollection<string>();

        // הרשימה החדשה שתחזיק את התגיות של הפרוטוקולים
        public ObservableCollection<ProtocolStat> Protocols { get; set;
} = new ObservableCollection<ProtocolStat>();

        public event PropertyChangedEventHandler PropertyChanged;
```

```

        protected void OnPropertyChanged([CallerMemberName] string name
= null)
        {
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
        }
    }
}

```